

Playing Dice with the Universe: Monte Carlo and Quantum Mechanics in Geochemistry

Victor W. Sletten

Contents

1	Introduction	1
1.1	Motivation	1
1.2	General Objectives	3
1.3	Specific Objectives and Contributions	4
1.3.1	Monte Carlo Simulation	4
1.3.2	Density Functional Calculations	5
1.4	Research Outline	6
1.5	Previous and Related Work	8
1.5.1	Monte Carlo Simulations	8
1.5.2	Organic Acid - Aluminosilicate Interactions	10
1.6	Overview of Remaining Chapters	13
1.6.1	Chapter 2	13
1.6.2	Chapter 3	13
1.6.3	Chapter 4	13
1.6.4	Chapter 5	14
2	Description of the Monte Carlo Model	15
2.1	Introduction	15
2.2	Kinetic Monte Carlo Model	16
2.3	Detailed Reaction Mechanism	19
2.4	Kinetic Parameters	21
2.5	Implementation Details	26
2.5.1	Structural Representation	26
2.5.2	Chemical Environment	27
2.5.3	Unattached Clusters	28
2.6	Adsorption/Desorption Parameters	29

3	Monte Carlo Simulation Results	30
3.1	Introduction	30
3.2	Choice of Kinetic Parameters	30
3.3	Precision Analysis	30
3.4	Rate vs. ΔG	30
3.5	Overall Activation Energy	31
3.6	Surface Orientation	31
3.7	Summary and Conclusions	31
4	Organic Acids and Aluminosilicates I: Reactions	32
4.1	Introduction	32
4.2	Procedures	33
4.2.1	Computational Method	33
4.2.2	Reaction Model	35
4.3	Results and Discussion	41
4.3.1	Dimer Structure	41
4.3.2	Adsorption	44
4.3.3	Hydrolysis	51
4.4	Summary and Conclusions	56
5	Organic Acids and Aluminosilicates II: Bonding	58
5.1	Introduction	58
5.1.1	Properties of the Electron Density Distribution	58
5.1.2	The Laplacian of the Electron Density	60
5.1.3	Bond Critical Point Properties and Nature of Bonding	62
5.2	H_2O and H_3O^+ Adsorption	63
5.3	Hydrolysis of the AlO_{br} Bond	63
5.3.1	Effect of H_3O^+ Adsorption	63
5.3.2	Effect of Chelating Ring Size	63
5.3.3	Evidence for Local Nature of Bonding	64
5.4	Effect of Chelation on Organic Acid Anions	64
5.4.1	Anion Total Strain Energy	64
5.4.2	Bonding in the Anion	64
5.5	Conclusions	64
A	Monte Carlo Model Source Code	65
A.1	actions.h	65
A.2	actions.c	65

A.3	bfsearch.h	67
A.4	bfsearch.c	67
A.5	common.h	69
A.6	envrn.h	72
A.7	envrn.c	73
A.8	evtlist.h	77
A.9	evtlist.c	78
A.10	futil.h	79
A.11	futil.c	80
A.12	lattice.h	80
A.13	lattice.c	82
A.14	myerr.h	87
A.15	myerr.c	87
A.16	output.h	88
A.17	output.c	88
A.18	ran2.h	93
A.19	ran2.c	93
A.20	reactions.h	94
A.21	reactions.c	95
A.22	rxnlist.h	102
A.23	rxnlist.c	102
A.24	sim.h	104
A.25	sim.c	105
A.26	ucell.h	105
A.27	ucell.c	106
A.28	mckaol.c	108
A.29	Makefile	109

B Minimum Energy Geometries 110

List of Figures

4.1	$H_6SiAlO_7^-$ structure	42
4.2	H_7SiAlO_7 structure	43
5.1	$H_6SiAlO_7^-$ electron density	59
5.2	$H_6SiAlO_7^-$ Laplacian of the electron density	61
B.1	$H_6SiAlO_7^- \cdot H_2O$	111
B.2	$H_6SiAlO_7^- \cdot H_3O^+$	111
B.3	$H_6C_2SiAlO_{11}^{-3} \cdot H_2O$	112
B.4	$H_6C_2SiAlO_{11}^{-2} \cdot H_3O^+$	113
B.5	$H_8C_3SiAlO_{11}^{-3} \cdot H_2O$	114
B.6	$H_8C_3SiAlO_{11}^{-2} \cdot H_3O^+$	115
B.7	$H_{10}C_4SiAlO_{11}^{-3} \cdot H_2O$	116
B.8	$H_{11}C_3SiAlO_{11}^{-2} \cdot H_3O^+$	117

List of Tables

2.1	Kaolinite hydrolysis reactions	21
4.1	Geometry of $H_6SiAlO_7^-$ and H_7SiAlO_7	43
4.2	Adsorption energies: water and hydronium	44
4.3	Effects of adsorption on bond lengths	45
4.4	Adsorption energies: oxalate	48
4.5	Adsorption energies: malonate	48
4.6	Adsorption energies: succinate	48
4.7	Hydrolysis reaction energies: no organic	51
4.8	Hydrolysis reaction energies: oxalate	53
4.9	Hydrolysis reaction energies: malonate	53
4.10	Hydrolysis reaction energies: succinate	53
4.11	Overall hydrolysis reaction energies	54

Chapter 1

Introduction

1.1 Motivation

Understanding the chemical reactions and characterizing the chemical properties of natural chemical systems is the mission of geochemistry. The size of these systems ranges from the global at one extreme to the atomic at the other. For example, the global cycling of elements encompasses numerous smaller scale processes such as weathering of minerals, metamorphism and mass transport by crustal fluids, and atmospheric reactions. Mineral weathering, in turn, is a manifestation of the processes of mineral growth and dissolution. The atomic-scale interactions occurring at the mineral-water interface, taken together, are the microscopic foundation of the macroscopic expression of mineral growth and dissolution. Therefore, a complete understanding of global geochemical cycles requires a knowledge of the fundamental atomic processes occurring at the mineral-water interface in natural systems.

Natural geochemical systems are extremely complex. Even under equilibrium conditions, the number of chemical and physical variables is quite daunting for all but the simplest natural chemical systems. Removing the constraint of equilibrium and including a kinetic treatment of the chemical reactions precludes the complete characterization of even relatively simple natural systems. Even then, extrapolation to other physical and/or chemical conditions is problematic.

Fortunately, the constraints imposed in laboratory experiments enable the experimentalist to extract information of fundamental importance to natural geochemical systems by fixing some and varying other physical and chemical

properties. However, even the most carefully controlled experiments contain a number of significant sources of uncertainty. The inability to perfectly characterize and control, without unduly influencing, the chemical and physical environment during the course of the experiment are the main sources of uncertainty. If two variables change during an experiment, which is responsible for the observed effect?

The answer to this question lies in the application of a theoretical context by the experimentalist to the design and interpretation of experiments. While theory is always used to some degree to guide the experimentalist, it can also be used to provide a framework in which to construct the perfect experiment. Calculations based on pure theory provide completely characterized and controllable systems. Unfortunately, analytical solutions to the equations provided by theory are unavailable in most situations for all but the most trivial systems. Therefore, numerical simulations are usually necessary to obtain the desired results. Numerical simulations introduce errors (e.g., truncation and rounding), but preserve the characterization and control properties of the systems. In addition, despite the recent explosion of readily available computing power, numerical techniques can be applied only to relatively simple systems.

Even so, carefully constructed numerical simulations, based on a sound theoretical foundation, are capable of providing important insights into fundamental aspects of the most complex natural systems. For example, today the pharmaceutical industry relies heavily on quantum mechanical, molecular dynamic, and Monte Carlo calculations to identify likely candidates from the thousands of possibilities when developing new drugs. So long as one remains mindful of the current limits of applying theory to natural systems, and also of the current limits of experiment in understanding natural systems, the two approaches provide excellent complements in gaining a complete understanding of natural systems.

With these considerations in mind, this dissertation presents an attempt to apply two important theoretical techniques to problems of fundamental importance in certain geochemical systems. The first, borrowed from the field of statistical physics, is the kinetic Monte Carlo simulation. The second, borrowed from theoretical physics and physical chemistry, is density functional theory, which comprises an alternative formulation of the Schrödinger equation, thereby simplifying the computations necessary to obtain approximate solutions.

1.2 General Objectives

In this dissertation, a full model of the detailed growth and dissolution mechanism for a real aluminosilicate mineral structure, kaolinite, is developed. One of the most important motivations driving this study is the ability to incorporate kinetic information obtained from *ab-initio* quantum mechanical calculations into a framework which provides insight into the role of the elementary reactions amenable to quantum mechanical calculations in the overall macroscopic processes of mineral growth and dissolution. Another important motivation is to develop a tool which can be used to study mineral growth and dissolution under conditions that are particularly difficult for the experimentalist, i.e., near equilibrium conditions.

The second half of this dissertation consists of a detailed quantum mechanical investigation of the role of organic acids in aluminosilicate dissolution and weathering. It is difficult to construct controlled experiments to assess the importance of organic acids, in particular oxalic acid, to aluminosilicate dissolution, although valiant efforts have been made (Pittman and Lewan, 1994, and references therein). The approach taken in the present work is to propose a detailed reaction mechanism, guided by previous experimental results, by which organic acid anions may affect aluminosilicate dissolution, and then to evaluate that mechanism by calculating reaction energies and investigating the nature of bonding in the various reactants and products using properties of the electron density distribution (Bader, 1990). This approach provides the opportunity to investigate directly processes occurring at the mineral-fluid interface, without introducing the complexity of secondary precipitation and complexation of organic anions with aluminum in solution, for example. While these complexities will surely be present in natural systems, to borrow an analogy from linear algebra, to fully describe a space, it is necessary to specify a set of orthogonal vectors which span the space. That is to say, in order to understand the full complexity of a natural system, and more importantly to be able to extrapolate to other similar systems, it is essential to possess a thorough understanding of the elementary processes occurring in the system.

1.3 Specific Objectives and Contributions

1.3.1 Monte Carlo Simulation

The first step in developing a detailed computer simulation is to translate the complex dynamics of the full detailed reaction mechanism into a set of mathematical algorithms, and to find efficient ways to implement those algorithms on a computer. In addition, it is necessary to represent the actual structure of kaolinite and track its evolution through time under varying conditions. This step represents a major effort of this thesis, and it was necessary to apply a number of concepts from the field of theoretical computer science to obtain the solutions. Details of the algorithms used and their implementations are discussed in detail in Chapter 3.

In order to construct a framework in which to assess the significance of quantum mechanical calculations on macroscopic behavior it is necessary that results of those calculations be available. Encompassing a dissertation in themselves, the complete set of results necessary for kaolinite are not yet available, but work on them is in progress (Felipe and Lasaga, 1997). In their absence, certain constraints from thermodynamics may be applied to estimate the necessary parameters, as discussed in Chapter 3.

Once the implementation of the simulation is complete, there are a number of issues in kaolinite growth and dissolution which can be addressed. The most important macroscopic properties which can be varied in these simulations are temperature and solution saturation state. Therefore, the functional dependence of the rates of growth and dissolution on saturation state, especially near equilibrium where it has proven difficult to devise accurate experiments (Nagy et al., 1991; Ganor et al., 1995), have been investigated. Since the detailed atomic structure of kaolinite is included explicitly in the simulation, it is also possible to test whether different kaolinite edges display different rates of growth and dissolution under the same conditions. Finally, the kinetic Monte Carlo simulation also provides the opportunity to explore the effect of variations of the kinetic parameters for the elementary reactions on the overall macroscopic behavior. More specifically, this includes the possible effects of local chemical environment on macroscopic growth and dissolution, i.e., collective phenomena arising from complex systems. Collective phenomena of this nature include focusing of growth or dissolution in certain areas, as a result of positive (or negative) feedback provided by local chemical environment only, and not external forcing such as expected by

defects. These applications are addressed in Chapter 4.

Ultimately, the potential applications of and extensions to the model are limited only by the imagination of the practitioner, the availability of necessary physical chemical data, and computing resources. For example, this model could easily be adapted to evaluate the importance of initial surface topography on subsequent dissolution and growth behavior, particularly in the context of collective phenomena. The necessary code is already present to include surface diffusion and adsorption of cations to the “wrong” sites. However, due to a lack of availability of theoretical or experimental data on some of the necessary kinetic parameters, this feature is not used in the present study. Ultimately, catalytic and inhibitory effects, for example of organic acids, could be incorporated into the simulation framework with minimal effort. All of the above provide fertile ground for future work.

1.3.2 Density Functional Calculations

Low molecular weight dicarboxylic organic acids, such as oxalic acid, have been found to accelerate the rate of aluminosilicate mineral dissolution under some experimental and field conditions. Hypotheses have been formulated to explain the source of this effect (e.g., pH effects, ligation of aluminum by the acid anion), but they are difficult to test experimentally, due to the inability to control or characterize completely the experimental conditions. One of the primary objectives of this dissertation is to provide some fundamental insight into the nature of the effect of a model dicarboxylic organic acid, oxalic acid, on the dissolution of an aluminosilicate mineral using the tools of quantum mechanics and theoretical physical chemistry. This understanding consists largely of the energetics of potential reactions occurring at the surface between the aluminosilicate and the acid anion, and the consequences to the geometry and bonding of the aluminosilicate structure.

When dicarboxylic organic acid anions form bidentate chelates with an aluminum atom, the result is a ring structure whose members include the aluminum, the two oxygen atoms bonded to the aluminum to form the chelate, and two or more carbon atoms comprising the backbone of the organic ligand. For oxalate, the resulting ring has five members, for succinate it is seven. Furrer and Stumm (1986) found a weak dependence of the dissolution rate constant on the character of the organic acid ligand. More specifically, they found that the rate constant was inversely proportional to the size of the resultant ring. In other words, oxalic acid (five member) produced the largest

dissolution rate constant, while succinic acid (seven member) produced the smallest, with malonic acid (six member) producing an intermediate rate constant. Understanding the fundamental cause of this effect, and its possible implications for other ligands, is the second major objective of the quantum mechanical investigation. This understanding extends not only to the energetics of the relevant reactions, but also to any changes in the character and strength of the bonding within the aluminosilicate as a function of ring size. To understand changes in the bond characteristics, the theoretical framework developed by Bader (1990) was applied, which interprets bonding in terms of local properties of the charge distribution.

1.4 Research Outline

The research plan for the Monte Carlo simulation follows very closely the specific objectives outlined in the previous section. First, a simulation which includes the full reaction mechanism, that is to say both growth and dissolution, has been built on the basis of the model presented by Lasaga (1995) . Since Lasaga's (1995) model was only designed to simulate dissolution, significant modification was required to include condensation and impingement reactions. Also, the new model incorporates surface diffusion.

Testing and debugging of the final version can be accomplished by long-term simulations under a variety of conditions and judicious use of assertion statements to check invariant conditions. Also, sanity checks were performed to ensure that the model is behaving correctly in a general sense, i.e., dissolving at undersaturated conditions and growing at supersaturated conditions. Tests of choices for the tunable parameters were also subjected to the same kind of sanity checks. As with any computer program of this complexity, the best check is performance against *a-priori* knowledge, derived from experimental observations, but this is not yet possible in this case, since the fundamental kinetic parameters for the elementary reactions are not yet available from theory or experiment.

Ideal simulation conditions are determined by running simulations of various lattice sizes and durations to determine, the computationally and statistically optimal choice of simulation time and lattice size. There is a tradeoff between simulation time, lattice size, and statistical fluctuations in the results. Larger lattices produce statistically superior results, but each step requires much more computational effort and significantly more memory. Due

to initial transient effects, simulation time must be long enough to capture the steady-state behavior, yet short enough to be computationally feasible.

To explore the functional dependence of rate versus saturation state, the chemical potentials of silicon and aluminum are varied systematically, while holding all other parameters constant. To compute an overall activation energy, temperature is varied. Activation energies for a few different saturation states have been computed by simply changing the chemical potentials to new values and varying temperature at these new values. Exploring the effect of surface orientation is simplified by the fact that the model contains the necessary code to arbitrarily choose one of two surface planes (100 and 010) to start the simulation. Finally, the collective phenomena have been investigated by arbitrarily and systematically changing the parameters representing the effect of local chemical environment on the elementary rates.

To investigate the effects of organic acids on aluminosilicate dissolution, an outline of a proposed reaction mechanism for each of the organic acid anions and for protonated (low pH conditions) and unprotonated (neutral pH conditions) bridging oxygens is necessary (see Chapter 5). Next, the minimum energy geometries are obtained for each of the reactant and product species in all of the reactions. Frequency calculations are performed at a higher level of theory on each of the minimum energy geometries to obtain more accurate estimates of the energies of each species. In anticipation of the bonding analysis, the complete wavefunction are simulataneously computed and output at this time. Reaction energies are calculated by subtracting the sum of the reactant species' energies from the sum of the product species' energies. Analysis of the geometric and energetic data produced can and did reveal dead-end reactions (i.e., energetically unfavorable), as well as unanticipated configurations which led to new reaction paths that were subsequently explored.

Analysis of the bonding within the aluminosilicate and the organic acid anions begins with calculating grids of the charge density and its laplacian for each species of interest. Bond critical points can then be found from analysis of the wavefuntion. The search for bond critical points can obviously be restricted to regions between the atoms presumed to be bonded to one another. Once located, the desired properties of the local charge distribution at the critical point are calculated. These results were tabulated and graphically displayed for interpretation and to suggest other interesting bond pairs to study.

1.5 Previous and Related Work

1.5.1 Monte Carlo Simulations

The Monte Carlo simulation method has enjoyed a rich history in numerous scientific disciplines, ranging from statistical physics to molecular biology. However, few serious attempts have been made to apply the technique to problems of interest to geochemists. This section presents a brief introduction to the method, and a survey of its applications in geochemistry and closely related fields.

Introduction to Monte Carlo

The Monte Carlo model was developed to solve the statistical physical problem of computing some average macroscopic property for a system with many degrees of freedom and a known Hamiltonian. Typically, analytical solutions are unavailable, so a form of numerical integration is needed. Due to the high dimension of the phase space of these types of problems, regular grid approaches are often computationally unwieldy. A better approach is to sample phase space at random, summing the points in the usual manner (Binder and Heermann, 1988). If the random sample is unweighted, it is called *simple* sampling.

For thermal averages, the probability distribution of energy is very localized in phase space, and not necessarily centered where the Gaussian distribution implied by simple sampling will be centered. This complicates the estimation of the thermal average by requiring the inclusion of a Boltzmann factor in each term of the summation. The Boltzmann factor can be eliminated by weighting the sample set itself by a Boltzmann factor (Binder and Heermann, 1988).

To implement this sampling scheme, Metropolis et al. (1953) introduced a method of *importance* sampling in which successive samples are determined via a Markov process, rather than independently. The transition between states (samples) typically corresponds to choosing one or more degrees of freedom and varying them by some small, often random, factor. The new state is then either rejected or accepted for inclusion in the ensemble with some transition probability, which usually depends on the change in energy relative to the previous state. By carefully constructing the transition probability, the limiting distribution of states approaches the equilibrium distri-

bution. This technique is known as *Metropolis* sampling.

A simple application is illustrated by the Ising spin model. This model is typically represented as a square grid, where each box has a discrete spin, either “up” or “down”. There is some known Hamiltonian which describes the energetics of the interactions between neighboring boxes, in terms of relative spins. A simple form of Metropolis sampling is to start with some distribution of spins, and successively attempt to flip spins in boxes chosen at random or systematically. A flip is automatically accepted if it results in a lower energy configuration. A spin resulting in a higher energy is accepted with some probability proportional to a Boltzmann factor involving the resulting change in energy. One can then look at average ensemble properties, such as the total magnetization, or follow the simulation dynamically to learn about the kinetics of the system.

Previous Simulations of Minerals and Crystals

Applications of the Monte Carlo method to mineral growth and dissolution are typically concerned with the time-dependent macroscopic behavior of the system, rather than ensemble averages, and derive from the subset of Monte Carlo methods known as *kinetic* Monte Carlo. In a typical kinetic Monte Carlo model, rates of various processes of interest, e.g., chemical reactions, are converted to probabilities. The state transitions are the transitions from reactants to products. In this manner, it is possible to compute the dynamics of arbitrarily complex systems numerically.

Gilmer (1980) adapted the kinetic Ising model (Huang, 1963, for overview) to crystal growth for a simple cubic structure. The model incorporated impingement and evaporation of atoms at the surface, and included impurity and dislocation effects on these processes. Despite the simplicity of the model, it captured the essence of the effects of surface roughening, dislocations, and impurities. In addition, this work pointed out the importance of transient effects in understanding surface kinetics.

The work of Gilmer (1980) was expanded by Blum and Lasaga (1987) to study dissolution, with the inclusion of surface diffusion. They studied in detail the quantitative form of the macroscopic dissolution rate law as well as the relationship between dislocations and etch-pit morphology, and roles of dissolution in defect-free regions versus at dislocations.

Despite the successes of these early models, they lacked a direct link to natural mineral structures in that they simulated only cubic structures with

model “atoms” and arbitrary kinetic parameters. Lasaga (1995) provided this link with an extension of the kinetic Monte Carlo model to kaolinite. Kaolinite is a convenient choice because although an important aluminosilicate, its sheet structure is relatively easy to discretize, assuming no interaction between the sheets. The work of Lasaga (1995) included the full details of the atomic structure of kaolinite, but was limited to hydrolysis and evaporation reactions. Since impingement and condensation reactions were not yet included, the model was unable to explore the relationship between dissolution rate and solution saturation state. However, it was significant as a departure from the previous cubic structures and simple reaction mechanisms, and provides a framework in which to assess the relative importance of some elementary processes, namely the various hydrolysis reactions, at far from equilibrium conditions.

Recent Simulations of Other Materials

Coincident with the development of Lasaga’s (1995) model, and continuing to the present, researchers (Battaile et al., 1997; Battaile, 1998; Battaile et al., 1998; Paritosh et al., 1999; Battaile et al., 1999; Dawnkaski et al., 1995; Dawnkaski et al., 1996) in the general field of material science have been working on simulations of diamond thin film vapor deposition, incorporating the actual diamond structure and a full suite of elementary reactions. These studies benefit from a more complete set of kinetic data, obtained from experiment and theory, for the elementary reactions which comprise the deposition mechanism than is available for the elementary reactions involved in aluminosilicate growth and dissolution. Conceptually, these models are quite similar to the present model.

1.5.2 Organic Acid - Aluminosilicate Interactions

Geochemists have recognized for some time the potential importance of organic acids in dissolution reactions. This importance extends to regimes ranging from the influence of biological agents (plants, micro-organisms) on weathering to the generation of secondary porosity in the subsurface and the potential impact on petroleum migration. This section presents a brief review of previous experimental studies related to the role of organic acids in aluminosilicate dissolution, a review of previous studies which used quantum mechanics to study mineral dissolution reactions, and a review of work

which has used the properties of the charge distribution to study bonding in silicates.

Experimental Work

There is a considerable literature related to organic acids and their role in geochemistry. Pittman and Lewan (1994) edited an excellent summary of work to that point on the role of "Organic Acids in Geological Processes". Chapters 6 (Drever and Vance, 1994), 7 (Bennett and Casey, 1994), and 8 (Hajash, 1994) are of particular interest to this study.

Numerous studies have demonstrated that organic acids are capable of complexing and mobilizing certain metals, notably aluminum, in solution and of enhancing the dissolution rates of various aluminosilicate minerals at low temperatures (Huang and Keller, 1970; Huang and Kiang, 1972; Stumm et al., 1985; Mast and Drever, 1987; Amrhein and Suarez, 1988; Chin and Mills, 1991; Amrhein and Suarez, 1992; Ullman et al., 1996; Poulson et al., 1997; Baptist et al., 1998). Others have noted that oxygen bearing functional groups found in humic substances may behave similarly to low molecular weight organic acids (LMWOA) of known composition in complexing polyvalent cations and, thereby, enhancing mineral dissolution (Stevenson and Fitch, 1986; Stevenson and Vance, 1989). In fact, since humic substances are much more common than LMWOA, they may be more important in complexing and mobilizing metals in soils (Tan, 1986; Stevenson and Vance, 1989). However, near plant roots (rhizosphere) and fungal hyphae, LMWOA may be present in significant quantities (Rovira and McDougall, 1967; Rovira and Davey, 1974; Graustein et al., 1977; Fox and Comerford, 1990).

Of the LMWOA's, oxalic acid has been found to be most effective in enhancing mineral dissolution (Stumm and Wieland, 1990), so has been studied the most extensively. The effect of oxalate on feldspar dissolution has been found to depend on the calcium, and therefore aluminum, content of the feldspar (Mast and Drever, 1987; Amrhein and Suarez, 1988; Amrhein and Suarez, 1992, e.g.), with Ca-rich feldspars experiencing a greater effect than Ca-poor feldspars.

The idea that organic acids present in petroleum reservoirs might be at least partly responsible for the generation of secondary porosity in the subsurface, and therefore might shed light on petroleum migration, drove many higher temperature studies (Surdam et al., 1984; Surdam and MacGowan, 1987; MacGowan and Surdam, 1988; Surdam et al., 1989; Bevan and Sav-

age, 1989; Stoessell and Pittman, 1990; Fein, 1991; Franklin et al., 1991; Hajash and Bloom, 1991; Manning et al., 1991; Reed and Hajash, 1992; Huang and Longo, 1992; Manning et al., 1994). This work has demonstrated the efficacy of oxalic acid, in particular, and other carboxylic organic acids in enhancing the dissolution of some aluminosilicate minerals, especially in the pH range 4–6.

***Ab-initio* Calculations**

One of the first references to the use of quantum mechanical calculations in the earth sciences is Tossell (1979). After that, quantum mechanical calculations became quite popular with mineral physicists and scores of references appear in the literature over the two decades following Tossell (1979). For a review of the application of quantum mechanics to geological problems, see Tossell (1992).

A number of studies have used *ab initio* calculations to investigate mineral dissolution reactions, particularly those involving hydrolysis of a bridging *SiOSi* or *SiOAl* oxygen site (Lasaga and Gibbs, 1990a; Lasaga and Gibbs, 1990b; Lasaga, 1992; Xiao and Lasaga, 1994; Xiao and Lasaga, 1996; Kubicki et al., 1996a). These results have generally agreed with available experimental observations. The important point made by these studies is that the *ab initio* approach provides a means to gain a more fundamental understanding of the macroscopic experimental observations. Kubicki et al. (1996) investigated possible structures for acetate-aluminum complexes in solution, and demonstrated that calculated vibrational spectra agreed well with observed vibrational spectra for many of these complexes, lending further support to the accuracy of the calculations. Gibbs et al. (1997a,b) introduced the use of the electron density distribution and its Laplacian (Bader, 1990) to the study of the *SiO* bond, and demonstrated how this approach reproduces many bond-property trends observed by experimentalists and predicted by such venerable yet semi-empirical theories such as VSEPR and Pauling's electronegativity considerations.

1.6 Overview of Remaining Chapters

1.6.1 Chapter 2

A detailed description of the theoretical framework, elementary reaction mechanism, and implementation of the Monte Carlo simulation is presented in Chapter 2. In addition, discussion of alternative methods for calculating the necessary kinetic parameters from both quantum mechanical results and thermodynamic data is presented. Source code for the Monte Carlo simulation is provided in Appendix A.

1.6.2 Chapter 3

Chapter 3 illustrates the utility of the Monte Carlo model in geochemistry. Results for a number of simulations are presented, which address the issues of overall activation energy, dissolution and growth rates as a function of solution saturation state, and the effect of surface orientation on the dissolution and growth rate. Finally, an exploration of the possibility of collective phenomena due to variations in the kinetics arising from local chemical environments is presented.

1.6.3 Chapter 4

Chapter 4 introduces the problem of organic acids and aluminosilicate dissolution. Geometries and reaction energies calculated from quantum mechanics for a proposed mechanism are presented and discussed. These results address the steric and energetic effects and constraints for the catalytic effect observed in experiments and for the dependence of this effect on chelate ring size. The results of the quantum mechanical calculations are shown to be consistent with experimentally observed relationship between chelating ring size and dissolution rate. A detailed analysis of the contributions of the elementary steps to the overall hydrolysis reaction energy is presented. Appendix B contains figures depicting the minimum energy geometries obtained for the various molecules and complexes involved in the reactions.

1.6.4 Chapter 5

Chapter 5 summarizes the relevant material from Bader's (1990) theoretical exploration of the local charge density and its implications for understanding the nature of chemical bonds. This material is then applied to the quantum mechanical results obtained in Chapter 4, in an attempt to explain, at a fundamental level, the nature of the aluminosilicate-organic acid interactions.

Chapter 2

Description of the Monte Carlo Model

2.1 Introduction

The growth and dissolution of a mineral in contact with a fluid phase is the net result of all the atomic interactions which occur between the mineral surface and the fluid. Mineral growth typically occurs due to the adsorption and incorporation, via condensation reactions, of surface complexes into the mineral surface. Likewise, dissolution is the result of the evolution, via hydrolysis and exchange reactions, and desorption of surface complexes. Taken together, the complete set of elementary reactions which occur at the mineral surface constitutes the detailed growth and dissolution mechanism. In order to fully understand the effect of environmental conditions such as fluid composition and temperature on the growth and dissolution rate, a detailed understanding of the elementary interactions is essential.

Despite recent advances in techniques for observing surfaces at near atomic resolution, direct observation of the dynamics of elementary reactions occurring at the mineral surface remains very difficult. Therefore, our knowledge of the nature and rates of these elementary surface reactions currently must be extracted from measurements of the overall dissolution behavior. Most of the experimental results have been interpreted in terms of phenomenological rate laws derived from proposed dissolution mechanisms. Although useful for making general predictions, experimental rate laws do not capture the full complexity of the detailed mechanism, ignoring crucial aspects of mineral

growth and dissolution such as surface morphology.

In the absence of observations, theoretical models can provide insight into the nature and rates of the elementary surface reactions. The increasing availability of computing power is making *ab initio* calculations of the energetics of surface complexes and rate of elementary reactions more feasible. However, the tremendous computational cost of accurate *ab initio* calculations still prohibits their application to the length and time scales of mineral growth and dissolution. Molecular dynamics simulations are capable of accessing the length scales appropriate to mineral surfaces, however they are limited to time scales on the order of nanoseconds. Monte Carlo simulations have been used successfully to investigate growth (Gilmer, 1980, e.g.) and dissolution (Blum and Lasaga, 1987) of simple cubic crystals. More recently, atomic scale kinetic Monte Carlo simulations have been developed to study chemical vapor deposition of diamond (Battaile, 1998).

Atomic scale kinetic Monte Carlo simulations offer a method to evaluate proposed growth and dissolution mechanisms, the ability to assess the significance of *ab initio* calculations on the overall growth and dissolution behavior, and the possibility of investigating phenomena which are not amenable to current experimental techniques. For example, carefully constructed simulations provide the opportunity to measure the ΔG dependence of the growth and dissolution rate. In this chapter, an atomic scale kinetic Monte Carlo model for the growth and dissolution of kaolinite is introduced and described. The model incorporates the atomic structure of kaolinite and is capable of simulating the full detailed reaction mechanism. Although the model has been developed specifically for kaolinite, application to other minerals is straightforward. In the next chapter, simulation results which illustrate the utility of the model in advancing our understanding of many aspects of mineral growth and dissolution are presented.

2.2 Kinetic Monte Carlo Model

In a kinetic Monte Carlo simulation, mineral growth and dissolution are treated as stochastic processes in which the rate constants for individual elementary reactions are taken as probabilities. Previous kinetic Monte Carlo simulations of crystal growth and dissolution (Van der Eerden et al., 1978; Gilmer, 1976; Gilmer, 1977; Gilmer, 1980; Lasaga and Blum, 1986; Blum and Lasaga, 1987) modeled simple structures and a limited number of elementary

processes. Recently, Monte Carlo simulations of crystal growth which include real structures and more detailed reaction mechanisms have begun to appear (Lasaga, 1995; Battaile et al., 1997; Battaile, 1998). The current model is the first applied to an aluminosilicate, and to our knowledge, the first which simulates both growth and dissolution.

In order to simulate the kinetics of mineral growth and dissolution at the atomic scale, the occupancies of all atomic sites on the mineral surface must be explicitly maintained. To achieve this, we employ a rigid lattice approximation, thus ignoring atomic relaxation and vibration. The rigid lattice approximation allows us to treat each atom as having a finite number of states corresponding to its status in terms of the detailed reaction mechanism, rather than a continuous set of states corresponding to smoothly varying atomic positions as would be the case for molecular dynamics and classical Monte Carlo techniques. For example, a Si – O – Si oxygen can be in the following states: (1) hydrolysis reactant (condensation product); (2) hydrolysis product (condensation reactant); (3) terminal siloxane; (4) unoccupied. Since there are a finite number of sites and a finite number of reactions, there are a finite number of possible configurations. Details of this approximation will be discussed in Subsection 2.5.1.

Since the configuration space is finite, we can enumerate all possible events in the system at each time step. The probability of a given event at a given site is determined by the rate constant for the corresponding reactions in relation to the rate constants for all other events possible at that time step. At each time step, an event m is randomly chosen from the M events possible at that time step:

$$\frac{\sum_{i=0}^{m-1} k_i}{\sum_{i=0}^M k_i} < \xi_1 \leq \frac{\sum_{i=0}^m k_i}{\sum_{i=0}^M k_i} \quad (2.1)$$

where ξ_1 is a random number in the interval $(0, 1)$ and k_i is the rate constant for event i . To execute the event, we simply change the state of the site from reactants to products, updating the states of neighboring sites if necessary. After choosing and executing an event m , the list of M possible events is updated and the process repeated.

For simulations involving processes which operate at very different time scales, such as mineral growth and dissolution, use of a fixed-length time step is problematic. If the time step is small enough to accommodate the fastest processes, then randomly choosing sites will almost always result in

null actions at sites where only slow processes are possible. As a result, the simulation effectively is unable to access the longer timescale of the slower processes. On the other hand, choosing a time step appropriate for the slower processes under-represents the importance of the faster processes. In order to avoid the problems associated with multiple timescales, we employ an algorithm similar to the N-fold way method (Bortz et al., 1975; Hassold and Holm, 1993; Battaile et al., 1997). In this approach, one event occurs at each time step and the length of the time step is dynamically and stochastically adjusted according to:

$$dt = -\frac{\ln(\xi_2)}{\sum_{i=1}^M k_i} \quad (2.2)$$

where ξ_2 is a random number in the range $(0, 1)$, and the denominator is the sum of the rates for all M events which can occur at that simulation step. This method has been shown to be statistically equivalent to fixed time-step methods (Hassold and Holm, 1993), while allowing the simulation of events with widely disparate time scales.

The expression for calculating the time step illustrates that the unit of time depends inversely on the preexponential part of the rate constants. Since magnitudes of the rate constants for elementary reactions are not generally available, we chose to assign magnitudes based on assumptions of the relative rates for the processes. In this scheme, the unit of time is arbitrary and scaling simulation time to real time requires comparison with experimental results. Assuming that the relative magnitudes of the rate constants are correct, simulation time scales linearly to real time.

Another issue which must be addressed is how to determine which reactions are enabled at each site for any given simulation step. It would not be desirable, for example, to allow hydrolysis reactions in the interior of the mineral, as this is not likely to occur in nature. We employ a set of simple criteria to determine which actions are enabled at a site. Hydrolysis reactions may only occur at surface bridging oxygen sites. The rationale for this is simply that hydrolysis requires a water molecule, expected to be present with significant probability only at the interface. The criteria for a bridging oxygen site to be a surface site is that one of the cations bridged by the oxygen must have a terminal hydroxyl attached. Since both reactants are already present, condensation reactions are allowed at any site which is in the hydrolyzed state. Adsorption is allowed only to cation sites which neighbor at least one occupied oxygen site. Desorption and surface diffusion are

allowed only at sites which have at least one vacant neighbor. Desorption from completely occluded sites would require a diffusion-like jump through the interlayer. Our exclusion of desorption is not intended to imply that diffusion through the interlayer never occurs, just that its frequency is not sufficient to justify the extra layer of complication it would introduce. Diffusion at a completely occluded site does not make sense in our model, since we are only simulating a single kaolinite sheet.

2.3 Detailed Reaction Mechanism

In order to simulate growth and dissolution for a particular mineral, a detailed reaction mechanism must be developed which incorporates the atomic structure of the mineral. A number of reaction mechanisms have been postulated for kaolinite dissolution (Carroll and Walther, 1990; Chin and Mills, 1991; Wieland and Stumm, 1992; Ganor et al., 1995; Devidal et al., 1997; Huertas et al., 1998). Wieland and Stumm (1992) outlined a mechanism which involves protonation of OH groups at the surface and rate-limiting detachment of a precursor Al complex which involves breaking Al – O – Si linkages. This is accompanied by fast hydrolysis of the tetrahedral Si. Devidal et al. (1997) suggested two similar mechanisms, one for acidic conditions and one for alkaline conditions. The low-pH mechanism includes reversible exchange of H^+ with Al^{3+} which involves breaking the Al – O – Si linkages to form an Al-deficient precursor. The rate-limiting step is the detachment of the precursor by irreversible hydrolysis of the Si – O – Si bonds. The high-pH mechanism includes reversible exchange of OH^- with Al^{3+} by breaking the Al – O – Si linkages to produce a siloxane layer, followed by reversible decondensation of the siloxane groups to form a silica rich precursor complex which then detaches from the kaolinite surface. A third mechanism is provided by Ganor et al. (1995). This mechanism includes the sequential hydrolysis of the bridging oxygen linkages between a Si or Al and the mineral surface. When all bridging oxygens have been hydrolyzed, the H_4SiO_4 or Al^{3+} desorbs into the solution.

We chose the mechanism proposed by Ganor et al. (1995) for our model for three reasons. First and most importantly, we believe that this mechanism is the most likely to represent the true reactions occurring at the kaolinite surface. Second, the mechanism contains a full set of explicit reactions which is essential for development of an atomic-scale simulation. Third, all of the

steps in the mechanism are elementary reactions, which means that they are all reversible and that the rate constants can be obtained from *ab initio* calculations and transition-state theory (TST). Reversibility is important because we are interested in simulating dissolution *and* growth over a wide range of ΔG .

The mechanism of Ganor et al. (1995) as incorporated in our model consists of two main classes of reactions: those which break and form bonds (hydrolysis and condensation reactions) and those which involve the movement of complexes at the surface (adsorption and desorption). In the remainder of this section, the reactions in each class will be discussed in detail, followed by a discussion of the relative rates for the various reactions and the factors which influence them.

Growth and dissolution of kaolinite is achieved by the addition, removal, and evolution of Si and Al complexes at the mineral surface. In the absence of other ligands, these surface complexes consist of a Si or Al with zero or more bonds to the bulk mineral via bridging oxygens and coordinating hydroxyls or waters. For example, a Si surface complex would have the stoichiometry: $\text{SiO}_m(\text{OH})_{4-m}$, where O_m represents m linkages to the bulk mineral via bridging oxygens and $(\text{OH})_{4-m}$ indicates that the remaining bridging oxygen linkages have been hydrolyzed and replaced with hydroxyls. The addition and removal of surface complexes is achieved by adsorption and desorption reactions, respectively. The evolution of Si and Al surface complexes results from the breaking and forming of these bridging oxygen linkages which occurs by hydrolysis and condensation reactions, respectively.

The number of hydrolysis and condensation reactions is determined by the kaolinite structure. A kaolinite sheet consists of a layer of Si tetrahedrally coordinated by oxygens joined to a layer of Al octahedrally coordinated by oxygens. In the tetrahedral layer, two Si atoms are linked to one another by a single Si – O – Si bridging oxygen. The two layers are linked together by the Si – O – Al₂ bridging oxygens. In the octahedral layer, two Al atoms are linked to one another either by a pair of Al – OH – Al bridging hydroxyls or by one Al – OH – Al bridging hydroxyl and one Si – O – Al₂ bridging oxygen. A separate hydrolysis reaction with a unique rate constant can be written for each of the three types of bridging oxygen. The condensation reactions are simply the reverse of the hydrolysis reactions. A complete set of hydrolysis reactions is given in Table 1. Note that reactions 6 and 7 are expected to have very similar rates, but are included separately for completeness.

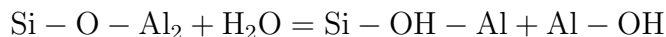
1	$\text{Si} - \text{O} - \text{Si} + \text{H}_2\text{O} = \text{Si} - \text{OH} + \text{Si} - \text{OH}$
2	$\text{Al} - \text{OH} - \text{Al} + \text{H}_2\text{O} = \text{Al} - \text{OH} + \text{Al} - \text{OH}_2$
3	$\text{Si} - \text{O} - \text{Al}_2 + \text{H}_2\text{O} = \text{Si} - \text{OH} + \text{Al} - \text{OH} - \text{Al}$
4	$\text{Si} - \text{OH} + \text{Al} - \text{OH} - \text{Al} + \text{H}_2\text{O} = \text{Si} - \text{OH} + \text{Al} - \text{OH} + \text{Al} - \text{OH}_2$
5	$\text{Si} - \text{O} - \text{Al}_2 + \text{H}_2\text{O} = \text{Si} - \text{OH} - \text{Al} + \text{Al} - \text{OH}$
6	$\text{Si} - \text{OH} - \text{Al} + \text{Al} - \text{OH} + \text{H}_2\text{O} = \text{Si} - \text{OH} + \text{Al} - \text{OH} + \text{Al} - \text{OH}_2$
7	$\text{Si} - \text{OH} - \text{Al} + \text{H}_2\text{O} = \text{Si} - \text{OH} + \text{Al} - \text{OH}_2$

Table 2.1: Hydrolysis Reactions

While a single pathway exists for each of the $\text{Si} - \text{O} - \text{Si}$ and $\text{Al} - \text{OH} - \text{Al}$ hydrolysis reactions, hydrolysis of the $\text{Si} - \text{O} - \text{Al}_2$ bridging oxygen can follow several different pathways, so multiple reactions must be written. For example, initial hydrolysis may result in a break in the link between the octahedral and tetrahedral layers:



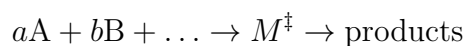
Alternatively, the initial hydrolysis might result in a break in the link between two Al's:



Each of these pathways is treated as a separate elementary reaction in our model. Considering all possible configurations at a $\text{Si} - \text{O} - \text{Al}_2$ site leads to a total of five new hydrolysis reactions. Counting the corresponding condensation reactions, our model includes 14 different elementary reactions related to the hydrolysis of the three distinct bridging oxygen sites.

2.4 Kinetic Parameters

One of the important advantages of our model is that it provides a framework in which to assess the importance of elementary rate constants obtained from quantum mechanics and TST. According to TST, the rate constant for an elementary process of any molecularity:



where $M^\ddagger$ is an activated complex or transition state, is given by (Eyring, 1935):

$$k = (LV)^{(a+b+\dots)} \left(\frac{RT}{Lh} \right) \frac{q^\ddagger}{q_A^a q_B^b} e^{-LE_0/RT} \quad (2.3)$$

where L is Avagadro's number, V is the molar volume of the system, R is the gas constant, h is Planck's constant, the q 's are partition functions, $\ddagger$ indicates a property of the activated complex, and E_0 is the activation energy. The partition function is the product of the translational, rotational, vibrational, and electronic partition functions. Rotations and translations are not applicable to mineral surface reactions, and the electronic partition function for ground-states is equal to one. The vibrational partition function is approximately:

$$q_v = \prod_{i=1}^s \left(1 - e^{-h\nu_i/kT} \right)^{-1} \quad (2.4)$$

where m is the mass, s is the number of vibrational degrees of freedom, and ν_i is the i -th vibrational frequency. The single imaginary frequency of the activated complex corresponding to the reaction coordinate is excluded from the calculation. Thus, if the appropriate energies and vibrational frequencies are available from *ab initio* calculations, the rate constant can be calculated from first principles.

Since *ab initio* results are computationally expensive, any constraints which we can place on the rate constants that reduce the number of calculations will be very helpful. One constraint is a result of the principle of microscopic reversibility. At equilibrium, the rate of the forward reaction equals the rate of the reverse reaction:

$$rate^+ = rate^- \quad (2.5)$$

From the definition of the rate constant, for a first-order reaction we have:

$$C_{react}k^+ = C_{prod}k^- \quad (2.6)$$

where k_+ and k_- are the rate constants for the forward and reverse reactions, respectively, and C_{react} and C_{prod} are the concentrations of the reactants and products, respectively. Rearranging terms and using the definition of the

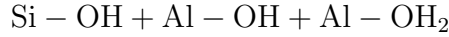
equilibrium constant yields:

$$\frac{k^+}{k^-} = \frac{C_{prod}}{C_{react}} = K_{eq} = e^{-\Delta E/k_b T} \quad (2.7)$$

where K_{eq} is the equilibrium constant, ΔE is the energy change from reactants to products, k_b is Boltzmann's constant, and T is temperature. Thus, for an elementary reaction, the rate constant for the reverse reaction is given by:

$$k^- = k^+ e^{\Delta E/k_b T} \quad (2.8)$$

Microscopic reversibility reduces the number of calculations by a factor of two, since we need only calculate the rate constants for the forward reactions and the energy difference between products and reactants. Another set of constraints may be imposed on the ΔE 's for reactions in the different hydrolysis pathways at the Si – O – Al₂ site. Thermodynamics dictates that the energy of the fully hydrolyzed site:



must be the same regardless of which reaction pathway is followed. Therefore, the sum of the ΔE 's for the individual hydrolysis reactions must be the same for both hydrolysis pathways. Referring to the reaction numbering scheme in Table 1, this means that:

$$\Delta E_3 + \Delta E_4 = \Delta E_5 + \Delta E_6$$

If we also assume that bonding in aluminosilicates is primarily covalent and short-range, it follows that ΔE for any given hydrolysis reaction will be determined primarily by which cation's linkage to the bridging oxygen is being broken. This leads to the following approximations:

$$\Delta E_1 \approx \Delta E_3 \approx \Delta E_6 \approx \Delta E_7$$

$$\Delta E_2 \approx \Delta E_4 \approx \Delta E_5 \approx \Delta E_8$$

Until *ab initio* calculations for each of these reactions are available, these relationships may be used to provide helpful constraints.

In the absence of *ab initio* results, we can approximate ΔE for a given hydrolysis reaction using the enthalpy of dissolution, ΔH_{diss} , for the appro-

priate single cation mineral:

$$\Delta E \approx \frac{2}{N_b} \Delta H_{diss} \quad (2.9)$$

where N_b is the coordination number of the cation in the mineral, and the factor of two indicates that each bond is shared by two atoms. For example, ΔE for Si – O – Si hydrolysis may be approximated by $\Delta H_{diss}^{qtz}/2$. The ΔE for Al – OH – Al hydrolysis may be approximated by $\Delta H_{diss}^{gibb}/3$. Using thermodynamic data tabulated in Krauskopf (1979), calculated values are:

$$\Delta E_2 = 4.80 \text{ kcal/mol}$$

$$\Delta E_1 = 2.57 \text{ kcal/mol}$$

The effect of catalysts such as H^+ , OH^- , and organic ligands may also be incorporated. This is achieved by first determining what fraction of the surface sites will be affected by the catalysis at the simulation conditions, *e.g.* from adsorption isotherms. Then, when determining the rate constant for a specific reaction at a specific site, use the rate constant for the catalyzed reaction with a probability equal to the fraction of sites just determined. Alternately, the rate constant for a reaction in the presence of a catalyst can be expressed as a function of the concentration of the catalyst and the simulation conditions. If done correctly, both approaches should yield similar results.

The rate of impingement of a cation complex on the surface depends upon the number of such complexes in the solution near the surface and is independent of the local surface structure. Assuming that there is only one dominant aqueous complex for a given cation, the rate of adsorption of a cation complex depends only on the chemical potential, μ^{aq} , of the cation in the aqueous solution:

$$k^{ads} = \nu e^{\mu^{aq}/k_b T} \quad (2.10)$$

where ν is a frequency factor corresponding to the rate constant at infinite temperature. We can rewrite this expression in terms of $\Delta\mu$ using the definition of $\Delta\mu$:

$$\Delta\mu = \mu^{aq} - \mu^e \quad (2.11)$$

where μ^e is the chemical potential of the cation in the solution at equilibrium with the solid. At equilibrium, $\Delta\mu = 0$, so $\mu^e = \mu^s$, where μ^s is the chemical

potential of the cation in the solid. Substituting into the expression for the rate constant gives:

$$k^{ads} = \nu e^{\mu^s/k_b T} e^{\Delta\mu/k_b T} \quad (2.12)$$

We can approximate μ^s by:

$$\mu^s = - \sum_i^{\text{bondtypes}} \frac{n_i}{Z_i} \Delta E_i \quad (2.13)$$

where Z_i is the number of cations bonded to a bridging oxygen of type i , n_i is the number of type i bridging oxygens bonded to each cation, and ΔE_i is energy of the corresponding hydrolysis reaction. Using this approximation, the chemical potentials of Si and Al in kaolinite, μ_{Si}^s and μ_{Al}^s , respectively, are given by:

$$\mu_{Si}^s = -\frac{3}{2}\Delta E_{siosi} - \frac{1}{3}\Delta E_{sioal} \quad (2.14)$$

$$\mu_{Al}^s = -3\Delta E_{alohal} - \frac{2}{3}\Delta E_{sioal} \quad (2.15)$$

where ΔE_{siosi} , ΔE_{sioal} , and ΔE_{alohal} are for hydrolysis of the Si – O – Si, Si – O – Al₂, and Al – OH – Al bridging oxygens, respectively.

The local chemical environment of a given surface site will influence the rate at which an adsorbed cation complex desorbs into the solution. The most important factor in determining the chemical environment of a cation surface site is the number of occupied neighboring oxygen sites, since adsorbed cations are held to the surface mainly by electrostatic, or van der Waals, interactions. Again, treating desorption as an activated process, we obtain the following expression for the desorption rate constant, k^{des} :

$$k^{des} = \nu e^{-n_i \Phi_i / k_b T} \quad (2.16)$$

where n_i is the number of type i neighbors and Φ_i is the energy change associated with eliminating the interaction with that type of neighbor. As a result, the desorption rate constant for a site with many occupied neighbors, *i.e.* a kink site, will be much lower than that for one with few occupied neighbors, *i.e.* an adatom.

2.5 Implementation Details

2.5.1 Structural Representation

Implementation of the kinetic Monte Carlo model as a computer program can be greatly simplified by using a few basic concepts from the theory of finite automata and graph theory. The rigid lattice approximation enables the kaolinite structure to be represented as a graph. The vertices of the graph correspond to “atomic” sites, and the edges correspond to bonds. Therefore, the cartesian coordinates of the atomic positions are necessary only when displaying the structure at various stages of the simulation.

The dynamics of the system can then be developed in the context of a nondeterministic finite automaton. Individual graphs correspond to states of the system at different time steps. From any given state, a finite number of transitions is possible. The chemical reactions comprise the set of transitions. The probability of selecting a specific transition is determined by the rate constant of the associated chemical reaction. At each time step, the graph is updated based on a transition selected at random from the set of possible transitions.

This approach greatly simplifies coding and debugging, in that certain invariants must hold at the end of each time step, based on the previous state of the system and the selected transition. These invariants are easily checked by inserting assertion statements in the debugging version of the code. Ignoring programming errors such as out-of-bounds array access, it is possible to “guarantee” program correctness assuming that the set of invariants has been completely covered by assertions. This is an important consideration in this type of simulation, since there are as yet no experimental observations with which to check the model’s correctness.

There are a few other details and assumptions in the model which deserve to be noted. First, since adjacent kaolinite sheets are independent in the context of our reaction model, we simulate a single sheet. It would not be difficult to simulate any arbitrary number of sheets, but this would require more time and memory while providing no benefit given that our reaction model ignores inter-layer interactions. Future versions of the model could incorporate such interactions. Second, the surface plane is either the ac or bc plane. This was done simply because the code required to slice the structure “cleanly” in a plane perpendicular to a crystallographic direction is much less complex than that required to slice in an arbitrary direction. In any case,

once the simulation has begun, all surface planes perpendicular to the inter-layer plane are accessible. Third, dangling cation bonds at the surface are terminated with hydroxyls. This assumption may be chemically questionable, but the current model does not incorporate variations in the rate which depend on the acidity of surface sites or pH. Fourth, periodic boundary conditions are employed parallel to the surface. Periodic boundary conditions are necessary for a number of reasons. “Pinned” boundary conditions, where no reactions are allowed at the boundaries, impose an unrealistic constraint on where reactions may occur. “Free” boundary conditions, where reactions may occur all along the boundaries, would imply that a slab of kaolinite is being modelled. This greatly complicates the issue of unattached clusters, described below, and imposes limits on simulation times due to the memory requirements of larger systems and the shorter expected lifetime of slabs due to the larger available surface area.

2.5.2 Chemical Environment

The hydrolysis of any given bridging oxygen site may depend on the details of the chemical environment experienced by that specific site. Although the local nature of bonding in aluminosilicates dictates that chemical environment effects on the rate constants should be less than an order of magnitude, even small variations may affect the kinetics enough to have an observable impact on the collective dissolution behavior.

To a first approximation, the chemical environment is determined by the state of the next-nearest neighbor oxygen sites. The chemical environment of a specific bridging oxygen will depend upon the number of the other oxygens attached to the bridged cations that have already been hydrolyzed. For example, the chemical environment of a specific Si – O – Si bridging oxygen is determined by the number of other Si – O – Si (0 to 4) and Si – O – Al₂ bridging oxygens (0 to 2) bonded to either of the Si that have already been hydrolyzed. Notice that, for the condensation reactions, we are not counting the bridging oxygen where the reaction is to occur. This leads to a total of 15 different chemical environments and, therefore, 15 possible rate constants for Si – O – Si hydrolysis.

For the reactions involving the Si – O – Al₂ bridging oxygens, the next-neighbor chemical environment is determined by the number of hydrolyzed Si – O – Si bridging oxygens (0 to 3) attached to the Si and the number of hydrolyzed bridging oxygens attached to the Al atoms (0 to 9). This leads

to a total of 40 different possible chemical environments.

Finally, the chemical environment of the Al – OH – Al bridging oxygens is determined by two factors. The first factor is whether or not the other member of the double bridge has been hydrolyzed. The second is the number of other hydrolyzed bridging oxygens, not including the other member of the double bridge, attached to the Al atoms (0 to 8). This leads to 18 different chemical environments.

Since our model incorporates the detailed atomic structure of a piece of kaolinite, we are able to include the effect of next-neighbor interactions on the rate constants. However, until *ab initio* calculations on the nature and magnitude of these effects have been completed, we are limited to performing a sensitivity analysis on their impact. That is, we can systematically vary the magnitudes of the rate constants for different chemical environments in a reasonable manner, and observe the effects on the evolution of the surface.

2.5.3 Unattached Clusters

Unattached clusters are small clusters of sites which have become isolated from the bulk surface as a result of hydrolysis and desorption reactions. In this context, isolated means that there is no path from the cluster sites to surface sites which contains only occupied sites. Although uncommon, such clusters were present in early simulation results. In fact, they are of particular concern in the free boundary condition, or slab, model presented by Lasaga (1995). How to treat such clusters is a debatable issue. For our model, we chose to treat them as having desorbed into solution, thereby removing them from consideration as part of the surface.

Determination of which sites are members of unattached clusters is problematic. The maximum size of a detached cluster can not be known *a priori*. Therefore, some kind of search algorithm is necessary. The questions of which algorithm and when to apply it must now be addressed. The latter is the easier of the two questions to address, in that detached clusters can only result from desorption of a “linking” cation site. Therefore, checking for detached clusters is only necessary after a desorption reaction.

In addressing the first question, graph theory is again helpful. In graph theoretical terms, a detached cluster is a set of occupied nodes which is disjoint to the set of nodes comprising the bulk surface. Therefore, the problem of identifying detached clusters can be reduced to the graph theory problem of identifying connected components. For our case, the problem

can be greatly simplified since it is easy to show that we are only concerned with finding the nodes in the component corresponding to the bulk surface. Any occupied nodes disjoint to this set can be assumed to be members of a detached cluster, and set to unoccupied. Because of the finite representation of the system, in the model there are “terminal” nodes at the interior edge of the surface. If one of these nodes is reached during the course of the simulation, the simulation is terminated, because we have simply run out of material. Therefore, these nodes will *always* be part of the bulk surface. So, to find the nodes in the bulk surface component, we simply do a breadth-first search with a terminal node as the source node. The breadth-first search traverses occupied nodes in the graph, following the edges and stopping at terminal hydroxyl nodes which imply that at least one neighboring cation is unoccupied. Initially, all nodes are “colored” white. Each node discovered by the breadth-first search is colored black. To get rid of the detached clusters, we simply mark as unoccupied all nodes which are occupied and colored white. This method completely solves the problem of unknown cluster size, and profiling of simulation executions shows that it is an insignificant part of the total CPU time expended during a typical run.

2.6 Adsorption/Desorption Parameters

As discussed in Section 2.4, many of the necessary kinetic parameters which are unknown can be estimated from thermodynamic considerations. However, this is not the case for the frequency factors needed to determine the rates of adsorption and desorption. One set of constraints which must be satisfied by the simulation is imposed by the general relationship between ΔG and rate. For $\Delta G < 0$, dissolution must occur. For $\Delta G > 0$, growth must occur. For $\Delta G = 0$, no net growth or dissolution should occur, at least for some sufficiently long time scale. By assuming a frequency factor for either adsorption or desorption, the remaining frequency factor can be fine-tuned to satisfy these constraints. Of course, the ultimate goal is to input frequency factors determined experimentally or theoretically.

Chapter 3

Monte Carlo Simulation Results

3.1 Introduction

Talk about goals of simulations and any problems encountered.

3.2 Choice of Kinetic Parameters

Tabulate and justify choices for all kinetic parameters.

3.3 Precision Analysis

What is a precision analysis.. Show results for numerous runs at same conditions with different random seed.

3.4 Rate vs. ΔG

How to calculate delta G.. Show results for same random seed and temperature, several different saturation states. Discuss any trends, esp. wrt any experimental work

3.5 Overall Activation Energy

Talk about overall activation energy in terms of complex mechanism. Show results for several different T's with calculated Ea's. Discuss values of Ea's in terms of kinetic parameters and implications for rate-limiting step.

3.6 Surface Orientation

Show results for two different orientations and compare.

3.7 Summary and Conclusions

Summarize results. Point out highlights and shortcomings. Discuss ideas for future improvements and necessary work.

Chapter 4

Organic Acids and Aluminosilicates I: Reactions

4.1 Introduction

The purpose of the present study is to gain further insight into the role of low molecular weight dicarboxylic acids in aluminosilicate weathering. Several such acids are present in appreciable amounts in certain soil environments, and also serve as proxies for reactive functional groups in the more common high molecular weight humic and fulvic substances. Several experimental studies have demonstrated that carboxylic acids such as oxalic acid and acetic acid enhance aluminosilicate dissolution over certain pH ranges. Dicarboxylic organic acids are thought to complex with aluminum in aqueous solutions, and by proxy at mineral surfaces, as bidentate chelates. The formation of such bidentate chelates results in a ring structure, formed by the chelating ligand and the aluminum atom. For example, if the chelate is an oxalate anion ($C_2O_4^-$), the ring consists of the two oxygen atoms bonded to the aluminum, the aluminum, and the two carbon atoms (five-member ring). Experiments have provided evidence that the strength of the effect of a dicarboxylic organic acid on the dissolution rate is inversely proportional to the number of members in the ring formed by the bidentate chelate and aluminum. The goals of this study are to provide theoretical confirmation and a quantum mechanical understanding of these observations.

To achieve these goals, high level density functional theory calculations have been performed on a number of clusters. The effect of ring chelate size

was investigated using oxalate, malonate, and succinate anions, representing five-, six-, and seven-member rings. Results obtained include the optimized geometries, energies, and vibrational frequencies of minimum energy structures for a number of species involved in the relevant reactions.

The remainder of this chapter is organized as follows. Section 4.2 outlines the reaction models and computational methods used. Section 4.3.1 presents results for the silicon-aluminum dimer and compares them to previous theoretical results and real mineral structures. Section 4.3.2 presents results obtained for adsorption of H_2O and the various organic acid anions to the model aluminosilicate. Section 4.3.3 presents results obtained for the hydrolysis reactions, both with and without the involvement of organic acid anions. Section 4.4 summarizes the results and concludes.

4.2 Procedures

4.2.1 Computational Method

All calculations were performed using the program Gaussian 94 (Frisch et al., 1995). Calculations were performed on workstations based on the DEC Alpha family of microprocessors at Yale University, and on Cray J90's at the National Energy Research Supercomputer Center (NERSC). Electronic structure calculations were done using the three parameter hybrid method (B3LYP) due to (Becke, 1993). This method combines a three parameter Hartree-Fock (HF) exchange term with correlation terms, both local (Vosko et al., 1980, equation III) and non-local (Lee et al., 1988). Because of the inclusion of electron correlation terms, the results are superior to pure HF calculations, and are comparable to the traditional second order perturbation correction (MP2), due to Møller and Plesset (1934), while being much less computationally expensive.

Minimum energy structures were obtained using the split valence 3-21G (3d,2p) basis set. In this notation, the 3 indicates that the inner, or core, atomic orbitals are expanded in three gaussian functions. The 2 and 1 indicate that this is a split valence basis set, in which two sets of gaussians are used to expand the valence atomic orbitals, 2 dense and 1 diffuse. This allows the valence orbitals to contract or expand, depending on the chemical environment. The terms in parentheses indicate that this is a polarized basis set, with 3d orbitals added to second row elements and 2p orbitals added

to first row elements. Larger basis sets, such as triple zeta or basis sets with diffuse components, provide superior results, but are much more computationally expensive. The 3-21G(3d,2p) choice is a tradeoff between basis set completeness and computational feasibility. Although larger basis sets, such as the triple-zeta 6-311G (3d,2p) basis set, are reasonable for geometry optimizations on the smaller molecules, for the larger clusters this choice is computationally prohibitive. Therefore, additional minima were determined using the 6-311G (3d,2p) basis set in order to assess the validity of the 3-21G (3d,2p) results. Energy and wavefunction calculations were performed on the 3-21G (3d,2p) minimum energy structures using the larger 6-31G (3d) set.

Basis-set superposition errors (BSSE) may be a consideration when calculating reaction energies involving intermolecular complexes (Paizs and Suhai, 1998, and references therein). BSSE is a result of overlap between the basis sets of the two monomers, and results in interaction energies that are too high. The most popular method for correcting BSSE is the counterpoise method, due to (Boys and Bernardi, 1970). In this method, the energy of each monomer is recalculated in the basis of the whole complex, and these energies are subtracted from the complex energy to obtain the interaction energy. Since the organic anions form strong (i.e., non-Hydrogen) bonds with the H_6SiAlO_7 dimer, these do not constitute intramolecular complexes in this sense. In this study, the only intermolecular complexes are those involving an adsorbed water molecule, $H_2O \cdot H_2O$, $OH^- \cdot H_2O$, and $H_3O^+ \cdot H_2O$. Reported values (Paizs and Suhai, 1998) for various BSSE corrections for the $H_2O \cdot H_2O$ dimer are on the order of 1 kcal/mol. BSSE corrections were not performed in this study, due to the small values of the water adsorption energy (several kcal per mole) and typical BSSE energies (a few kcal per mole) relative to the overall reaction energies (tens of kcal per mole), combined with the computational effort involved in correcting for BSSE. It should be noted that previous *ab-initio* geochemical studies (Xiao and Lasaga, 1994; Xiao and Lasaga, 1996; Kubicki et al., 1996a; Strandh et al., 1997) have also ignored BSSE.

Besides BSSE, calculations of this nature incorporate errors from a number of sources, broadly categorized as numerical errors and model errors. Numerical errors include rounding errors, inaccuracies in the numerical evaluation of various integrals, incomplete self-consistent field convergence, and incomplete convergence of the geometry optimizations. Model errors include errors which arise from the underlying assumptions inherent in density functional theory (e.g., the approximate nature of the functionals), the use of

finite-size clusters to represent mineral surfaces, incomplete treatment of solvation effects, the limits of finite un-saturated basis sets and spurious hydrogen bonding involving structure-terminating hydrogen atoms. Due to a lack of experimental data, there is no good way to assess the relative significance of these sources of error, however, the agreement between previous calculations of geometries and reaction energetics with experimental observations suggests that they are not prohibitive (Xiao and Lasaga, 1994; Xiao and Lasaga, 1996; Kubicki et al., 1996a; Strandh et al., 1997, for example).

An additional source of error due to the particular method used in this study is the presence of imaginary frequencies in the calculated vibrational spectra due to the calculation of vibrational frequencies using a larger basis set than for the geometry optimization. This will affect the zero-point corrections to the energies, and therefore most of the discussion will focus on the reaction energies ignoring the zero-point correction, although zero-point corrected energies are presented for completeness. The effect of ignoring zero-point corrections is small (on the order of tenths to a few kcal/mol) relative to the energy differences between the various reactions discussed (typically tens of kcal/mol).

4.2.2 Reaction Model

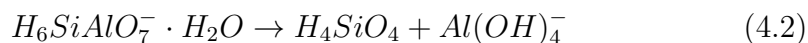
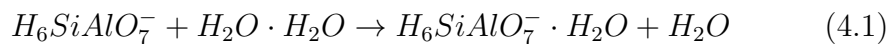
For all calculations, the aluminosilicate surface has been modelled using the $H_6SiAlO_7^-$ cluster, hereafter referred to as the dimer. This cluster has been successfully used to model the aluminosilicate structure by previous studies (Xiao and Lasaga, 1994, for example). Like previous work (Xiao and Lasaga, 1994), this study models the adsorption of H_3O^+ to the dimer by adding an H^+ to the bridging oxygen and adsorbing a water molecule to the resultant H_7SiAlO_7 dimer. Xiao and Lasaga (1994) demonstrate that this approach is equivalent to putting an optimized H_3O^+ near an optimized $H_6SiAlO_7^-$ dimer and obtaining a minimum energy configuration. Reaction energies were calculated by subtracting the sum of the energies of the reactants from the sum of the energies of the products. Reaction energies using both the zero-point corrected energies and the uncorrected energies were calculated.

The following discussion outlines the reactions used to model the hydrolysis of the bridging oxygen bond in the dimer for the various organic acid anions. Some of the reactions listed are not elementary reactions, meaning that they would not be amenable to use in a transition state theory treatment. However, they are more suited to capturing the salient features of the

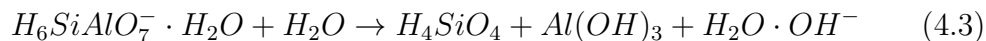
reaction energy landscape. As discussed in Chapter 2, it is widely believed that the organic ligands complex with the aluminum atom much more readily than the silicon. Therefore, the mechanism used in this study only considers the case where the organic acid anion attaches to the dimer at the aluminum site. An analogous study could be performed for attachment at the silicon site to verify this assumption.

Dimer

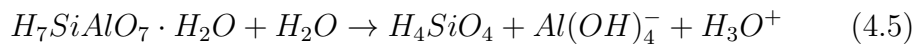
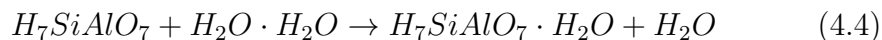
The following reactions were used to model the hydrolysis of the bridging oxygen site for the bare dimer, that is, without any organic acid anion. Reactions 4.1 and 4.2 are the two steps in the hydrolysis of the un-protonated bridging oxygen bond.



In Reaction 4.1, a water dimer dissociates and one of the waters adsorbs to the dimer. In Reaction 4.2, the adsorbed water hydrolyzes the bridging oxygen. Taking Reaction 4.2 one step further, and allowing the $Al(OH)_4^-$ to lose one OH^- , we have:

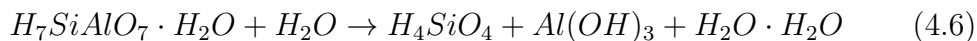


Reactions 4.4 and 4.5 are the two steps in the hydrolysis of the protonated bridging oxygen, that is, hydrolysis catalyzed by H_3O^+ . Reactions involving H_3O^+ catalyzed hydrolysis assume that the attachment of the proton to the bridging oxygen is a separate step, unrelated to the actual hydrolysis other than having the effect of protonating the bridging oxygen. These reactions are analogous to Reactions 4.1 and 4.2.



The addition of the H^+ to the bridging oxygen follows the method used by
Taking Reaction 4.5 one step further and removing a OH^- from the

$Al(OH)_4^-$ yields an alternative reaction to 4.5:



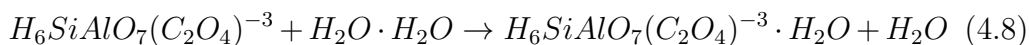
Oxalate

Adding an organic acid anion to the system introduces several new reactions, namely those involving the adsorption of the anion to the aluminum site. The order in which the oxalate and water adsorb to the dimer may or may not be important, so both sets of reactions are given.

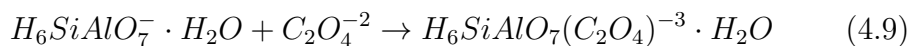
First, consider hydrolysis of an un-protonated bridging oxygen. The first step involves the chemisorption of an oxalate anion to the aluminum site in the dimer:



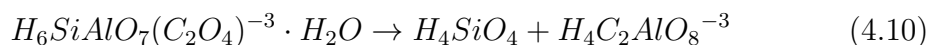
The second step is the adsorption of a water molecule:



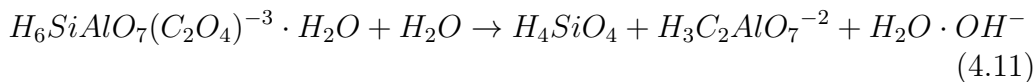
We may also have the case where a water molecule adsorbed first, in which case the second step is the chemisorption of the oxalate:



The final step involves the actual hydrolysis of the bridging oxygen by the adsorbed water molecule:



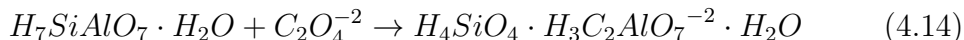
This results in a silic acid molecule, an aluminum–oxalate–OH complex, and a water molecule. An alternative set of products is possible, in which the aluminum–oxalate–OH complex has one fewer OH:



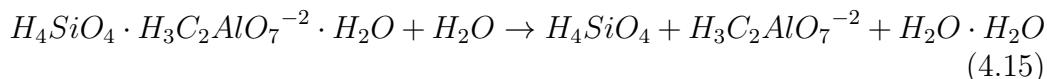
Next, consider the case of hydrolysis of a protonated bridging oxygen (H_3O^+ catalysis). First, the oxalate chemisorbs to the protonated dimer:



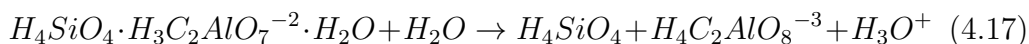
As for the unprotonated case, the second step depends upon whether oxalate or water adsorbed first, although the products are identical. If Reaction 4.12 was first, Reaction 4.13 follows. Otherwise, Reaction 4.14 is next.



From this point, there are several possible final configurations resulting from the actual hydrolysis of the bridging oxygen, depending upon the stoichiometry of the final aluminum-oxalate-*OH* complex. The structure of the precursor strongly suggest one possibility:

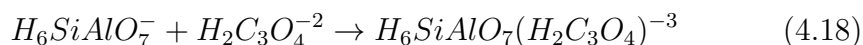


Other possibilities need to be examined, however, to determine the most energetically favorable configuration:

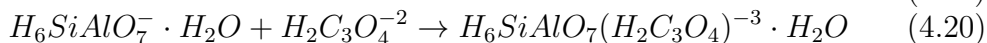
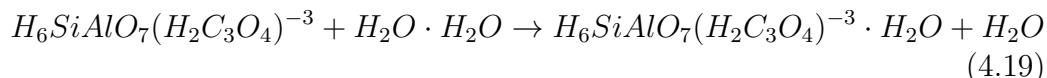


Malonate

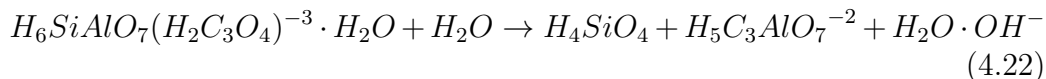
The reactions involving malonate are virtually identical to those involving oxalate, and are presented without discussion. Adsorption of malonate:



Progress to full precursor:



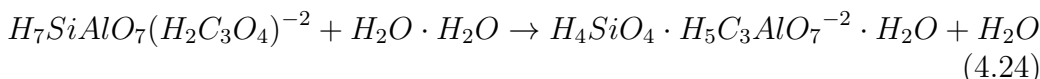
And finally, the possibilities for the hydrolysis step:



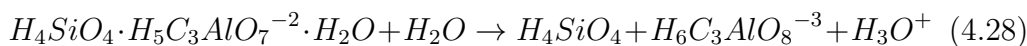
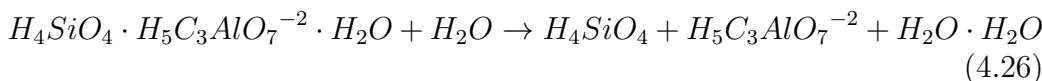
Adsorb malonate to the protonated dimer:



Again, the second adsorption step depends on the first:



Several possibilities exist for the final hydrolysis step:

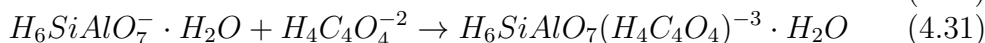
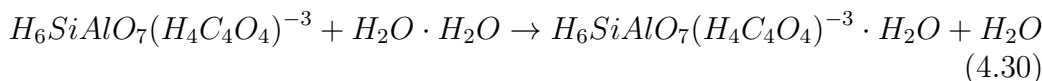


Succinate

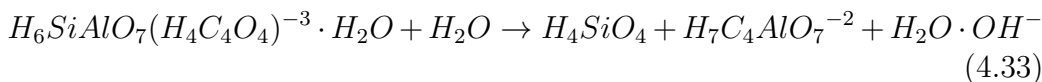
Again, the reactions involving succinate are virtually identical to those involving oxalate and malonate. Adsorption of succinate to the unprotonated dimer:



For the second adsorption step we again have two possibilities:



And finally, possibilities for the hydrolysis step include:

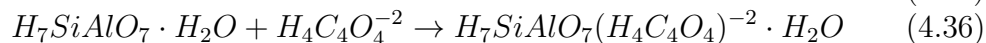
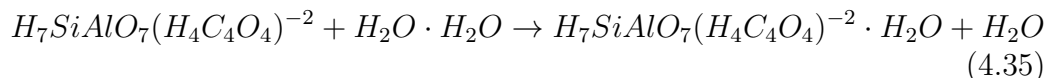


For the case of a protonated bridging oxygen, the initial adsorption step:



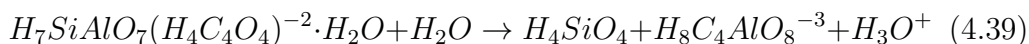
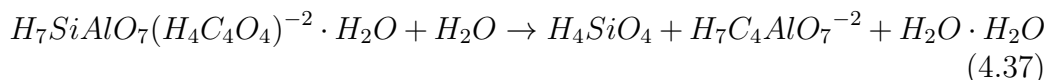
Depending on the first adsorption step, two possibilities exist for the

second adsorption step:



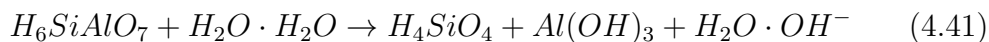
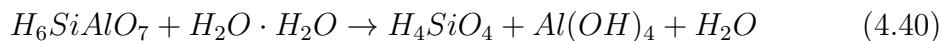
Note that unlike oxalate and malonate, succinate forms a stable complex bonded to the protonated dimer. This result was not assumed, but rather it was a result of the geometry minimizations.

At least three possibilities exist for the final hydrolysis step:

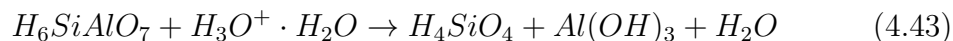
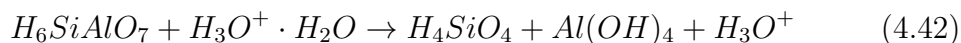


Overall Hydrolysis Reactions

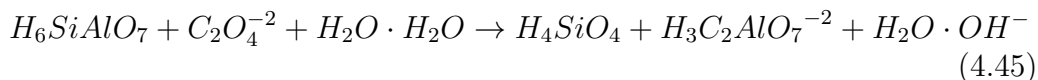
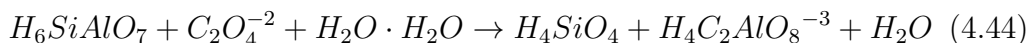
Reaction steps from the previous section can be combined to form overall hydrolysis reactions for the different cases. For H_6SiAlO_7 hydrolysis, there are two choices:



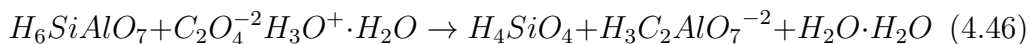
For H_7SiAlO_7 hydrolysis, we also have two choices:

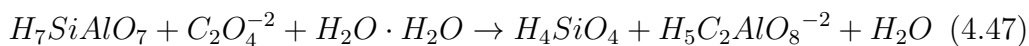


For $H_6SiAlO_7^-$ hydrolysis in the presence of oxalate, there are two choices:

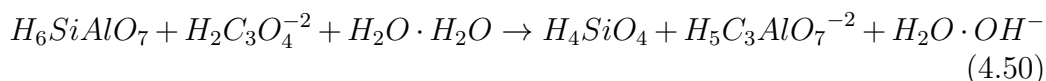
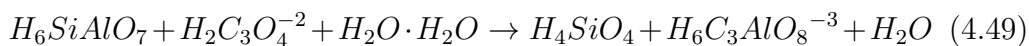


For H_7SiAlO_7 hydrolysis in the presence of oxalate, there are three choices:

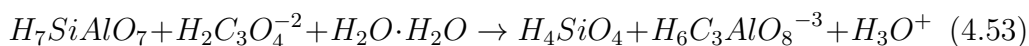
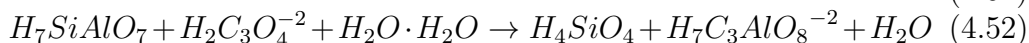
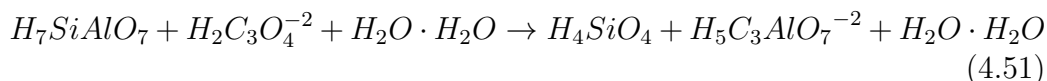




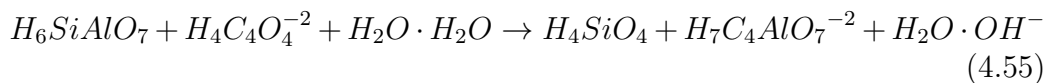
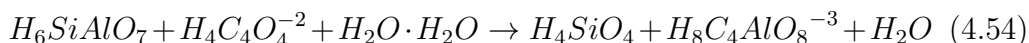
For $H_6SiAlO_7^-$ hydrolysis in the presence of malonate, there are two choices:



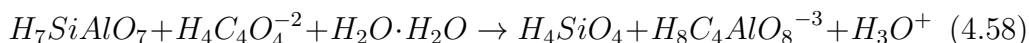
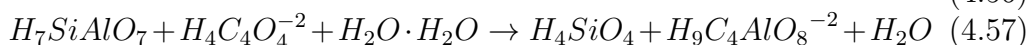
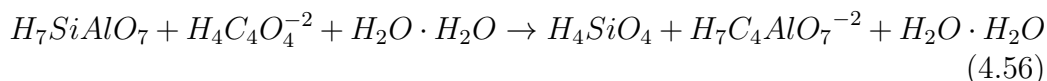
For H_7SiAlO_7 hydrolysis in the presence of malonate, there are three choices:



For $H_6SiAlO_7^-$ hydrolysis in the presence of succinate, there are two choices:



For H_7SiAlO_7 hydrolysis in the presence of succinate, there are three choices:



4.3 Results and Discussion

4.3.1 Dimer Structure

Figures 4.1 and 4.2 show the minimum energy structures obtained in this study for the $H_6SiAlO_7^-$ and H_7SiAlO_7 dimers, respectively. Some geometric properties of the minimum energy $H_6SiAlO_7^-$ and H_7SiAlO_7 dimers obtained in this study are presented in Table 4.1. Also presented are results obtained by Xiao and Lasaga (1994) for the same cluster using Hartree-Fock

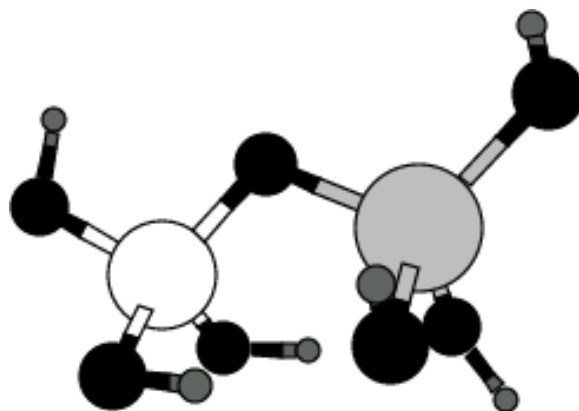


Figure 4.1: $H_6SiAlO_7^-$ Dimer structure optimized at the B3LYP/3-21G** level. In this and all subsequent figures, the large grey circles represent Al, the large white circles Si, the medium black circle O, and the small dark grey circles H atoms.

theory with a second order Møller-Plesset perturbation treatment of electron correlation.

Although the results from the two studies show generally good agreement, some discrepancies are worth noting. First, the bond lengths obtained using the B3LYP/3-21G** method differ slightly from those obtained by Xiao and Lasaga (1994) using the MP2/6-31G* method. The latter are expected to be slightly more accurate due to the use of a larger basis set, as discussed previously in Computational Methods. Second, the $SiOAl$ angles differ significantly between the two studies. This is most likely due to the difference between the studies in structure termination and choice of basis set. Xiao and Lasaga (1990) terminated the structure with hydrogen atoms, whereas in this study the structure was terminated with hydroxyls. In addition, use of the 3-21G** basis set adds polarization to the hydrogen orbitals, which can result in spurious hydrogen bonding between the terminating hydroxyls on the aluminum and silicon atoms, thereby pulling the structure into a tighter configuration.

Inspection of the results in Table 4.1 shows that upon protonation, the SiO_{br} and AlO_{br} bonds relax slightly. The AlO_{br} bond lengthens by 0.1128Å and the SiO_{br} bond lengthens by 0.1122Å. The $SiOAl$ bond angle increases by 1.5° upon protonation. While the change in the SiO_{br} bond length is similar to that found by Xiao and Lasaga (1994), their calculations showed

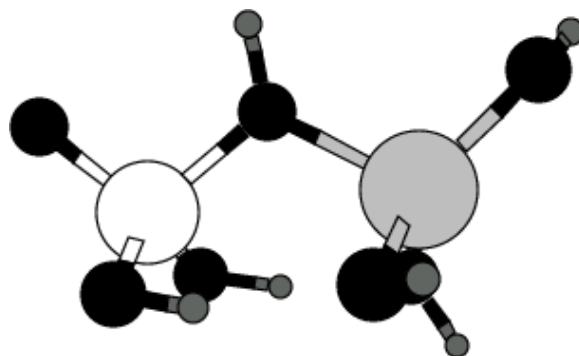


Figure 4.2: H_7SiAlO_7 Dimer structure optimized at the B3LYP/3-21G** level.

Molecule	Bond	B3LYP	Xiao
$H_6SiAlO_7^-$	AlO_{br}	1.766	1.825
	SiO_{br}	1.637	1.611
	$SiOAl$	113.1	138.9
H_7SiAlO_7	AlO_{br}	1.878	2.024
	SiO_{br}	1.749	1.724
	$SiOAl$	114.6	122.1
	HO_{br}	0.962	0.972

Table 4.1: Comparison of geometric properties obtained in this study using B3LYP/3-21G** (B3LYP) and by Xiao and Lasaga (1994) using MP2/6-31G* (Xiao). Bond lengths are in Angstroms and the $SiOAl$ angle is in degrees.

	<i>Reaction</i>	ΔE
(a)	$H_6SiAlO_7^- + H_2O \cdot H_2O \rightarrow H_6SiAlO_7^- \cdot H_2O + H_2O$	-5.9
(b)	$H_6SiAlO_7^- + H_2O \cdot H_3O^+ \rightarrow H_7SiAlO_7 + H_2O \cdot H_2O$	-108.7
(c)	$H_7SiAlO_7 + H_2O \cdot H_2O \rightarrow H_7SiAlO_7 \cdot H_2O + H_2O$	-11.7

Table 4.2: Calculated zero-point corrected energies (kcal/mol) for adsorption of H_2O to dimer.

twice as large a change for the AlO_{br} bond. Also, their bond angle *decreased* by over 16° .

4.3.2 Adsorption

Calculated energies for all adsorption reactions involving water, oxalate, malonate, and succinate are presented in Tables 4.2, 4.4, 4.5, and 4.6, respectively. A few significant geometric parameters for a number of complexes are given in Table 4.3. Appendix B contains figures picturing fully optimized geometries for all of the complexes discussed in the following section, and for which figures do not accompany the text.

H_2O and H_3O^+ Adsorption

An essential step in a hydrolysis reaction is the adsorption of a water molecule to the surface. The adsorbed water interacts with atoms on the surface via hydrogen and Van der Waals interactions. This may either increase or decrease the stability the various bonds between the atoms in the dimer, in particular the SiO_{br} and AlO_{br} bonds.

A water molecule adsorbs to the H_6SiAlO_7 dimer on top of the aluminum, but out of the plane of the dimer, with its oxygen 2.843 Angstroms from the aluminum (Figure B.1). The hydrogens are hydrogen bonded to oxygens in the dimer, one of which is the bridging oxygen, the other of which is the oxygen of a terminal hydroxyl on the aluminum atom. The hydrogen of another of the aluminum's terminal hydroxyls is hydrogen bonded to the oxygen in the water. Lengths of the hydrogen bonds range from 1.882 to 1.948 Angstroms.

Xiao and Lasaga (1994) present similar results for water adsorption, calculated using the Hartree-Fock technique with a second order perturbation

	$Al - O_{br}$	$Si - O_{br}$	$H - O_{br}$
H_6SiAlO_7	1.7655	1.6371	—
$H_6SiAlO_7 \cdot H_2O$	1.7851	1.6507	1.9484
H_7SiAlO_7	1.8783	1.7493	0.9624
$H_7SiAlO_7 \cdot H_2O$	1.8339	1.7085	1.0476
$H_6C_2SiAlO_{11}$	1.8917	1.6127	—
$H_6C_2SiAlO_{11} \cdot H_2O$	1.9264	1.6417	1.4911
$H_7C_2SiAlO_{11}$	—	1.6404	0.9809
$H_7C_2SiAlO_{11} \cdot H_2O$	—	1.6632	0.9968
$H_8C_3SiAlO_{11}$	1.8749	1.6137	—
$H_8C_3SiAlO_{11} \cdot H_2O$	1.9061	1.6406	1.4958
$H_9C_3SiAlO_{11}$	—	1.6419	0.9853
$H_9C_3SiAlO_{11} \cdot H_2O$	—	1.6381	0.9834
$H_{10}C_4SiAlO_{11}$	1.8970	1.6174	—
$H_{10}C_4SiAlO_{11} \cdot H_2O$	1.9384	1.6464	1.5510
$H_{11}C_4SiAlO_{11}$	2.1317	1.6824	0.9857
$H_{11}C_4SiAlO_{11} \cdot H_2O$	2.1617	1.7047	1.0143

Table 4.3: Effect of adsorption of protons, water, and organic acid anions on bond lengths, in Angstroms.

correction for electron correlation and a 6-31G* basis set. The most significant difference between their work and this study is that the water adsorbs very nearly in the $AlOSi$ plane, and is closer to the bridging oxygen. For example, the distance from the water oxygen to the aluminum atom in Xiao and Lasaga (1994) is 3.261 Å, whereas it is 2.843 Å in this study. The distance from the nearest water hydrogen to the bridging oxygen, however, is nearly identical in both studies, 1.934 Å in Xiao and Lasaga (1994) and 1.948 Å in this study. The most likely explanation for the different adsorption positions of the water molecule is that Xiao and Lasaga (1994) terminate their dimer with hydrogen atoms, whereas the structure is terminated with hydroxyls in this study. In addition, unlike Xiao and Lasaga (1994), in this study p orbitals were added to the hydrogens to more accurately incorporate hydrogen bonding.

Adsorption of a water molecule to the H_6SiAlO_6 dimer lengthens the AlO_{br} and SiO_{br} bonds by 0.02 and 0.014 Å, respectively. Lengthening, and presumably weakening, of these bonds is due to hydrogen bonding between the bridging oxygen and a hydrogen in the water molecule. This polarizes the oxygen's valence shell charge, decreasing the charge density which contributes to the bonds with the aluminum and silicon atoms. These changes are similar to those presented by Xiao and Lasaga (1994), who report bond lengthening of 0.014 and 0.016 Å for the AlO_{br} and SiO_{br} bonds, respectively.

The zero-point corrected ΔE for water adsorption is -5.9 kcal/mol. Xiao and Lasaga (1994) report a zero-point corrected ΔE of -4.61 kcal/mol, calculated at the MP2/6-31G* level of theory. In addition, they report a ΔE of -5.76 kcal/mol, calculated using HF/6-31G* which ignores electron correlation.

A water molecule also adsorbs to the protonated H_7SiAlO_7 dimer over the aluminum atom, although in this case more closely aligned with the $SiO_{br}Al$ plane (Figure B.2). The water's oxygen is 3.261 Angstroms from the aluminum atom, which is significantly further away than in the unprotonated case. The proton attached to the bridging oxygen is hydrogen bonded to the oxygen atom of the water. One of the water's hydrogen atoms is hydrogen bonded to the oxygen atom of a terminal hydroxyl on the aluminum. The lengths of these two hydrogen bonds are 1.464 and 1.632 Angstroms, respectively, both of which are significantly shorter than any of the hydrogen bonds for the unprotonated case.

Adsorption of a water to the protonated dimer, H_7SiAlO_7 , results in a *shortening* of the AlO_{br} and SiO_{br} bonds by 0.044 and 0.041 Angstroms,

respectively. In addition, the HO_{br} bond lengthens by 0.085 Angstroms. The lengthening of the HO_{br} bond is due to hydrogen bonding between the oxygen of the water molecule and the hydrogen attached to the bridging oxygen. This slightly decreases the effect of the hydrogen on the bridging oxygen's charge distribution, causing the AlO_{br} and SiO_{br} bonds to adjust towards those in the H_6SiAlO_7 dimer.

Xiao and Lasaga (1994) report similar results for adsorption of water to the protonated dimer. In both studies, the water adsorbs slightly to the aluminum side of the bridging oxygen. However, in this study the water adsorbs closer to the dimer, with the distance between the water oxygen and the proton on the bridging oxygen being 1.464Å, as opposed to 1.661Å in Xiao and Lasaga (1994). The distance between the water oxygen and the aluminum atoms is similar in both studies, 3.479Å in Xiao and Lasaga (1994) and 3.261Å in this study. The most likely explanation for the difference between the studies is the use of polarization functions (p orbitals) on the hydrogen atoms in this study, allowing a more accurate representation of the hydrogen bonding between the water oxygen and the hydrogen attached to the bridging oxygen.

The zero-point corrected energy of water adsorption to the protonated dimer is -11.7 kcal/mol. Xiao and Lasaga (1994) report a zero-point corrected adsorption energy of -10.02 kcal/mol. An important point to note is the large negative ΔE for protonation of the bridging oxygen, -108.7 kcal/mol. The energy of formation of the water-hydronium dimer calculated in this study is -40.9 kcal/mol, compared to Xiao and Lasaga's (1994) result of -38.47 kcal/mol and an experimental value of -36.0 kcal/mol (Yamabe and Minato, 1984). The result is a zero-point corrected ΔE of -120.4 kcal/mol for adsorption of a hydronium ion. Xiao and Lasaga (1994) report a value of -3.87 kcal/mol for hydronium adsorption to $H_6Si_2O_7$, but do not report a value for hydronium adsorption to H_6SiAlO_7 . The large discrepancy in adsorption energy is most likely due to the negative charge of the H_6SiAlO_7 dimer, whereas the $H_6Si_2O_7$ dimer is neutral.

Organic Acid Anion Adsorption

Adsorption of the organic acid anions differs in a very important respect from adsorption of a water molecule. While the water molecule adsorbs via hydrogen bonding, the organic anions form more traditional bonds with the aluminum atom. The exact nature of these bonds (i.e., covalent, ionic, or

	Reaction	ΔE
(a)	$H_6SiAlO_7^- + C_2O_4^{-2} \rightarrow H_6SiAlO_7(C_2O_4)^{-3}$	75.5
(b)	$H_6SiAlO_7^- \cdot H_2O + C_2O_4^{-2} \rightarrow H_6SiAlO_7(C_2O_4)^{-3} \cdot H_2O$	61.7
(c)	$H_6SiAlO_7(C_2O_4)^{-3} + H_2O \cdot H_2O \rightarrow H_6SiAlO_7(C_2O_4)^{-3} \cdot H_2O + H_2O$	-19.6
(d)	$H_7SiAlO_7 + C_2O_4^{-2} \rightarrow H_7SiAlO_7(C_2O_4)^{-2}$	-129.0
(e)	$H_7SiAlO_7 \cdot H_2O + C_2O_4^{-2} \rightarrow H_4SiO_4 \cdot H_3C_2AlO_7^{-2} \cdot H_2O$	-127.0
(f)	$H_7SiAlO_7(C_2O_4)^{-2} + H_2O \cdot H_2O \rightarrow H_4SiO_4 \cdot H_3C_2AlO_7^{-2} \cdot H_2O + H_2O$	-9.7

Table 4.4: Calculated zero-point corrected energies (kcal/mol) for adsorption of oxalate to dimer, and adsorption of H_2O to dimer-oxalate complex.

	Reaction	ΔE
(a)	$H_6SiAlO_7^- + H_2C_3O_4^{-2} \rightarrow H_6SiAlO_7(H_2C_3O_4)^{-3}$	91.6
(b)	$H_6SiAlO_7^- \cdot H_2O + H_2C_3O_4^{-2} \rightarrow H_6SiAlO_7(H_2C_3O_4)^{-3} \cdot H_2O$	77.8
(c)	$H_6SiAlO_7(H_2C_3O_4)^{-3} + H_2O \cdot H_2O \rightarrow H_6SiAlO_7(H_2C_3O_4)^{-3} \cdot H_2O + H_2O$	-19.7
(d)	$H_7SiAlO_7 + H_2C_3O_4^{-2} \rightarrow H_7SiAlO_7(H_2C_3O_4)^{-2}$	-109.3
(e)	$H_7SiAlO_7 \cdot H_2O + H_2C_3O_4^{-2} \rightarrow H_4SiO_4 \cdot H_5C_3AlO_7^{-2} \cdot H_2O$	-102.3
(f)	$H_7SiAlO_7(H_2C_3O_4)^{-2} + H_2O \cdot H_2O \rightarrow H_4SiO_4 \cdot H_5C_3AlO_7^{-2} \cdot H_2O + H_2O$	-4.7

Table 4.5: Calculated zero-point corrected energies (kcal/mol) for adsorption of malonate to dimer, and adsorption of H_2O to dimer-malonate complex.

	Reaction	ΔE
(a)	$H_6SiAlO_7^- + H_4C_4O_4^{-2} \rightarrow H_6SiAlO_7(H_4C_4O_4)^{-3}$	108.6
(b)	$H_6SiAlO_7^- \cdot H_2O + H_4C_4O_4^{-2} \rightarrow H_6SiAlO_7(H_4C_4O_4)^{-3} \cdot H_2O$	98.3
(c)	$H_6SiAlO_7(H_4C_4O_4)^{-3} + H_2O \cdot H_2O \rightarrow H_6SiAlO_7(H_4C_4O_4)^{-3} \cdot H_2O + H_2O$	-16.1
(d)	$H_7SiAlO_7 + H_4C_4O_4^{-2} \rightarrow H_7SiAlO_7(H_4C_4O_4)^{-2}$	-72.8
(e)	$H_7SiAlO_7 \cdot H_2O + H_4C_4O_4^{-2} \rightarrow H_7SiAlO_7(H_4C_4O_4)^{-2} \cdot H_2O$	-85.9
(f)	$H_7SiAlO_7(H_4C_4O_4)^{-2} + H_2O \cdot H_2O \rightarrow H_7SiAlO_7(H_4C_4O_4)^{-2} \cdot H_2O + H_2O$	-24.9

Table 4.6: Calculated zero-point corrected energies (kcal/mol) for adsorption of succinate to dimer, and adsorption of H_2O to dimer-succinate complex.

intermediate) will be discussed at greater detail in the next chapter.

As expected, adsorption of an organic acid anion at the aluminum site has a much greater effect on the AlO_{br} bond than on the SiO_{br} bond. In fact, for all three anions the AlO_{br} bond lengthens by roughly an order of magnitude more than the SiO_{br} bond shortens. This supports the conventional wisdom that bonding in silicates is predominantly local in character.

Examination of the AlO_{br} data in Table 4.3 for the unprotonated dimer reveals that no consistent relationship exists between AlO_{br} bond length and chelating ring size. In all three cases, the increase in the AlO_{br} bond length is similar in magnitude, between 6.2% and 7.5% of the initial dimer bond length, for adsorption of malonate and succinate, respectively. While this lengthening suggests a weakening of the bond upon adsorption of an organic acid anion, the lack of a clear trend contrasts experimental evidence indicating an inverse relationship between chelating ring size and strength of effect on dissolution rate.

Adsorption of oxalate and malonate to the protonated dimer results in a rupture of the AlO_{br} bond, based on maximum bond-length criteria. The absence of any type of bonding between the aluminum and any of the oxygens bound to the silicon will be confirmed by a more sophisticated analysis in the next chapter. This result obviates the need for a true hydrolysis step. In fact, the only role water plays in this dissolution pathway is to deliver the proton to the bridging oxygen. The result of this rupture is a silicic acid molecule triply hydrogen bonded to a complex consisting of the aluminum, three hydroxyls, and the organic acid anion. In future discussions, the dissociation of the hydrogen bonded complex is considered to be the hydrolysis step. While not strictly accurate, this interpretation is not overly misleading since it is the intervention of polar water molecules that effects the dissociation.

The three hydrogen bonds are between siloxane hydrogens and two hydroxyl and one of the anion's oxygens, all of which are bonded to the aluminum. The lengths of the hydrogen bonds are 1.592, 1.689, and 1.788 Å, respectively, in the case of oxalate. Hydrogen bond lengths are 1.535, 1.701, and 1.746 Å, respectively, in the malonate case. Whereas oxalate distorts only slightly upon adsorption, malonate and succinate are both significantly distorted from their normal structures, which have planar carbon backbones. This distortion will be discussed in more detail in the next chapter.

The presence or absence of an adsorbed water does not affect the AlO_{br} bond rupture. For oxalate, the most significant influence of adsorbed water is to distort the adsorbed oxalate by hydrogen bonding with one of the carboxyl

oxygens which is not a part of the chelating ring. This is evident in the $O - C - C - O$ torsions of -15.6° and -17.6° for the oxygens bonded and not bonded to the aluminum, respectively, with adsorbed water present. This is in contrast to torsions of 0.2° and -0.1° , respectively as before, when there is no adsorbed water. One significant result of this torsion is an asymmetry in the bonds between the aluminum and the carboxyl oxygens, one of which is $2.175\text{\AA}$ and the other is $1.860\text{\AA}$, as opposed to $1.878\text{\AA}$ and $1.968\text{\AA}$ in the absence of adsorbed water. The water's oxygen also hydrogen bonds with an aluminum hydroxyl hydrogen, and the other hydrogen in the water bonds to a siloxane oxygen. The lengths of these hydrogen bonds are 1.912 , 1.857 , and $1.807\text{\AA}$, respectively.

Adsorbed water has a smaller effect on malonate than on oxalate, because the water hydrogen bonds to a carboxyl oxygen which is a part of the chelating ring, unlike oxalate. This supplies less torque to the malonate, resulting in minimal additional distortion of the malonate structure. The adsorbed water also hydrogen bonds to a siloxane oxygen. Lengths of these hydrogen bonds are 2.047 and $1.845\text{\AA}$, respectively.

Note that the AlO_{br} bond is not ruptured when succinate adsorbs to the protonated dimer. The bond length increases by $0.253\text{\AA}$ in the absence of water, and by $0.328\text{\AA}$ in the presence of adsorbed water. These are significant increases, 13.4% and 17.9% of the bond length in the protonated H_7SiAlO_7 dimer, respectively. This is the first indication of the experimentally observed relationship between chelating ring size and dissolution rate.

An interesting effect of adsorbed water is that one of the carboxyl oxygen bonds to the aluminum is broken. This appears to be caused mostly by hydrogen bonding between the water and a non-chelating carboxyl oxygen, much the same as for oxalate. In the minimum energy structure for the isolated $H_7C_4AlO_7$ complex, the succinate is also singly-bonded to the aluminum. This is most likely due to the significant distortion of the succinate structure which must occur to accommodate a bidentate chelate. This distortion and its effects will be discussed in greater detail in the next chapter.

Adsorption of all three organic acid anions to the unprotonated dimer is endothermic. Energies for adsorption of oxalate, malonate, and succinate are given in Tables 4.4, 4.5, and 4.6, respectively. Unlike the geometric parameters, adsorption energies show a distinct trend, increasing with increasing size of the chelating ring. In the absence of adsorbed water, adsorption energies increase from a minimum of 75.5 kcal/mol for oxalate to a maximum of 108.6 kcal/mol for succinate. The presence of adsorbed water slightly decreases the

	Reaction	ΔE
(a)	$H_6SiAlO_7^- \cdot H_2O \rightarrow H_4SiO_4 + Al(OH)_4^-$	35.8
(b)	$H_6SiAlO_7^- \cdot H_2O + H_2O \rightarrow H_4SiO_4 + Al(OH)_3 + H_2O \cdot OH^-$	109.4
(c)	$H_7SiAlO_7 \cdot H_2O + H_2O \rightarrow H_4SiO_4 + Al(OH)_4^- + H_3O^+$	183.6
(d)	$H_7SiAlO_7 \cdot H_2O + H_2O \rightarrow H_4SiO_4 + Al(OH)_3 + H_2O \cdot H_2O$	42.6

Table 4.7: Calculated zero-point corrected energies (kcal/mol) for dimer hydrolysis reactions.

adsorption energies, to 61.7 kcal/mol for oxalate and 98.3 kcal/mol for succinate. Adsorption of the organic acid anion is the only endothermic step in the hydrolysis sequences considered for the unprotonated dimer.

Adsorption of organic acid anions to the protonated dimer, on the other hand, is exothermic in all three cases. The same relationship between adsorption energy and chelating ring size is evident as for the unprotonated case. In the absence of adsorbed water, oxalate has the lowest adsorption energy at -129.0 kcal/mol and succinate the highest at -72.8 kcal/mol with malonate intermediate at -109.3 kcal/mol. With a protonated bridging oxygen, addition of an adsorbed water increases the adsorption energy for oxalate (-127.0 kcal/mol) and malonate (-102.3 kcal/mol) but decreases the adsorption energy for succinate (-85.9 kcal/mol).

4.3.3 Hydrolysis

Calculated energies for all hydrolysis reactions involving no organics, oxalate, malonate, and succinate are presented in Tables 4.7, 4.8, 4.9, and 4.10, respectively. Calculated energies for the lowest energy overall hydrolysis mechanisms are presented in Table 4.11.

Dimer Hydrolysis

As of this writing, there are no calculated or experimental thermochemical data in the literature for hydrolysis of the H_6SiAlO_7 dimer. Xiao and Lasaga (1994) report transition states and activation energies for the H_6SiOAl dimer, but do not report total reaction energies. The present study focuses on total reaction energies rather than transition states, so comparison with Xiao and Lasaga (1994) is impossible. However, the good agreement

between this study, Xiao and Lasaga (1994), and available experimental data with respect to water adsorption energies helps to establish the validity of the calculations in this study.

The results for the dimer hydrolysis reactions, presented in Table 4.7, reveal that speciation of the aluminum product complex is crucial to the reaction energy calculated for both the dimer and the protonated dimer. The ΔE for hydrolysis of the H_6SiAlO_7 dimer with $Al(OH)_4^-$ as a product is 35.8 kcal/mol, whereas the energy calculated using $Al(OH)_3$ as the product is 109.4 kcal/mol. The latter value may be slightly large, because the energy of formation of the hydroxyl-water dimer are more negative than that obtained by Xiao and Lasaga (1994) (-33.18 kcal/mol) and the experimental value of -29.3 kcal/mol reported by Paul and Kebarle (1990). This is due to the limitations of the B3LYP hybrid method when calculating the energy of very small, charged molecules, particularly those containing hydrogen. Given the good agreement between this study, Xiao and Lasaga (1994), and experimental results for the association energy of the $H_3O^+ \cdots H_2O$ dimer, however, it appears that the method's limitation is restricted to the OH^- molecule. Alternatively, the problem may be a manifestation of BSSE in the calculation of the energy of the $OH^- \cdots H_2O$ dimer. However, the minimum energy geometries obtained for the $OH^- \cdots H_2O$ dimer are very similar to those obtained by Xiao and Lasaga (1994). For example, the $O-O$ distance is 2.479 Å in this study and 2.466 Å in Xiao and Lasaga (1994). If the goal were to compare reaction energies with experimental data, it would be important to investigate and correct this source of error. However, since we are interested mainly in relative trends in reaction energies, this is not necessary.

For hydrolysis of the protonated dimer, H_7SiAlO_7 , the situation is reversed. Using $Al(OH)_4^-$ as the product, the reaction ΔE is 183.6 kcal/mol, and using $Al(OH)_3$ as the product it is 42.6 kcal/mol. At first glance, one might expect a similar source of error in the former calculation, since H_3O^+ is a product. However, the good agreement between this study, Xiao and Lasaga (1994), and experimental results for the association energy of the $H_3O^+ \cdots H_2O$ dimer suggest that this reaction energy is correct. For hydrolysis of the dimer, by both H_2O and H_3O^+ , the actual hydrolysis step is endothermic.

When adsorption energy is included, hydrolysis by H_3O^+ is exothermic, whereas hydrolysis by H_2O is endothermic. By breaking the reactions into these elementary steps, it is clear that the H_3O^+ is exothermic due to the

	Reaction	ΔE
(a)	$H_6SiAlO_7(C_2O_4)^{-3} \cdot H_2O \rightarrow H_4SiO_4 + H_4C_2AlO_8^{-3}$	99.4
(b)	$H_6SiAlO_7(C_2O_4)^{-3} \cdot H_2O + H_2O \rightarrow H_4SiO_4 + H_3C_2AlO_7^{-2} + H_2O \cdot OH^-$	-59.2
(c)	$H_4SiO_4 \cdot H_3C_2AlO_7^{-2} \cdot H_2O + H_2O \rightarrow H_4SiO_4 + H_3C_2AlO_7^{-2} + H_2O \cdot H_2O$	62.7
(d)	$H_4SiO_4 \cdot H_3C_2AlO_7^{-2} \cdot H_2O + H_2O \rightarrow H_4SiO_4 + H_4C_2AlO_8^{-3} + H_3O^+$	438.0

Table 4.8: Calculated zero-point corrected energies (kcal/mol) for hydrolysis reactions for dimer, oxalate present.

	Reaction	ΔE
(a)	$H_6SiAlO_7(H_2C_3O_4)^{-3} \cdot H_2O \rightarrow H_4SiO_4 + H_6C_3AlO_8^{-3}$	38.6
(b)	$H_6SiAlO_7(H_2C_3O_4)^{-3} \cdot H_2O + H_2O \rightarrow H_4SiO_4 + H_5C_3AlO_7^{-2} + H_2O \cdot OH^-$	-56.0
(c)	$H_4SiO_4 \cdot H_5C_3AlO_7^{-2} \cdot H_2O + H_2O \rightarrow H_4SiO_4 + H_5C_3AlO_7^{-2} + H_2O \cdot H_2O$	57.3
(d)	$H_4SiO_4 \cdot H_5C_3AlO_7^{-2} \cdot H_2O + H_2O \rightarrow H_4SiO_4 + H_6C_3AlO_8^{-3} + H_3O^+$	368.5

Table 4.9: Calculated zero-point corrected energies (kcal/mol) for hydrolysis reactions for dimer, malonate present.

contribution from the adsorption step. Therefore, from a thermodynamic standpoint, H_3O^+ hydrolysis is driven by adsorption of the H_3O^+ to the dimer. At this point it is important to note that these thermodynamic results say very little about the kinetics of hydrolysis. To understand the kinetics, it is necessary to also find transition states and calculate the activation energies for the reactions. With this in mind, the results of this study are consistent with those of Xiao and Lasaga (1994), in which H_3O^+ is shown to catalyze the hydrolysis reaction, primarily during the adsorption step.

	Reaction	ΔE
(a)	$H_6SiAlO_7(H_4C_4O_4)^{-3} \cdot H_2O \rightarrow H_4SiO_4 + H_8C_4AlO_8^{-3}$	-60.6
(b)	$H_6SiAlO_7(H_4C_4O_4)^{-3} \cdot H_2O + H_2O \rightarrow H_4SiO_4 + H_7C_4AlO_7^{-2} + H_2O \cdot OH^-$	-66.6
(c)	$H_7SiAlO_7(H_4C_4O_4)^{-2} \cdot H_2O + H_2O \rightarrow H_4SiO_4 + H_7C_4AlO_7^{-2} + H_2O \cdot H_2O$	50.9
(d)	$H_7SiAlO_7(H_4C_4O_4)^{-2} \cdot H_2O + H_2O \rightarrow H_4SiO_4 + H_8C_4AlO_8^{-3} + H_3O^+$	273.5

Table 4.10: Calculated zero-point corrected energies (kcal/mol) for hydrolysis reactions for dimer, succinate present.

	Reaction	ΔE
(a)	$H_6SiAlO_7^- + H_2O \cdot H_2O \rightarrow H_4SiO_4 + Al(OH)_4^- + H_2O$	29.9
(b)	$H_6SiAlO_7^- + H_2O \cdot H_3O^+ \rightarrow H_4SiO_4 + Al(OH)_3 + H_2O \cdot H_2O$	-77.8
(c)	$H_6SiAlO_7^- + C_2O_4^{-2} + H_2O \cdot H_2O \rightarrow H_4SiO_4 + H_3C_2AlO_7^{-2} + H_2O \cdot OH^-$	-3.4
(d)	$H_6SiAlO_7^- + C_2O_4^{-2} + H_2O \cdot H_3O^+ \rightarrow H_4SiO_4 + H_3C_2AlO_7^{-2} + H_2O \cdot H_2O$	-184.7
(e)	$H_6SiAlO_7^- + H_2C_3O_4^{-2} + H_2O \cdot H_2O \rightarrow H_4SiO_4 + H_5C_3AlO_7^{-2} + H_2O \cdot OH^-$	16.0
(f)	$H_6SiAlO_7^- + H_2C_3O_4^{-2} + H_2O \cdot H_3O^+ \rightarrow H_4SiO_4 + H_5C_3AlO_7^{-2} + H_2O \cdot H_2O$	-165.4
(g)	$H_6SiAlO_7^- + H_4C_4O_4^{-2} + H_2O \cdot H_2O \rightarrow H_4SiO_4 + H_7C_4AlO_7^{-2} + H_2O \cdot OH^-$	25.9
(h)	$H_6SiAlO_7^- + H_4C_4O_4^{-2} + H_2O \cdot H_3O^+ \rightarrow H_4SiO_4 + H_7C_4AlO_7^{-2} + H_2O \cdot H_2O$	-155.5

Table 4.11: Lowest calculated zero-point corrected energies (kcal/mol) for overall hydrolysis reactions.

Hydrolysis in Presence of Organic Anions

When an oxalate anion is chemisorbed to the aluminum site, hydrolysis by H_2O is exothermic. The hydrolysis step compensates for the endothermic adsorption step in this case, and the overall hydrolysis reaction is also exothermic. One result of these calculations is that a fivefold coordinated aluminum product complex is thermodynamically favored over the, perhaps expected, sixfold coordinated complex. This is the case for all organic acid anions.

When the organic anion is malonate, the hydrolysis step involving H_2O is also exothermic, and the ΔE is 3 kcal/mol larger than that for oxalate catalysis. However, in this case it is not enough to compensate for the endothermic anion adsorption, and the overall hydrolysis is endothermic. Remembering that thermodynamics say little about kinetics, this result is consistent with experimental results obtained by Furrer and Stumm (1986), where the dissolution rate of $\alpha - Al_2O_3$ is shown to be inversely correlated with chelating ring size. The difference between the overall hydrolysis reaction energies for the oxalate and malonate catalyzed cases is 19.4 kcal/mol.

The H_2O hydrolysis step with a succinate cation chemisorbed to the dimer is also exothermic. In fact, the reaction is 7.4 kcal/mol more exothermic than the case where oxalate is the anion. Despite this result, the endothermicity of the adsorption step makes the overall hydrolysis reaction endothermic, with a ΔE 29.3 kcal/mol larger than the oxalate case. Additionally, the overall hydrolysis ΔE is larger than that for malonate, preserving the trend observed by Furrer and Stumm (1986).

So, results of quantum mechanical calculations for H_2O hydrolysis in the presence of organic acid anions are completely consistent with the experimentally observed relationship between chelating ring size and dissolution rate. These quantum mechanical calculations also reveal how each step in the overall mechanism contributes to the overall reaction energy. In fact, the calculations show that the most important factor contributing to the differences in the energetics of the overall hydrolysis reactions is the step involving chemisorption of the carboxylic acid anion to the aluminum site.

Next, let us consider hydrolysis by H_3O^+ , catalyzed by each of the three anions. For both oxalate and malonate, the hydrolysis step is actually just the dissociation of a hydrogen bonded complex, because adsorption of both H_3O^+ and either oxalate or malonate ruptures the AlO_{br} bond. Adsorption of H_3O^+ and succinate, on the other hand, does not rupture this bond, so the hydrolysis step is a true hydrolysis reaction.

Dissociation of the hydrogen bonded complex is endothermic for both oxalate and malonate. However, adsorption of both H_3O^+ and the anions are exothermic, resulting in exothermic overall hydrolysis reactions for both oxalate and malonate. As for hydrolysis by H_2O , the relationship between chelating ring size and hydrolysis reaction energy is preserved, with hydrolysis involving malonate having a hydrolysis ΔE 19.3 kcal/mol greater than hydrolysis involving oxalate. However, the dissociation energies for the two cases are reversed from the overall hydrolysis energies. Dissociation of the complex containing malonate is 5.4 kcal/mol less than dissociation of the oxalate-bearing complex. Again, it is the step involving adsorption of the anion and rupturing of the AlO_{br} bond that is the most significant contributor to the overall reaction energy trend.

The hydrolysis step for H_3O^+ hydrolysis with succinate is also endothermic, and the energy of the hydrolysis step is comparable to but less than the dissociation energy of the oxalate and malonate cases (50.9 kcal/mol versus 62.7 kcal/mol and 57.3 kcal/mol, respectively). The ΔE for adsorption of succinate is greater than either oxalate or malonate, though, so the overall hydrolysis reaction energy is still consistent with the experimentally observed relationship between chelating ring size and dissolution rate.

While H_2O hydrolysis is only exothermic in the presence of oxalate, H_3O^+ hydrolysis is exothermic in the presence of any of the organic anions. This suggests that the effect of these organic ligands on the dissolution rate should be more pronounced at acidic pH than at neutral pH. One factor not considered in the calculations, though, is the speciation of the organic acid.

Cases involving singly and doubly protonate organics were not considered, although at lower pH's these would be the dominant species. The effect of additional protons on the organic ligands is an avenue that should be explored in future research.

4.4 Summary and Conclusions

Chemisorption of organic acid anions to the aluminum site in the H_6SiAlO_7 dimer results in a pronounced lengthening of the AlO_{br} bond. Adsorption of the organic ligands has a larger effect on the AlO_{br} bond than protonation of the bridging oxygen (H_3O^+ adsorption). The degree of lengthening does not correlate with the size of the chelating ring formed by the organic ligand and the aluminum atom. However, the adsorption energies for the organic acid anions correlates negatively with chelating ring size, consistent with the experimental results of Furrer and Stumm (1986).

The organic acid anions oxalate, malonate, and succinate reduce the energy of the overall hydrolysis reactions for hydrolysis of the H_6SiAlO_7 dimer by both H_2O and H_3O^+ relative to dimer hydrolysis in their absence. Hydrolysis by H_2O is exothermic in the presence of oxalate, and endothermic for both malonate and succinate. Hydrolysis by H_3O^+ is exothermic in the presence of each of the organic acid anions. In fact, adsorption of H_3O^+ and either oxalate or malonate ruptures the AlO_{br} bond, resulting in a hydrogen bonded complex consisting of silicic acid, water, and an aluminum complex. The aluminum complex is fivefold coordinated as opposed to sixfold coordinated, as might be expected.

For H_2O hydrolysis in the presence of organic acid anions, the lowering of overall hydrolysis reaction energy is due to the exothermic adsorption of the organic ligand. The actual hydrolysis step is endothermic in all three cases. For H_3O^+ hydrolysis, the exothermicity of the overall reaction is due to the adsorption of the the organic ligand and rupture of the AlO_{br} bond (for oxalate and malonate). Meanwhile, dissociation of the hydrogen bonded complex in the cases of oxalate and malonate, and hydrolysis of the AlO_{br} bond in the case of succinate is exothermic.

These quantum mechanical calculations are consistent with the experimentally observed relationship between dissolution rate and chelating ring size. However, unlike the experimental work, quantum mechanical calculations reveal much about the nature of and relationships between the el-

elementary reactions in the hydrolysis mechanism. Many previous workers have suggested that the organic acids promote aluminosilicates by complexing aluminum in solution and lowering its activity, thereby changing the saturation state of the solution to one more favorable to dissolution. The results presented here suggest that organic acids have a true catalytic effect on aluminosilicate dissolution, in that they lower the overall energy change for the hydrolysis reactions. In order to fully assess the role of organic acids in aluminosilicate dissolution, further work involving locating transition states and calculating activation energies for each of the reactions is needed.

Chapter 5

Organic Acids and Aluminosilicates II: Bonding

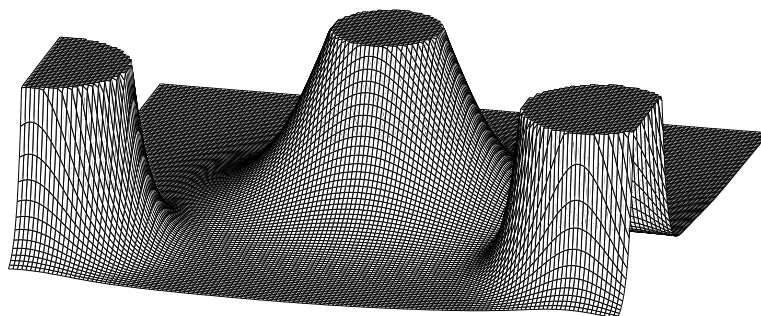
5.1 Introduction

Bader (1990, and references therein) fully develops a theory which establishes a quantum mechanical basis for the notion of atoms in molecules and their interactions. This theory applies ideas of classical field theory to the electron density, or charge, distribution, and uses properties of the local charge density as a foundation for a quantum theory of atoms in molecules. The charge density can be obtained from *ab-initio* quantum mechanics calculations of the type presented in the previous chapter. In this chapter, an attempt is made to ascertain the quantum mechanical nature of the effect of organic acid ligands on aluminosilicate dissolution. Since the AlO_{br} bond is of critical importance in this context, the analysis focuses on this bond. Some discussion of the bonding within the organic ligand is also included.

In the remaining subsections of this section, the important points from Bader's theory are summarized, to help orient the reader unfamiliar with Bader's theoretical work. All of this material is condensed from Bader (1990) and references therein.

5.1.1 Properties of the Electron Density Distribution

The most striking feature of the electron density distribution, $\rho(r)$, of a molecule is that local maxima exist only at the positions of the nuclei. In

Figure 5.1: H_6SiAlO_7 : $\rho(r)$ in $Si - O - Al$ plane

order to gain a deeper understanding of the topological features of $\rho(\mathbf{r})$, it is useful to examine the Hessian of $\rho(r)$.

The Hessian is a matrix of curvatures, defined as:

$$A_{i,j} = \frac{\partial^2 \rho}{\partial x_i \partial x_j} \quad (5.1)$$

where x_i and x_j are vectors in some coordinate system. Since the Hessian is real and symmetric, it is diagonalizable. The eigenvalues of the Hessian indicate the relative magnitudes of the principal curvatures, whose directions are given by the corresponding eigenvectors, or principal axes of curvature, which define a coordinate system.

A critical point in $\rho(\mathbf{r})$ is a point at which the gradient of $\rho(\mathbf{r})$ is zero. Critical points are classified by their *rank* and *signature*. The rank is the number of non-zero eigenvalues of the Hessian. The signature is the sum of the signs of the eigenvalues. For example, a critical point associated with a local minimum is classified as a (3,3) critical point, since all three eigenvalues of the Hessian are positive.

An interaction between two atoms is indicated by the presence of a critical point between them. Critical points associated with bonded interaction are typically (3,-1) critical points, or saddle points. The eigenvector corresponding to the positive eigenvalue at a bond critical point generates a pair of paths in the gradient vector field of $\rho(\mathbf{r})$. These two paths define the atomic

interaction line.

Since an electron has an infinite Coulomb interaction with a nucleus at zero separation, a cusp exists in $\rho(\mathbf{r})$ at the nuclear positions. Therefore, the nuclear positions are not true (3,-3) critical points, but for all practical purposes behave mathematically identical to local maxima.

The electron density distribution for the H_6SiAlO_7 dimer in the $Si - O - Al$ plane is shown in Figure 5.1 for illustration. In the figure, the charge density has been truncated to better illustrate the topological features. Note that maxima exist at the Si , Al , and O and nowhere else. There are critical points between $Si - O$ and $Al - O$, although they are difficult to see at the scale of the figure.

5.1.2 The Laplacian of the Electron Density

The trace, or sum of the diagonals, of the Hessian is invariant to rotation and defines the Laplacian:

$$\nabla^2\rho = \frac{\partial^2\rho}{\partial^2x} + \frac{\partial^2\rho}{\partial^2y} + \frac{\partial^2\rho}{\partial^2z} \quad (5.2)$$

For a scalar field, such as $\rho(\mathbf{r})$, the Laplacian indicates areas of local concentration and depletion. When the Laplacian at a point x is negative, the value of $f(x)$ is greater than the average of $f(x + \delta)$ and $f(x - \delta)$, indicating local concentration of charge density. In three dimensions, $\rho(\mathbf{r})$ is locally concentrated with respect to an infinitesimal sphere centered at r when the Laplacian is negative. Finally, $\rho(\mathbf{r})$ is maximally concentrated at points where the negative of the Laplacian exhibits a maximum, even though $\rho(r)$ may not necessarily exhibit a maximum at these points.

The *zero-flux surface condition* (Bader, 1990), an explanation of which is beyond the scope of this text, requires that the integral of the Laplacian must vanish over an atom, or quantum subsystem. The result is spherical nodes in $\nabla^2\rho(\mathbf{r})$, the number of which is related to the quantum shell structure of the atom. For each principal quantum shell, there is a pair of negative and positive regions in the Laplacian.

From this nodal structure, the valence-shell structure can be observed. In chemical interactions, the valence-shell charge distribution distorts yielding maxima in the Laplacian which correspond in number and relative position to Lewis and VSEPR electron pairs.

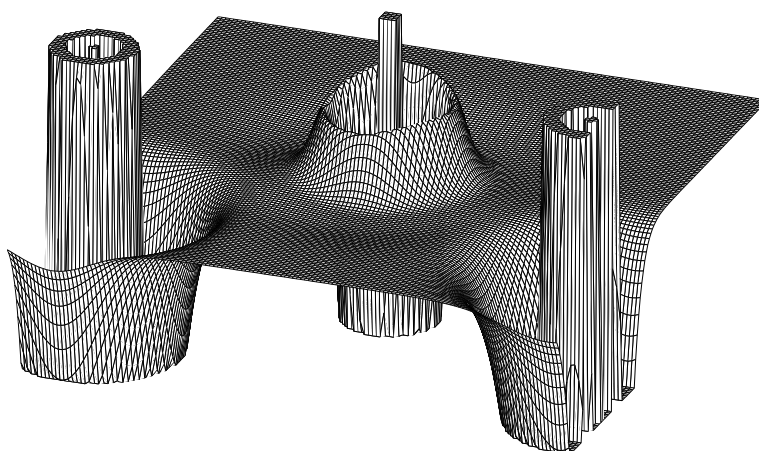


Figure 5.2: H_6SiAlO_7 : $-\nabla^2\rho$ in $Si-O-Al$ plane

Figure 5.2 presents the negative of the Laplacian of the H_6SiAlO_7 dimer in the $Si-O-Al$ plane for illustration. The figure shows $-\nabla^2\rho(\mathbf{r})$, as a convenience, so that areas of concentration are shown in the positive direction, and areas of depletion are negative. As before for the electron density distribution, the scale has been truncated so as not to obscure the important features. Note the nodal structure of each of the atoms. For the bridging oxygen, the single inner node corresponds to the 1s electrons. The outer, irregularly shaped node corresponds to the valence shell. Close inspection of the valence shell reveals local maxima between the oxygen and the silicon and aluminum atoms, and a “lone-pair” on the opposite side of the silicon and aluminum. Note that the valence shells of the aluminum and silicon are regions of charge depletion, consistent with the electropositive nature of these atoms. Also evident is the presence of two nodes of concentration in the inner shells of the aluminum and silicon, consistent with their status as third-row atoms.

5.1.3 Bond Critical Point Properties and Nature of Bonding

Properties of the Laplacian at the bond critical point provide insight into the nature of the chemical interaction. From the local expression of the virial theorem (Bader, 1990), areas of local charge concentration are regions where the potential energy dominates the local energy density, defined as:

$$H(\mathbf{r}_c) = G(\mathbf{r}_c) + \mathcal{V}(\mathbf{r}_c) \quad (5.3)$$

where $G(\mathbf{r}_c)$ is the kinetic energy and $\mathcal{V}(\mathbf{r}_c)$ is the potential energy. In these regions, the local energy density is negative. In areas of local charge depletion, the kinetic energy dominates the total energy, and the local energy density is positive. Therefore, the potential energy density can be used to define a range of interactions from shared to closed-shell. In addition, the eigenvalues of the Hessian at a bond critical point indicate whether contraction of the charge density perpendicular to the atomic interaction line or expansion parallel to the interaction line dominates. The result of this tension can be used to further distinguish the nature of an atomic interaction.

For interactions where the Laplacian at the bond critical point is negative and large in magnitude, and where ρ is also large, a concentration of charge density is indicated. This type of interaction corresponds to the more familiar shared, or covalent, interaction. In shared interactions, the critical point and the valence shells of both atoms lie in the same contiguous region of negative Laplacian. The potential energy contribution dominates the total energy at the bond critical point, leading to a negative local energy density.

Closed-shell, or ionic, interactions are characterized by a small $\rho(\mathbf{r}_c)$ and positive Laplacian at $\mathbf{r}_c$. Here, the Laplacian is positive over the region of interaction, and the regions where the Laplacian is negative are confined to the atomic basins and, excluding polarization effects, identical to the case for the free atoms. The kinetic energy is dominant at the bond critical point for closed-shell interactions, leading to a positive local energy density.

A hydrogen bond is indicated when the interaction between the hydrogen and the acid fragment is typical of a shared interaction and the interaction between the hydrogen and the base is typical of a closed-shell interaction. In this case, as for shared and closed-shell interactions, the bond critical point will be found relatively far from any nodal surfaces in the Laplacian.

In some cases, however, the bond critical point is located quite close to a

nodal surface. The Laplacian of the atoms on either side of the interatomic surface, which is perpendicular to the interaction line at the critical point, display regions of opposite sign. This indicates an interaction which is intermediate in nature, and a complete spectrum of interactions between shared and closed-shell exists.

Hill et al. (1997) derived an expression for the electronegativities of oxide and nitride molecules containing main group M atoms:

$$\chi_M = 1.31F_M^{-0.23} \quad (5.4)$$

In this expression,

$$F_M = \frac{r_b(X)}{N_M \times \rho(\mathbf{r}_c)} \quad (5.5)$$

where $r_b(X)$ is the bonded radius of the anion X and N_M is the number of valence electrons on M (Boyd and Edgecombe, 1988).

5.2 H_2O and H_3O^+ Adsorption

talk about the hydrogen bonding between water and dimer.

5.3 Hydrolysis of the AlO_{br} Bond

Talk about bcp properties for alo bond in various complexes. Present and describe salient features of figures of the laplacian of the electron density distribution in the Si-O-Al plane for the Al-O bond.

5.3.1 Effect of H_3O^+ Adsorption

Discuss effect on alo and sio bonds. Focus on alo bond.

5.3.2 Effect of Chelating Ring Size

Discuss implications for bonding. Compare with proton adsorption.

5.3.3 Evidence for Local Nature of Bonding

Present some results for Si-O bond and discuss how they indicate very local nature of bonding in aluminosilicates, supporting use of cluster model

5.4 Effect of Chelation on Organic Acid Anions

Describe methods... what is meant by various types of calculations on the strained versus unstrained anions.

5.4.1 Anion Total Strain Energy

tabulate and discuss energy differences between unstrained (fully optimized) and strained (chelated - the Al) ligands

5.4.2 Bonding in the Anion

need to do some more Bader-type calculations for all the CO and CC bonds within the various organic anions, both bonded (strained) and isolated (unstrained)

Compare the results of these calculations and note interesting differences

Possibly show some laplacian maps of CC and/or CO bonds to graphically illustrate the results

5.5 Conclusions

Summarize hypothesis for dissolution effects. Suggest implications for relative K_d's of the different chelates. Implications for other polydentate chelates.

Appendix A

Monte Carlo Model Source Code

A.1 actions.h

```
/* actions.h: exports functions for doing state transitions */

#ifndef actions_h
#define actions_h

#include "lattice.h"
#include "evtlist.h"

/* doEvent: randomly pick event and update state accordingly
 *          return time increment for the event */
float doEvent(Lattice lattice, eventList el);

/* doReaction: update state of site and its nbrs based on reaction rxn */
void doReaction(Lattice lattice, int site, int rxn);

#endif
```

A.2 actions.c

```
#include <stdio.h>
#include "common.h"
#include "actions.h"
#include "reactions.h"
#include "ran2.h"
#include "myerr.h"
#include "math.h"
```

```

/* doEvent: randomly pick event and update state accordingly
 *      return time increment for the event */
float doEvent(Lattice lattice, eventList el)
{
    float ratesum, partsum, eps, dt;
    struct event *e;

    e = el;
    ratesum = 0;

    while (e->next) {
        ratesum += e->rate;
        e = e->next;
    }
    ratesum += e->rate;

    if (ratesum == 0.0)
        die("ratesum is zero!");
    dt = - log(ran2()) / ratesum;

    eps = ran2();
    partsum = 0;
    e = el;
    while (e->next) {
        partsum += e->rate / ratesum;
        if (eps <= partsum)
            break;
        else
            e = e->next;
    }

    doReaction(lattice, e->site, e->rxn);
    return dt;
}

/* doReaction: update state of site and its nbrs based on reaction rxn */
void doReaction(Lattice lattice, int site, int rxn)
{
    switch (rxn) {
        case 0: r0(lattice, site); break;
        case 1: r1(lattice, site); break;
        case 2: r2(lattice, site); break;
        case 3: r3(lattice, site); break;
        case 4: r4(lattice, site); break;
        case 5: r5(lattice, site); break;
        case 6: r6(lattice, site); break;
        case 7: r7(lattice, site); break;
        case 8: r8(lattice, site); break;
        case 9: r9(lattice, site); break;
        case 10: r10(lattice, site); break;
        case 11: r11(lattice, site); break;
        case 12: r12(lattice, site); break;
        case 13: r13(lattice, site); break;
    }
}

```



```

    case 14: r14(lattice, site); break;
    case 15: r15(lattice, site); break;
    case 16: adsorbAl(lattice, site); break;
    case 17: adsorbAl(lattice, site); break;
    case 18: adsorbSi(lattice, site); break;
    case 19: adsorbSi(lattice, site); break;
    case 20: desorbAl(lattice, site);
             clusters(lattice);
             break;
    case 21: desorbSi(lattice, site);
             clusters(lattice);
             break;
    case 22: desorbSi(lattice, site);
             clusters(lattice);
             break;
    case 23: desorbAl(lattice, site);
             clusters(lattice);
             break;
    default: diffuse(lattice, site, rxn);
             clusters(lattice);
             break;
}
}

```

A.3 bfsearch.h

```

/* bfsearch.h: exports routine to do breadth-first search
 *              on a lattice */

#ifndef bfsearch_h
#define bfsearch_h

#include "lattice.h"

#define WHITE 0
#define GRAY 1
#define BLACK 2

/* BFS: perform breadth-first search to mark all "solid" nodes
   start in a known occupied node, s, and discover all
   occupied neighbors. (discovered => BLACK)
   At the end, undiscovered, occupied nodes are unbonded
   clusters. */
void BFS(Lattice lattice, int s, int n);

#endif

```

A.4 bfsearch.c

```

#include <stdlib.h>
#include "myerr.h"

```

```

#include "common.h"
#include "bfsearch.h"
#include "lattice.h"

/* Qnode: node of a queue used in BFS for clusters */
struct Qnode {
    int val;
    struct Qnode *next;
};

static int deQ(struct Qnode **q);
static void enQ(struct Qnode **q, int val);
static int headQ(struct Qnode *q);

/* BFS: perform breadth-first search to mark all "solid" nodes
start in a known occupied node and discover all
occupied neighbors. (discovered => BLACK)
At the end, undiscovered, occupied nodes are unbonded
clusters. */
void BFS(Lattice lattice, int s, int n)
{
    int u, v, i;
    struct Qnode *Q = NULL;

    for (u = 0; u < n; u++)
        lattice[u].color = WHITE;
    lattice[s].color = GRAY;
    enQ(&Q, s);
    while (Q) {
        u = headQ(Q);
        for (i = 0; i < 6; i++) {
            v = lattice[u].nbr[i];
            if (!EXISTS(v) ||          /* no ith neighbor */
                !ISOCC(lattice[v])) /* unoccupied */
                continue;
            else if (lattice[v].color == WHITE) {
                lattice[v].color = GRAY;
                enQ(&Q, v);
            }
        }
        deQ(&Q);
        lattice[u].color = BLACK;
    }
}

/* enQ: add val to tail of queue */
void enQ(struct Qnode **q, int val)
{
    struct Qnode *new;

    if (!(new = (struct Qnode *) malloc (sizeof(struct Qnode))))
        die("malloc failed");
    new->val = val;
    if (NULL == *q) {
        new->next = new;          /* single el't; tail = head */
    } else {
        new->next = (*q)->next;  /* tail points to head */
    }
}

```

```

    (*q)->next = new;          /* old tail points to new tail */
}
*q = new;                     /* queue points to new tail */
}

/* headQ: return val for head of queue */
int headQ(struct Qnode *q)
{
    if (NULL == q) {          /* empty queue */
        return -1;
    } else {
        return q->next->val;
    }
}

/* deQ: remove head of queue */
int deQ(struct Qnode **q)
{
    int val;
    struct Qnode *head;

    if (NULL == *q)          /* empty queue */
        return -1;
    head = (*q)->next;
    val = head->val;
    if (*q == head) {        /* was only el't in queue, so */
        *q = NULL;          /* make queue empty */
    } else {
        (*q)->next = head->next;
    }
    free(head);
    return val;
}

```

A.5 common.h

```

/*****
* Here is the site classification scheme:
*
*   EDGE   used to identify site at edge of lattice
*
*   Al sites
*   -----
*   100    empty
*   101    Al(OH,H2O)0
*   102    Al(OH,H2O)1
*   103    Al(OH,H2O)2
*   104    Al(OH,H2O)3
*   105    Al(OH,H2O)4
*   106    Al(OH,H2O)5
*   107    Al(OH,H2O)6
*****/

```

```

* 199 Si(OH)4
*
* Si sites
* -----
* 200 empty
* 201 Si(OH)0
* 202 Si(OH)1
* 203 Si(OH)2
* 204 Si(OH)3
* 205 Si(OH)4
* 299 Al(OH,H2O)6
*
* Si-O-Si type 0 sites
* -----
* 300 empty
* 301 Si-O-Si
* 302 Si-OH HO-Si
* 303 Si-OH
*
* Si-O-Al2 type 0 sites
* -----
* 400 empty
* 401 Si-O-Al2
* 402 Si-OH HO-Al2
* 403 Si-OH HO-Al H2O-Al
* 404 HO-Al2
* 405 HO-Al H2O-Al
* 406 Si-OH-Al
* 407 Si-OH HO-Al
* 408 Si-OH
* 409 HO-Al
* 410 Si-OH-Al HO-Al
*
* Al-OH-Al type 0(H) sites
* -----
* 500 empty
* 501 Al-OH-Al
* 502 Al-OH H2O-Al
* 503 Al-OH
*
*****
* Within this scheme, there are 16 possible reactions (not
* including adsorption/desorption), with some duplicated effort:
*
* R0. 301 --> 302 Si-O-Si + H2O = Si-OH + Si-OH
* R1. 302 --> 301
* R2. 401 --> 402 Si-O-Al2 + H2O = Si-OH + Al-OH-Al
* R3. 402 --> 401
* R4. 401 --> 410 Si-O-Al2 + H2O = Si-OH-Al + Al-OH
* R5. 410 --> 401
* R6. 402 --> 403 Si-OH + Al-OH-Al + H2O = Si-OH + Al-OH + Al-H2O
* R7. 403 --> 402
* R8. 410 --> 403 Si-OH-Al + Al-OH + H2O = Si-OH + Al-OH + Al-H2O
* R9. 403 --> 410
* R10. 406 --> 407 Si-OH-Al + H2O = Si-OH + Al-H2O
* R11. 407 --> 406
* R12. 404 --> 405 Al-OH-Al + H2O = Al-OH + Al-H2O

```

```

* R13. 405 --> 404
* R14. 501 --> 502   Al-OH-Al + H2O = Al-OH + Al-H2O
* R15. 502 --> 501
*
*
* Transitions possible upon adsorption:
*
*   1. 100 --> 107 (Al)
*   2. 200 --> 205 (Si)
*   3. 300 --> 303 (Si)
*   4. 303 --> 302 (Si)
*   5. 400 --> 408 (Si)
*   6. 400 --> 409 (Al)
*   7. 404 --> 402 (Si)
*   8. 405 --> 403 (Si)
*   9. 406 --> 410 (Al)
*  10. 407 --> 403 (Al)
*  11. 408 --> 407 (Al)
*  12. 409 --> 405 (Al)
*  13. 409 --> 407 (Si)
*  14. 500 --> 503 (Al)
*  15. 503 --> 502 (Al)
*
* Transitions possible upon desorption:
*
*   1. 107 --> 100 (Al)
*   2. 205 --> 200 (Si)
*   3. 303 --> 300 (Si)
*   4. 302 --> 303 (Si)
*   5. 402 --> 404 (Si)
*   6. 403 --> 405 (Si)
*   7. 410 --> 406 (Al)
*   8. 403 --> 407 (Al)
*   9. 407 --> 408 (Al)
*  10. 405 --> 409 (Al)
*  11. 407 --> 409 (Si)
*  12. 409 --> 400 (Al)
*  13. 408 --> 400 (Si)
*  14. 502 --> 503 (Al)
*  15. 503 --> 500 (Al)
*
* Next-neighbor state classification scheme (index, x, y):
*
* Adsorption:
*
* 100's:  x = (double 500 occupied) ? 1 : 0
*         y = number of occupied 400's
*         (0 - 2)
*         index = x * 3 + y
*
* 200's:  x = (400 occupied) ? 1 : 0
*         y = number of occupied 300's
*         (0 - 3)
*         index = x * 4 + y
*
* Desorption:
*

```

```

* 100's: x = (double 500 occupied) ? 1 : 0
*         y = number of occupied 400's
*         (0 - 2)
*         index = x * 3 + y
*
* 200's: x = (400 occupied) ? 1 : 0
*         y = number of occupied 300's
*         (0 - 3)
*         index = x * 4 + y
*
* Hydrolysis (and reverse):
*
* 300's: x = number of (402, 403, 406, 407, 408, 410) on Si
*         (0 - 2)
*         y = number of (302, 303) on Si
*         (0 - 4)
*         index = x * 5 + y
*
* 400's: x = number of (302, 303) on Si
*         (0 - 3)
*         y = number of (403, 405, 406, 407, 409, 410, 502, 503) on Al
*         (0 - 9)
*         index = x * 10 + y
*
* 500's: x = (other bridge hydrolyzed) ? 1 : 0
*         y = number of (403, 405, 406, 407, 409, 410, 502, 503) on Al
*         (0 - 8)
*         index = x * 9 + y
*
*
*****/

#ifndef common_h
#define common_h

#ifndef FALSE
#define FALSE 0
#define TRUE !FALSE
#endif

#endif

```

A.6 envrn.h

```

/* envrn.h: functions for checking the environment of a site */

#ifndef envrn_h
#define envrn_h

#include "lattice.h"

int checkEnv(Lattice lattice, int site);
int isActive(int site, Lattice lattice, int rxn);

```

```
#endif
```

A.7 envrn.c

```
#include <stdio.h>
#include "common.h"
#include "envrn.h"
#include "myerr.h"
#include "rxnlist.h"

static int check100(Lattice lattice, int site);
static int check200(Lattice lattice, int site);
static int check300(Lattice lattice, int site);
static int check400(Lattice lattice, int site);
static int check500(Lattice lattice, int site);

/* checkEnv: returns index which tells environment for determination
 *           of appropriate rate constant */
int checkEnv(Lattice lattice, int site)
{
    int status;
    char msg[100];

    switch (lattice[site].state / 100) {
        case 1:
            if (lattice[site].state == 100)
                status = 0;
            else
                status = check100(lattice, site);
            break;
        case 2:
            if (lattice[site].state == 200)
                status = 0;
            else
                status = check200(lattice, site);
            break;
        case 3: status = check300(lattice, site); break;
        case 4:
            if (lattice[site].state == 404 || lattice[site].state == 405)
                status = check500(lattice, site);
            else
                status = check400(lattice, site);
            break;
        case 5: status = check500(lattice, site); break;
        default:
            status = -1;
            sprintf(msg, "invalid environment: site %d, state %d", site,
                    lattice[site].state);
            die(msg);
            break;
    }
    return status;
}
```

```
}

```

```
/* check100: returns index for environment of an Al */
int check100(Lattice lattice, int site)
{
    int i, nbr, x, y;

    /* x = double 500 occupied ?
       * y = number of occupied 400's (0 - 2) */

    x = y = 0;
    for (i = 0; i < 6; i++) {
        nbr = lattice[site].nbr[i];
        if (lattice[nbr].state == EDGE)
            die("ran into lattice edge in check100");
        switch (lattice[nbr].state) {
            case 502: x++; break;
            case 403: case 405: case 407: case 409: case 410: y++; break;
            default: break;
        }
    }
    return ((x + y) / 2);
}

```

```
/* check200: returns index for environment of an Si */
int check200(Lattice lattice, int site)
{
    int i, nbr, x, y;

    /* x = 400 occupied ?
       * y = number of occupied 300's (0 - 3) */

    y = 0;
    x = 1;
    for (i = 0; i < 4; i++) {
        nbr = lattice[site].nbr[i];
        if (lattice[nbr].state == EDGE)
            die("ran into lattice edge in check200");
        switch (lattice[nbr].state) {
            case 408: x = 0; break;
            case 302: y++; break;
            default: break;
        }
    }
    return (x + y);
}

```

```
/* check300: returns index for environment of Si-O-Si type 0 */
int check300(Lattice lattice, int site)
{
    int si1, si2, sio1, sio2, x, y;

```



```

/* x = number of broken 400s (0 - 2)
 * y = number of broken 300s (0 - 4) */

x = y = 0;
si1 = lattice[site].nbr[0];
sio1 = lattice[si1].nbr[0];      /* Si-O-Al is Si nbr[0] */
si2 = lattice[site].nbr[1];
sio2 = lattice[si2].nbr[0];
if (si1 < 0 || si2 < 0 || sio1 < 0 || sio2 < 0 ||
    lattice[si1].state == EDGE || lattice[si2].state == EDGE ||
    lattice[sio1].state == EDGE || lattice[sio2].state == EDGE)
    die("ran into lattice edge in check300");

if (lattice[sio1].state == 402 ||      /* 400s */
    lattice[sio1].state == 403 ||
    lattice[sio1].state == 407 ||
    lattice[sio1].state == 408)
    x++;
if (lattice[sio2].state == 402 ||      /* 400s */
    lattice[sio2].state == 403 ||
    lattice[sio2].state == 407 ||
    lattice[sio2].state == 408)
    x++;

y = (lattice[si1].state - 201) +
    (lattice[si2].state - 201) - x; /* 300s */
if (lattice[site].state > 301)
    y -= 2;

return (5 * x + y);
}

/* check400: returns index for environment of Si-O-Al2 type 0 */
int check400(Lattice lattice, int site)
{
    int i, x, y, al1, al2, si, o1, p;

    /* x = number of broken 300s (0 - 3)
     * y = number of broken 400s and 500s (0 - 9) */

    x = y = 0;
    al1 = lattice[site].nbr[0];
    al2 = lattice[site].nbr[1];
    si = lattice[site].nbr[2];
    p = lattice[site].pair;
    if (al1 < 0 || al2 < 0 || si < 0 || (p = lattice[site].pair) < 0 ||
        lattice[al1].state == EDGE || lattice[al2].state == EDGE ||
        lattice[si].state == EDGE || lattice[p].state == EDGE)
        die("ran into lattice edge in check400");

    for (i = 1; i < 3; i++) {          /* 300s */
        if ((o1 = lattice[si].nbr[i]) < 0) /* 400 site is Si nbr[0] */
            die("ran into lattice edge");
        if (lattice[o1].state > 301)

```

```

        x++;
    }

    y = (lattice[al1].state - 101) + (lattice[al2].state - 101);
    if (lattice[site].state == 403 ||      /* subtract out the site */
        lattice[site].state == 405)
        y -= 2;
    if (lattice[site].state == 407 ||
        lattice[site].state == 410 ||
        lattice[p].state == 502)
        y--;

    if (lattice[p].state == 502)
        y--;

    if (lattice[site].state == 406 || lattice[site].state == 407)
        return (x * 5 + y);
    else
        return (x * 10 + y);
}

```

```

/* check500: returns index of environment for Al-OH-Al type 0 */
int check500(Lattice lattice, int site)
{
    int al1, al2, x, y, p;

    /* x = 0 if normal double bridge else 1
       * y = number of broken 500s (0 - 9) */

    y = 0;
    al1 = lattice[site].nbr[0];
    al2 = lattice[site].nbr[1];
    if ((p = lattice[site].pair) < 0 || al1 < 0 || al2 < 0 ||
        lattice[al1].state == EDGE || lattice[al2].state == EDGE ||
        lattice[p].state == EDGE)
        die("ran into lattice edge in check500");

    if (lattice[p].state == 502 ||
        lattice[p].state == 403 ||
        lattice[p].state == 405 ||
        lattice[p].state == 410)
        x = 1;
    else
        x = 0;

    y = (lattice[al1].state - 101) + (lattice[al2].state - 101);
    if (lattice[p].state == 502 ||      /* subtract out doubly
                                         counted bridges */
        lattice[p].state == 403 ||
        lattice[p].state == 405)
        y--;
    if (lattice[site].state == 502 || /* subtract out site */
        lattice[site].state == 405)
        y -= 2;
}

```

```

    return (x * 9 + y);
}

/* isActive: determine whether site is active to do reaction rxn
 *          return TRUE or FALSE */
int isActive(int site, Lattice lattice, int rxn)
{
    int result, nbr, nbr2, i, j;

    result = FALSE;
    if (rxn < NHYD && rxn % 2 == 0) { /* hydrolysis
                                      * only allowed
                                      * at "surface" */
        for (i = 0; (nbr = lattice[site].nbr[i]) >= 0 && !result; i++)
            for (j = 0; (nbr2 = lattice[nbr].nbr[j]) >= 0 &&
                      j < 6 && !result; j++)
                switch (lattice[nbr2].state) {
                    case 303: case 404: case 405: case 406: case 408: case 409:
                        result = TRUE; break;
                    default: break;
                }
    } else if (rxn < NHYD) { /* reverse hydrolysis
                             * always allowed*/
        result = TRUE;
    } else if (rxn == 16 || rxn == 19) { /* adsorption to correct site */
        result = FALSE; /* must be at least one occupied
                        * nbr */
        for (i = 0; (nbr = lattice[site].nbr[i]) >= 0 && i < 6; i++)
            if (ISOCC(lattice[nbr]) && !ISEGE(lattice[nbr]))
                result = TRUE;
    } else if (rxn == 20 || rxn == 22) { /* desorption */
        result = TRUE;
    } else { /* diffusion */
        result = FALSE;
    }
    return result;
}

```

A.8 evtlist.h

```

/* evtlist.h: exports structs and functions for the event list */

#ifndef evtlist_h
#define evtlist_h

#include "lattice.h"
#include "sim.h"
#include "rxnlist.h"

```

```

/* event: Uniquely identifies an event which may occur. */
struct event {
    int    site,      /* site where reaction happens */
          rxn;        /* reaction which happens
                       * for diffusion, this will contain OFFSET + index
                       * of target for jump */
    float rate;        /* specific rate for this event (e.g., adsorption at
                       * a specific site) */
    struct event *next;
};

typedef struct event *eventList;

eventList new_evtList(Lattice l, reactionList rl, int nsites);
void free_evtList(eventList el);

#endif

```

A.9 evtlist.c

```

#include <stdlib.h>
#include <stdio.h>
#include "common.h"
#include "evtlist.h"
#include "envrn.h"
#include "myerr.h"
#include "rxnlist.h"

eventList new_evtList(Lattice l, reactionList rl, int nsites)
{
    eventList el = NULL;
    struct event *e;
    int s, i, env, lo, hi;
    char msg[100];

    for (s = 0; s < nsites; s++) { /* build event list */
        if ((l[s].state % 100 == 0) && l[s].state > 200) {
            continue;
        } else if (l[s].state > 500) {
            lo = N400; hi = NHYD;
        } else if (l[s].state > 400) {
            lo = N300; hi = N400;
        } else if (l[s].state > 300) {
            lo = 0; hi = N300;
        } else {
            lo = NHYD; hi = NRXN;
        }
        for (i = lo; i < hi; i++) {
            if (l[s].state == rl[i].reactant /* matches reactant
                                             * and is active? */
                && isActive(s, l, i)) {
                if (!(e = (struct event *) malloc (sizeof(struct event))))

```

```

        die("malloc failed");
    e->next = el;
    e->site = s;
    e->rxn = i;
    env = checkEnv(l, s);
    if (env < 0 || env >= rl[i].nrates) {
        sprintf(msg,
            "invalid environment: site %d state %d env %d nrates %d",
            s, l[s].state, env, rl[i].nrates);
        die(msg);
    }
    e->rate = rl[i].rate[env];
    el = e;
}
}
}
return el;
}

void free_evtList(eventList el)
{
    struct event *e;

    while(el) {
        e = el->next;
        free(el);
        el = e;
    }
}

```

A.10 futil.h

```

/* futil.h: exports functions for dealing with file I/O */

#ifndef futil_h
#define futil_h

#include <stdio.h>

void eatComment(FILE *f, char flag);
FILE *openFile(const char *name, const char *mode);
int closeFile(FILE *f);

#endif

```

A.11 futil.c

```

#include <ctype.h>
#include "common.h"
#include "futil.h"
#include "myerr.h"

/* eatComment: advance file pointer to beginning of next line
 *          flag is first character of all comments          */
void eatComment(FILE *f, char flag)
{
    int i;
    char c;

    i = TRUE;
    while (i) {
        /* do this for multiline comments */
        while (isspace(c = fgetc(f))) /* eat leading whitespace */
            ;
        if (c == flag) {
            while ((c = fgetc(f)) != '\n') /* comment, read to end of line */
                ;
        } else {
            ungetc(c, f);
            i = FALSE;
        }
    }
}

/* openFile: try to open a file die if unsuccessful */
FILE *openFile(const char *name, const char *mode)
{
    FILE *f;
    char msg[100];

    if (!(f = fopen(name, mode))) {
        sprintf(msg, "could not open %s with mode %s", name, mode);
        die(msg);
    }
    return f;
}

int closeFile(FILE *f)
{
    return (fclose(f));
}

```

A.12 lattice.h

```

/* lattice.h: exports struct for a lattice site and functions
 *          for changing or obtaining info about a lattice */

```

```

#ifndef lattice_h
#define lattice_h

#include "ucell.h"
#include "sim.h"

#define EDGE 9
#define WRONG 99
#define AL 1
#define SI 2
#define SIOSI 3
#define SIOAL 4
#define ALOHAL 5
#define OFFSET 100

/* macros */
#define ISOCC(A) ((A).state % 100 > 0)
#define EXISTS(A) ((A) >= 0)
#define ISEEDGE(A) ((A).state == EDGE)
#define TYPE(A) ((A).state / 100)
#define ISSI(A) (TYPE(A) == SI)
#define ISAL(A) (TYPE(A) == AL)
#define ISOX(A) ((A).state > 299)
#define WRONGCATION(A) ((A).state % 100 == WRONG)

/* latticeSite: Uniquely identifies a lattice site and describes
 * its current state. */
struct latticeSite {
    int a, b, /* unit cell coordinates (for single sheet) */
        n, /* site within unit cell */
        state, /* current state */
        color, /* for BFS for clusters, WHITE, GRAY, BLACK */
        pair, /* for 400s and 500s, other part of double bridge */
        lostal, /* for 400s, when lose one al which it is */
        nbr[6]; /* neighbor list */
};
typedef struct latticeSite *Lattice;

void clusters(Lattice lattice);
int countNbrs(Lattice l, int s);
void findPairs(Lattice l);
void free_lattice(Lattice l);
int getNeighbor(Lattice l, unitCell c, int i, int j);
int getNsites(void);
Lattice makeLattice(unitCell uc);
void populateSolid(Lattice lattice);
void terminateLattice(Lattice lattice);
void terminateSurface(Lattice lattice);
void getDim(int *a, int *b);

#endif

```

A.13 lattice.c

```

#include <stdlib.h>
#include <stdio.h>
#include "futil.h"
#include "myerr.h"
#include "bfsearch.h"
#include "common.h"
#include "lattice.h"
#include "rxnlist.h"

static int Acells;    /* number of cells in a direction */
static int Bcells;    /* number of cells in b direction */
static int Surfplane; /* 0 for ac surface, 1 for bc surface */
static int Nsites;    /* total number of sites in simulation */

/* clusters: check for and remove any unattached clusters */
void clusters(Lattice lattice)
{
    int i;

    BFS(lattice, 0, Nsites);
    for (i = 0; i < Nsites; i++) {
        if (lattice[i].color == WHITE &&
            lattice[i].state != 9)
            lattice[i].state = (int) (lattice[i].state / 100) * 100;
        else if (lattice[i].color == GRAY) {
            printf("bad color at %d %d\n", i, lattice[i].state);
            die("error in clusters");
        }
    }
}

/* countNbrs: returns actual number of neighbors
 *           (for lattice edge detection) */
int countNbrs(Lattice lattice, int site)
{
    int i;

    for (i = 0; lattice[site].nbr[i] >= 0 && i < 6; i++)
        ;
    return i;
}

/* findPairs: sets up the double bridge information */
void findPairs(Lattice lattice)
{
    int j, al1, al2, al3, al4, o1, o2, found;

    for (o1 = 0; o1 < Nsites; o1++) {

```



```

    if (lattice[o1].state < 400 || lattice[o1].pair >= 0)
        continue;
    al1 = lattice[o1].nbr[0];
    al2 = lattice[o1].nbr[1];
    if (al1 == -1 || al2 == -1)
        continue;
    found = FALSE;
    for (j = 0; j < 6 && !found; j++) {
        if ((o2 = lattice[al1].nbr[j]) < 0) /* no neighbor, must be edge */
            found = TRUE;
        else {
            al3 = lattice[o2].nbr[0];
            al4 = lattice[o2].nbr[1];
            if ((al1 == al3 || al1 == al4) &&
                (al2 == al3 || al2 == al4)) {
                found = TRUE;
                lattice[o1].pair = o2;
                lattice[o2].pair = o1;
            }
        }
    }
}
}
}
}

```

```

void free_lattice(Lattice l)
{
    free(l);
}

```

```

void getDim(int *a, int *b)
{
    *a = Acells;
    *b = Bcells;
}

```

```

/* getNeighbor: determine lattice index of nbr j for site i
 *
 * depends upon index generation scheme in makeLattice */
int getNeighbor(Lattice l, unitCell c, int i, int j)
{
    int a, b, nbr, npos;

    npos = getNpos();
    a = l[i].a + c[l[i].n].nbr[j].a;
    b = l[i].b + c[l[i].n].nbr[j].b;
    if (c[l[i].n].nbr[j].n < 0) { /* no jth neighbor */
        nbr = c[l[i].n].nbr[j].n;
    } else if (((a == Acells || a < 0) && Surfplane) ||
               ((b == Bcells || b < 0) && !Surfplane)) {
        nbr = -1;
    } else {
        if (a >= Acells) /* boundary conditions - a */
            a = 0;
        else if (a < 0)
            a = Acells - 1;
    }
}

```

```

        if (b >= Bcells)                /* boundary conditions - b */
            b = 0;
        else if (b < 0)
            b = Bcells - 1;
        nbr = a * Bcells * npos + b * npos + c[l[i].n].nbr[j].n;
    }
    return nbr;
}

int getNsites(void)
{
    return Nsites;
}

/* makeLattice: repeat unit cell to make the lattice and
 *              initialize state of sites
 *              index generation scheme important for getNeighbor */
Lattice makeLattice(unitCell c)
{
    int a, b, n, i, j, npos;
    FILE *f;
    Lattice lattice;

    f = openFile("data.lattice","r");
    fscanf(f, " %d %d %d", &Acells, &Bcells, &Surfplane);
    closeFile(f);

    npos = getNpos();
    Nsites = Acells * Bcells * npos;
    lattice = (Lattice) malloc (sizeof(*lattice) * Nsites);
    for (a = 0; a < Acells; a++) {
        for (b = 0; b < Bcells; b++) {
            i = a * Bcells * npos + b * npos;
            for (n = 0; n < npos; n++) {
                lattice[i].a = a;
                lattice[i].b = b;
                lattice[i].n = n;
                lattice[i].state = c[n].state;
                lattice[i].pair = -1;
                lattice[i].lostal = -1;
                for (j = 0; j < 6; j++) {
                    lattice[i].nbr[j] = getNeighbor(lattice, c, i, j);
                }
                i++;
            }
        }
    }
    return lattice;
}

```

```

/* populateSolid: update state of sites which start as part of the solid
 *
 *      as determined by input parameters */
void populateSolid(Lattice lattice)
{
    int z, x, n, i, top, len, npos;
    float dms, dma, frac;
    char s[20];

    npos = getNpos();
    getChem(&dms, &dma);
    if (dms + dma > 0.5) {
        frac = 0.3;
        sprintf(s, "supersaturated");
    } else if (dms + dma < -0.5) {
        frac = 0.7;
        sprintf(s, "undersaturated");
    } else {
        frac = 0.5;
        sprintf(s, "near equilibrium");
    }
    top = (Surfplane ? (Acells * frac) : (Bcells * frac));
    printf("Detected %s conditions -- filling %d unit cells\n", s, top);
    len = (Surfplane ? Bcells : Acells);
    for (z = 0; z < top; z++) {
        for (x = 0; x < len; x++) { /* if bc, then x = b else x = a */
            i = (Surfplane ? (z * Bcells * npos + x * npos) :
                (x * Bcells * npos + z * npos));
            for (n = 0; n < npos; n++) {
                lattice[i].state++;
                i++;
            }
        }
    }
}

```

```

/* terminateLattice: set to EDGE state of any atoms at very top or
 *
 *      bottom of the lattice */
void terminateLattice(Lattice lattice)
{
    int top, len, x, i, n, npos;

    npos = getNpos();
    top = (Surfplane ? (Acells - 1) : (Bcells - 1));
    len = (Surfplane ? Bcells : Acells);
    for (x = 0; x < len; x++) { /* if bc, then x = b else x = a */
        i = (Surfplane ?
            (top * Bcells * npos + x * npos) :
            (x * Bcells * npos + top * npos));
        for (n = 0; n < npos; n++, i++)
            if (getNumNbrs(lattice[i].state) != countNbrs(lattice, i))
                lattice[i].state = EDGE;
        i = (Surfplane ? (x * npos) :
            (x * Bcells * npos));
        for (n = 0; n < npos; n++, i++)

```

```

        if (getNumNbrs(lattice[i].state) != countNbrs(lattice, i))
            lattice[i].state = EDGE;
    }
}

/* terminateSurface: set state of 0 sites bonded to cations at surface
 *                    so as to terminate dangling bonds with OH's */
void terminateSurface(Lattice lattice)
{
    int i, i2, j, type;

    /* "dangle" the 0's which are missing cations */
    for (i = 0; i < Nsites; i++) {
        if (lattice[i].state % 100 == 0 || lattice[i].state / 100 < 3 ||
            lattice[i].state == EDGE)
            continue;
        for (j = 0; j < 6; j++) {
            i2 = lattice[i].nbr[j];
            if (i2 >= 0 && (type = lattice[i2].state % 100) == 0) {
                switch (lattice[i].state) {
                    case 401: lattice[i].state = (type == 2) ? 404 : 406; break;
                    case 404: lattice[i].state = 409; break;
                    case 406: lattice[i].state = (type == 2) ? 408 : 409; break;
                    case 409: case 408: lattice[i].state = 400; break;
                    case 301: lattice[i].state = 303; break;
                    case 303: lattice[i].state = 300; break;
                    case 501: lattice[i].state = 503; break;
                    case 503: lattice[i].state = 500; break;
                    default: break;
                }
            }
        }
    }

    /* turn on dangling oxygens to terminate dangling bonds */
    for (i = 0; i < Nsites; i++) {
        if ((type = lattice[i].state / 100) > 2 ||
            lattice[i].state % 100 != 1)
            continue;
        for (j = 0; j < 6; j++) {
            i2 = lattice[i].nbr[j];
            if (lattice[i2].state % 100 != 1)
                switch (lattice[i2].state) {
                    case 300: lattice[i2].state = 303; break;
                    case 400: lattice[i2].state = ((type == 2) ? 408 : 409); break;
                    case 408: lattice[i2].state = 406; break;
                    case 409: lattice[i2].state = ((type == 2) ? 406 : 404); break;
                    case 404: lattice[i2].state = 401; break;
                    case 406: lattice[i2].state = 401; break;
                    case 500: lattice[i2].state = 503; break;
                    case 503: lattice[i2].state = 501; break;
                    default: break;
                }
        }
    }
}

```

```

}

/* update state of Si and Al to reflect terminal OH's */
for (i = 0; i < Nsites; i++) {
    if (lattice[i].state % 100 == 0 || /* unoccupied */
        lattice[i].state / 100 > 2 || /* 0 or OH */
        lattice[i].state == EDGE) /* edge */
        continue;
    if (lattice[i].state / 100 == 1) { /* Al */
        for (j = 0; j < 6; j++) {
            i2 = lattice[i].nbr[j];
            switch (lattice[i2].state) {
                case 503: case 409: lattice[i].state++; break;
                default: break;
            }
        }
    } else {
        for (j = 0; j < 6; j++) {
            i2 = lattice[i].nbr[j];
            switch (lattice[i2].state) {
                case 303: case 408: lattice[i].state++; break;
                default: break;
            }
        }
    }
}
}
}

```

A.14 myerr.h

```

/* myerr.h: exports error handling routines */

#ifndef myerr_h
#define myerr_h

void die(char *msg);

#endif

```

A.15 myerr.c

```

#include <stdio.h>
#include "common.h"
#include "myerr.h"

/* die: print msg to stderr then exit */
void die(char *msg)
{
    fprintf(stderr, "mckaol: %s\n", msg);
}

```

```
    exit(1);
}
```

A.16 output.h

```
/* output.h: functions for writing out various data files */

#ifndef output_h
#define output_h

#include "lattice.h"
#include "ucell.h"

void initDatafile(void);
void writeData(Lattice lattice, int n, float t);
void writeMSI(Lattice l, unitCell cell, char *name, int bonds);
void writeSurf(Lattice l, unitCell cell);
void writeXYZ(Lattice l, unitCell cell);

#endif
```

A.17 output.c

```
#include <stdio.h>
#include <unistd.h>
#include "common.h"
#include "output.h"
#include "futil.h"
#include "math.h"
#include "myerr.h"
#include "sim.h"

void initDatafile(void)
{
    unlink("results.dat");
}

void writeData(Lattice lattice, int n, float t)
{
    FILE *f;
    int i, nsi[5], nal[7], oh, type, sitot, altot;

    f = openFile("results.dat", "a");
    sitot = altot = 0;
```

```

for (i = 0; i < 5; i++)
    nsi[i] = 0;
for (i = 0; i < 7; i++)
    nal[i] = 0;

for (i = 0; i < n; i++) {
    if (lattice[i].state % 100 == 0) /*skip empty sites */
        continue;
    if ((type = lattice[i].state / 100) == 1) {
        oh = lattice[i].state - 101;
        nal[oh]++; altot++;
    } else if (type == 2) {
        oh = lattice[i].state - 201;
        nsi[oh]++; sitot++;
    }
}

fprintf(f, "%f", t);
fprintf(f, "%d", sitot);
for (i = 0; i < 5; i++)
    fprintf(f, "%d", nsi[i]);
fprintf(f, "%d", altot);
for (i = 0; i < 7; i++)
    fprintf(f, "%d", nal[i]);
fprintf(f, "\n");

closeFile(f);
}

void writeXYZ(Lattice l, unitCell cell)
{
    int i, j, natom, acells, bcells;
    float x, y, z, al, bl, cl;
    FILE *xyz;
    float cd[][3] = {{4.9725, -0.0262374, -1.3362},
                     {0.0, 8.92893, -0.30084},
                     {0.0, 0.0, 7.384}};
    al = sqrt(cd[0][0] * cd[0][0] + cd[0][1] * cd[0][1] +
              cd[0][2] * cd[0][2]);
    bl = sqrt(cd[1][1] * cd[1][1] + cd[1][2] * cd[1][2]);
    cl = cd[2][2];

    getDim(&acells, &bcells);
    /* open file and print header */
    xyz = openFile("start.xyz", "w");

    natom = getNsites();

    for (i = 0; i < natom; i++) {
        if ((l[i].state > 200 && l[i].state < 206)
            || (l[i].state > 100 && l[i].state < 108)
            || (l[i].state > 300 && l[i].state < 400)
            || (l[i].state > 400 && l[i].state < 500)
            || (l[i].state > 500 && l[i].state < 600)) {
            x = (cell[l[i].n].x/al + l[i].a) * cd[0][0] +
                (cell[l[i].n].y/bl + l[i].b) * cd[0][1] +

```

```

        (cell[l[i].n].z/cl) * cd[0][2];
    y = (cell[l[i].n].y/bl + l[i].b) * cd[1][1] +
        (cell[l[i].n].z/cl) * cd[1][2];
    z = (cell[l[i].n].z/cl) * cd[2][2];
    if (l[i].state / 100 == 1)
        fprintf(xyz, "%d,%f,%f,%f\n", 13, x, y, z);
    else if (l[i].state / 100 == 2)
        fprintf(xyz, "%d,%f,%f,%f\n", 14, x, y, z);
    else
        fprintf(xyz, "%d,%f,%f,%f\n", 8, x, y, z);
    }
}
closeFile(xyz);
}

void writeSurf(Lattice l, unitCell cell)
{
    int i, j, nbr, flag, natom, acells, bcells;
    float x, y, z, al, bl, cl;
    FILE *fsi, *fal;
    float cd[][3] = {{4.9725, -0.0262374, -1.3362},
                     {0.0, 8.92893, -0.30084},
                     {0.0, 0.0, 7.384}};
    al = sqrt(cd[0][0] * cd[0][0] + cd[0][1] * cd[0][1] +
              cd[0][2] * cd[0][2]);
    bl = sqrt(cd[1][1] * cd[1][1] + cd[1][2] * cd[1][2]);
    cl = cd[2][2];

    getDim(&acells, &bcells);
    /* open file and print header */
    fsi = openFile("surfSi.out", "w");
    fal = openFile("surfAl.out", "w");

    natom = getNsites();

    for (i = 0; i < natom; i++) {
        if ((l[i].state > 201 && l[i].state < 206)
            || (l[i].state > 101 && l[i].state < 108)) {
            flag = 0;
            for (j = 0; j < 6 && l[i].nbr[j] >= 0; j++) {
                nbr = l[i].nbr[j];
                if (l[nbr].state == 303
                    || (l[nbr].state > 403 && l[nbr].state < 410)
                    || (l[nbr].state == 503))
                    flag = 1;
            }
            if (flag == 1) {
                x = (cell[l[i].n].x/al + l[i].a) * cd[0][0] +
                    (cell[l[i].n].y/bl + l[i].b) * cd[0][1] +
                    (cell[l[i].n].z/cl) * cd[0][2];
                y = (cell[l[i].n].y/bl + l[i].b) * cd[1][1] +
                    (cell[l[i].n].z/cl) * cd[1][2];
                z = (cell[l[i].n].z/cl) * cd[2][2];
                if (l[i].state > 200)
                    fprintf(fsi, "%f,%f\n", x, y);
            }
        }
    }
}

```



```

        else
            fprintf(fal, "%f,%f\n", x, y);
    }
}
}

closeFile(fal);
closeFile(fsi);
}

void writeMSI(Lattice l, unitCell cell, char *name, int bonds)
{
    int i, j, natom, nthing = 1, *id, i2, acells, bcells;
    char fname[100];
    float x, y, z, al, bl, cl;
    FILE *fxyz;
    float cd[][3] = {{4.9725, -0.0262374, -1.3362},
                     {0.0, 8.92893, -0.30084},
                     {0.0, 0.0, 7.384}};
    al = sqrt(cd[0][0] * cd[0][0] + cd[0][1] * cd[0][1] +
              cd[0][2] * cd[0][2]);
    bl = sqrt(cd[1][1] * cd[1][1] + cd[1][2] * cd[1][2]);
    cl = cd[2][2];

    getDim(&acells, &bcells);
    /* open file and print header */
    sprintf(fname, "%s.msi", name);
    fxyz = openFile(fname, "w");
    fprintf(fxyz, "# MSI CERIU2 DataModel File Version 3 5\n");
    fprintf(fxyz, "(1 Model\n");
    fprintf(fxyz, "(A I Id 1)\n");
    fprintf(fxyz, "(A C Label \"%s\")\n", name);

    natom = getNsites();
    if (!id = (int *) malloc (sizeof(int) * natom))
        die("malloc failed");

    for (i = 0; i < natom; i++) {
        if (l[i].state % 100 > 0 && l[i].state != EDGE) {
            fprintf(fxyz, "(%d Atom\n", ++nthing);
            id[i] = nthing;
            fprintf(fxyz, "(A C ACL ");
            switch (l[i].state / 100) {
                case 0: fprintf(fxyz, "\"10 Ne\")\n"); break;
                case 1: fprintf(fxyz, "\"13 Al\")\n"); break;
                case 2: fprintf(fxyz, "\"14 Si\")\n"); break;
                case 3: case 4:
                    if (l[i].state % 100 > 1)
                        fprintf(fxyz, "\"1 H\")\n");
                    else
                        fprintf(fxyz, "\"8 O\")\n");
                    break;
                case 5:
                    if (l[i].state % 100 > 1)
                        fprintf(fxyz, "\"1 H\")\n");
                    else

```

```

        fprintf(fxyz, "\"9 F\")\n");
        break;
    default: break;
}
x = (cell[l[i].n].x/al + l[i].a) * cd[0][0] +
    (cell[l[i].n].y/bl + l[i].b) * cd[0][1] +
    (cell[l[i].n].z/cl) * cd[0][2];
y = (cell[l[i].n].y/bl + l[i].b) * cd[1][1] +
    (cell[l[i].n].z/cl) * cd[1][2];
z = (cell[l[i].n].z/cl) * cd[2][2];
fprintf(fxyz, " (A D XYZ (%f %f %f))\n", x, y, z);
fprintf(fxyz, " (A I Id %d)\n", nthng - 1);
fprintf(fxyz, " (A C Label \"\");
switch (l[i].state / 100) {
    case 0: fprintf(fxyz, "Edge"); break;
    case 1: fprintf(fxyz, "Al"); break;
    case 2: fprintf(fxyz, "Si"); break;
    case 3: case 4: fprintf(fxyz, "O"); break;
    case 5: fprintf(fxyz, "OH"); break;
    default: break;
}
fprintf(fxyz, "%d)\n", i);
fprintf(fxyz, " (A I LabelType 0)\n");
fprintf(fxyz, " )\n");
}
}

for (i = 0; i < natom; i++) {
    if (bonds && (l[i].state % 100 > 0 && l[i].state != EDGE)) {
        for (j = 0; j < 6; j++) {
            if ((i2 = l[i].nbr[j]) >= 0
                && i2 < i
                && l[i2].state % 100 != 0
                && l[i2].state != EDGE
                && !(l[i].a == 0 && l[i2].a == acells - 1 &&
                    cell[l[i].n].nbr[j].a == -1)
                && !(l[i].a == acells - 1 && l[i2].a == 0 &&
                    cell[l[i].n].nbr[j].a == 1)
                && !(l[i].b == 0 && l[i2].b == bcells - 1 &&
                    cell[l[i].n].nbr[j].b == -1)
                && !(l[i].b == bcells - 1 && l[i2].b == 0 &&
                    cell[l[i].n].nbr[j].b == 1)) {
                fprintf(fxyz, " (%d Bond\n", ++nthng);
                fprintf(fxyz, " (A 0 Atom1 %d)\n", id[i]);
                fprintf(fxyz, " (A 0 Atom2 %d)\n", id[i2]);
                fprintf(fxyz, " )\n");
            }
        }
    }
}

fprintf(fxyz, ")\n");
closeFile(fxyz);

free(id);
}

```

A.18 ran2.h

```
#ifndef _ran2_h
#define _ran2_h

/* initran2: call with seed only once */

void initran2(long *i);

/* ran2: return random float on open interval 0 to 1 */

float ran2(void);

#endif
```

A.19 ran2.c

```
#include "ran2.h"

/* ran2: generates pseudo random number in open interval 0 to 1.0
   idum should not be changed between calls, and should
   be < 0 on first call.
   from Numerical Recipes. */

#define IM1 2147483563
#define IM2 2147483399
#define AM (1.0/IM1)
#define IMM1 (IM1-1)
#define IA1 40014
#define IA2 40692
#define IQ1 53668
#define IQ2 52774
#define IR1 12211
#define IR2 3791
#define NTAB 32
#define NDIV (1+IMM1/NTAB)
#define EPS 1.2e-7
#define RNMX (1.0-EPS)

static long *idum;

void initran2(long *i)
{
    idum = i;
}

float ran2(void)
{
    int j;
    long k;
    static long idum2=123456789;
    static long iy=0;
```

```

static long iv[NTAB];
float temp;

if (*idum <= 0) {
    if (-(*idum) < 1) *idum=1;
    else *idum = -(*idum);
    idum2=(*idum);
    for (j=NTAB+7;j>=0;j--) {
        k=(*idum)/IQ1;
        *idum=IA1*(*idum-k*IQ1)-k*IR1;
        if (*idum < 0) *idum += IM1;
        if (j < NTAB) iv[j] = *idum;
    }
    iy=iv[0];
}
k=(*idum)/IQ1;
*idum=IA1*(*idum-k*IQ1)-k*IR1;
if (*idum < 0) *idum += IM1;
k=idum2/IQ2;
idum2=IA2*(idum2-k*IQ2)-k*IR2;
if (idum2 < 0) idum2 += IM2;
j=iy/NDIV;
iy=iv[j]-idum2;
iv[j] = *idum;
if (iy < 1) iy += IMM1;
if ((temp=AM*iy) > RNMX) return RNMX;
else return temp;
}

#undef IM1
#undef IM2
#undef AM
#undef IMM1
#undef IA1
#undef IA2
#undef IQ1
#undef IQ2
#undef IR1
#undef IR2
#undef NTAB
#undef NDIV
#undef EPS
#undef RNMX

```

A.20 reactions.h

```

/* reactions.h: functions for updating as result of reaction */

#ifndef reactions_h
#define reactions_h

#include "lattice.h"

void adsorbAl(Lattice lattice, int site);
void adsorbSi(Lattice lattice, int site);

```

```

void desorbAl(Lattice lattice, int site);
void desorbSi(Lattice lattice, int site);
void diffuse(Lattice lattice, int source, int target);
void r0(Lattice lattice, int site);
void r1(Lattice lattice, int site);
void r2(Lattice lattice, int site);
void r3(Lattice lattice, int site);
void r4(Lattice lattice, int site);
void r5(Lattice lattice, int site);
void r6(Lattice lattice, int site);
void r7(Lattice lattice, int site);
void r8(Lattice lattice, int site);
void r9(Lattice lattice, int site);
void r10(Lattice lattice, int site);
void r11(Lattice lattice, int site);
void r12(Lattice lattice, int site);
void r13(Lattice lattice, int site);
void r14(Lattice lattice, int site);
void r15(Lattice lattice, int site);

#endif

```

A.21 reactions.c

```

#include <stdio.h>
#include "common.h"
#include "reactions.h"
#include "myerr.h"
#include "ran2.h"

/* adsorbAl: update state of 0 site upon adsorbing Al(OH,H2O)6 */
void adsorbAl(Lattice lattice, int site)
{
    int i, nbr;

    if (ISAL(lattice[site])) {
        lattice[site].state = 107;
        for (i = 0; i < 6; i++) {
            nbr = lattice[site].nbr[i];
            switch (lattice[nbr].state) {
                case 503: lattice[nbr].state = 502; break;
                case 500: lattice[nbr].state = 503; break;
                case 407: lattice[nbr].state = 403; break;
                case 409: lattice[nbr].state = 405; break;
                case 408: lattice[nbr].state = 407; break;
                case 400: lattice[nbr].state = 409; break;
                case 406:
                    lattice[nbr].state = 410;
                    lattice[nbr].lostal = site;
                    break;
                default:
                    printf("state %d\n", lattice[nbr].state);
                    die("invalid state in adsorbAl");
                    break;
            }
        }
    }
}

```

```

    }
  }
} else {
  lattice[site].state = 200 + WRONG;
}
}

/* adsorbSi: update state of 0 site upon adsorbing Si(OH)4 */
void adsorbSi(Lattice lattice, int site)
{
  int i, nbr;

  if (ISSI(lattice[site])) {
    lattice[site].state = 205;
    for (i = 0; i < 4; i++) {
      nbr = lattice[site].nbr[i];
      switch (lattice[nbr].state) {
        case 300: lattice[nbr].state = 303; break;
        case 303: lattice[nbr].state = 302; break;
        case 404: lattice[nbr].state = 402; break;
        case 405: lattice[nbr].state = 403; break;
        case 409: lattice[nbr].state = 407; break;
        case 400: lattice[nbr].state = 408; break;
        default:
          printf("state %d\n", lattice[nbr].state);
          die("invalid state in adsorbSi");
          break;
      }
    }
  } else {
    lattice[site].state = 100 + WRONG;
  }
}

/* desorbAl: update state of 0 site upon desorbing Al(OH,H2O)6 */
void desorbAl(Lattice lattice, int site)
{
  int i, nbr;

  if (ISAL(lattice[site])) {
    lattice[site].state = 100;
    for (i = 0; i < 6; i++) {
      nbr = lattice[site].nbr[i];          /* nbr is an o site */
      switch (lattice[nbr].state) {
        case 502: lattice[nbr].state = 503; break;
        case 503: lattice[nbr].state = 500; break;
        case 403: lattice[nbr].state = 407; break;
        case 405: lattice[nbr].state = 409; break;
        case 407: lattice[nbr].state = 408; break;
        case 409: lattice[nbr].state = 400; break;
        case 410:
          lattice[nbr].state = 406;
          lattice[nbr].lostal = -1;
          break;
      }
    }
  }
}

```

```

        default: break;
    }
}
} else {
    lattice[site].state = 200;
}
}

/* desorbSi: update state of 0 site upon desorbing Si(OH)4 */
void desorbSi(Lattice lattice, int site)
{
    int i, nbr;

    if (ISSI(lattice[site])) {
        lattice[site].state = 200;
        for (i = 0; i < 4; i++) {
            nbr = lattice[site].nbr[i];
            switch (lattice[nbr].state) {
                case 303: lattice[nbr].state = 300; break;
                case 302: lattice[nbr].state = 303; break;
                case 402: lattice[nbr].state = 404; break;
                case 403: lattice[nbr].state = 405; break;
                case 407: lattice[nbr].state = 409; break;
                case 408: lattice[nbr].state = 400; break;
                default: break;
            }
        }
    } else {
        lattice[site].state = 100;
    }
}

void diffuse(Lattice lattice, int source, int target)
{
    if (lattice[source].state == 107 || lattice[source].state == 299) {
        desorbAl(lattice, source);
        adsorbAl(lattice, target);
    } else {
        desorbSi(lattice, source);
        adsorbSi(lattice, target);
    }
}

/* r0: =Si-O-Si= + H2O => 2(=Si-OH) */
void r0(Lattice lattice, int site)
{
    int si, i;

    lattice[site].state = 302;
    for (i = 0; i < 2; i++) {

```

```

        si = lattice[site].nbr[i];
        lattice[si].state++;
    }
}

/* r1: 2(=Si-OH) => =Si-O-Si= + H2O */
void r1(Lattice lattice, int site)
{
    int si, i;

    lattice[site].state = 301;
    for (i = 0; i < 2; i++) {
        si = lattice[site].nbr[i];
        lattice[si].state--;
    }
}

/* r2: =Si-O-2(Al=) + H2O => =Si-OH + =Al-OH-Al= */
void r2(Lattice lattice, int site)
{
    int nbr;

    lattice[site].state = 402;
    nbr = lattice[site].nbr[2];
    lattice[nbr].state++; /* increment for si only */
}

/* r3: =Si-OH + =Al-OH-Al= => =Si-O-2(Al=) + H2O */
void r3(Lattice lattice, int site)
{
    int nbr;

    lattice[site].state = 401;
    nbr = lattice[site].nbr[2];
    lattice[nbr].state--; /* decrement for si only */
}

/* r4: =Si-O-2(Al=) + H2O => =Si-OH-Al= + =Al-OH */
void r4(Lattice lattice, int site)
{
    int nbr;
    float r;

    lattice[site].state = 410;
    r = ran2();
    if (r < 0.5)
        nbr = lattice[site].nbr[0];
    else
        nbr = lattice[site].nbr[1];
}

```



```

    lattice[nbr].state++;
    lattice[site].lostal = nbr;
}

/* r5: =Si-OH-Al= + =Al-OH => =Si-O-2(Al=) + H2O */
void r5(Lattice lattice, int site)
{
    int nbr;

    lattice[site].state = 401;
    nbr = lattice[site].lostal;
    lattice[nbr].state--;
    lattice[site].lostal = -1;
}

/* r6: =Si-OH + =Al-OH-Al= + H2O => =Si-OH + =Al-OH + =Al-H2O */
void r6(Lattice lattice, int site)
{
    int i, nbr;

    lattice[site].state = 403;
    for (i = 0; i < 2; i++) {
        nbr = lattice[site].nbr[i];
        lattice[nbr].state++;
    }
}

/* r7: =Si-OH + =Al-OH + =Al-H2O => =Si-OH + =Al-OH-Al= + H2O */
void r7(Lattice lattice, int site)
{
    int i, nbr;

    lattice[site].state = 402;
    for (i = 0; i < 2; i++) {
        nbr = lattice[site].nbr[i];
        lattice[nbr].state--;
    }
}

/* r8: =Si-OH-Al= + =Al-OH + H2O => =Si-OH + =Al-OH + =Al-H2O */
void r8(Lattice lattice, int site)
{
    int nbr;

    lattice[site].state = 403;
    nbr = lattice[site].nbr[2]; /* si */
    lattice[nbr].state++;
    nbr = (lattice[site].lostal == lattice[site].nbr[0]) ?
        lattice[site].nbr[1] : lattice[site].nbr[0];
}

```

```

    lattice[nbr].state++;
    lattice[site].lostal = -1;
}

/* r9: =Si-OH + =Al-OH + =Al-H2O => =Si-OH-Al= + =Al-OH + H2O */
void r9(Lattice lattice, int site)
{
    int nbr, lost;
    float r;

    lattice[site].state = 410;
    r = ran2();
    nbr = lattice[site].nbr[2];    /* si */
    lattice[nbr].state--;
    if (r < 0.5) {
        nbr = lattice[site].nbr[0];    /* al */
        lost = lattice[site].nbr[1];
    } else {
        nbr = lattice[site].nbr[1];
        lost = lattice[site].nbr[0];
    }
    lattice[nbr].state--;
    lattice[site].lostal = lost;
}

/* r10: =Si-OH-Al= + H2O => =Si-OH + =Al=H2O */
void r10(Lattice lattice, int site)
{
    int nbr, al1, al2;

    lattice[site].state = 407;
    nbr = lattice[site].nbr[2];    /* si */
    lattice[nbr].state++;
    al1 = lattice[site].nbr[0];
    al2 = lattice[site].nbr[1];
    nbr = (lattice[al1].state == 100) ? al2 : al1;
    lattice[nbr].state++;    /* al */
}

/* r11: =Si-OH + =Al=H2O => =Si-OH-Al= + H2O */
void r11(Lattice lattice, int site)
{
    int nbr, al1, al2;

    lattice[site].state = 406;
    nbr = lattice[site].nbr[2];
    lattice[nbr].state--;
    al1 = lattice[site].nbr[0];
    al2 = lattice[site].nbr[1];
    nbr = (lattice[al1].state == 100) ? al2 : al1;

```

```

    lattice[nbr].state--;          /* a1 */
}

/* r12: =Al-OH-Al= + H2O => =Al-OH + =Al-H2O (400s) */
void r12(Lattice lattice, int site)
{
    int nbr, i;

    lattice[site].state = 405;
    for (i = 0; i < 2; i++) {
        nbr = lattice[site].nbr[i];
        lattice[nbr].state++;
    }
}

/* r13: =Al-OH + =Al-H2O => =Al-OH-Al= + H2O (400s) */
void r13(Lattice lattice, int site)
{
    int nbr, i;

    lattice[site].state = 404;
    for (i = 0; i < 2; i++) {
        nbr = lattice[site].nbr[i];
        lattice[nbr].state--;
    }
}

/* r14: =Al-OH-Al= + H2O => =Al-OH + =Al-H2O (500s) */
void r14(Lattice lattice, int site)
{
    int nbr, i;

    lattice[site].state = 502;
    for (i = 0; i < 2; i++) {
        nbr = lattice[site].nbr[i];
        lattice[nbr].state++;
    }
}

/* r15: =Al-OH + =Al-H2O => =Al-OH-Al= + H2O (500s) */
void r15(Lattice lattice, int site)
{
    int nbr, i;

    lattice[site].state = 501;
    for (i = 0; i < 2; i++) {
        nbr = lattice[site].nbr[i];
        lattice[nbr].state--;
    }
}

```

```
}

```

A.22 rxnlist.h

```
/* rxnlist.h: exports types and functions for reaction list */

#ifndef rxnlist_h
#define rxnlist_h

#define N300    2
#define N400   14
#define NHYD   16
#define NADS   20
#define NDES   24
#define NRXN   28
#define Kb 1.38066e-23 /* boltzman in J/mol/K */
#define R  1.987e-3    /* R in kcal/mol/K */
#define JtoCAL 4.1842 /* multiply cal by JtoCAL to get J */

/* reaction: Gives reactant and product states for given reaction,
 *              and reaction rate. */
struct reaction {
    int    reactant, /* reactant state */
        info,       /* multi-purpose info */
        nrates;     /* number of actual rates */
    float *rate;     /* reaction rate list since actual rate
                     * depends on next neighbors' states */
};
typedef struct reaction *reactionList;

reactionList readRxns(void);
void free_rxnList(reactionList rl);
void getChem(float *si, float *al);

#endif

```

A.23 rxnlist.c

```
#include <stdio.h>
#include <stdlib.h>
#include "common.h"
#include "futil.h"
#include "rxnlist.h"
#include "math.h"

static float dmsi, dmal;

void free_rxnList(reactionList rl)
{
    int i;

    for (i = 0; i < NRXN; i++)

```

```

    free(rl[i].rate);
    free(rl);
}

/* readRxns: read in rates, dE's for forward rxns
 *          calculate rates for reverse
 *          complete reactionList */
reactionList readRxns(void)
{
    FILE *f;
    reactionList l;
    int rct, prd, i, j, nrts;
    float kp, de, t, a, rt, kt;

    f = openFile("data.rxn","r");
    fscanf(f, "%f", &t); eatComment(f, '#');
    fscanf(f, "%f", &dmsi); eatComment(f, '#');
    fscanf(f, "%f", &dmal); eatComment(f, '#');
    l = (reactionList) malloc (sizeof(*l) * NRXN);
    rt = R * t;
    kt = Kb * t;

    /* hydrolysis reactions
     * de is in kcal/mol/K */
    for (i = 0; i < NHYD; i += 2) {
        fscanf(f, "%d %d", &rct, &prd); eatComment(f, '#');
        l[i].reactant = rct;
        l[i+1].reactant = prd;
        fscanf(f, "%d", &nrts); eatComment(f, '#');
        l[i].nrates = l[i+1].nrates = nrts;
        l[i].rate = (float *) malloc (sizeof(float) * nrts);
        l[i+1].rate = (float *) malloc (sizeof(float) * nrts);
        for (j = 0; j < nrts; j++) {
            fscanf(f, "%f %f", &kp, &de);
            l[i].rate[j] = kp;
            l[i+1].rate[j] = kp * exp(de / rt);
        }
    }

    /* adsorption : a is \nu * exp(\mu_solid / RT)
     * dm's are in kcal/mol/K */
    eatComment(f, '#');
    for (i = NHYD; i < NADS; i++) {
        fscanf(f, "%d %d", &rct, &prd); eatComment(f, '#');
        fscanf(f, "%f %f", &a, &de); eatComment(f, '#');
        l[i].reactant = rct;
        l[i].info = prd;
        l[i].nrates = 1;
        l[i].rate = (float *) malloc (sizeof(float));
        if (i < 18) /* si adsorption or al adsorption */
            l[i].rate[0] = a * exp(de / rt) * exp(dmsi / rt);
        else
            l[i].rate[0] = a * exp(de / rt) * exp(dmal / rt);
    }
}

```

```

/* desorption */
eatComment(f, '#');
for (i = NADS; i < NDES; i++) {
    fscanf(f, " %d", &rct); eatComment(f, '#');
    l[i].reactant = rct;
    fscanf(f, " %d", &nrts);
    l[i].nrates = nrts;
    l[i].rate = (float *) malloc (sizeof(float) * nrts);
    for (j = 0; j < nrts; j++) {
        fscanf(f, " %f %f", &a, &de);
        l[i].rate[j] = a * exp(- de / rt);
    }
}

/* diffusion */
for (i = NDES; i < NRXN; i++) {
    l[i].reactant = l[i-4].reactant;
    l[i].nrates = l[i-4].nrates;
    l[i].rate = (float *) malloc (sizeof(float) * nrts);
    for (j = 0; j < nrts; j++)
        l[i].rate[j] = l[i-4].rate[j];
}

fclose(f);
return l;
}

```

```

void getChem(float *si, float *al)
{
    *si = dmsi;
    *al = dmal;
}

```

A.24 sim.h

```

/* sim.h: exports struct and function for simulation parameters */

#ifndef sim_h
#define sim_h

struct simulation {
    int nsteps,      /* total number of steps in simulation */
        wsteps,     /* number of steps between data writes */
        msteps,     /* number of steps between movie frames */
        drawbonds;  /* TRUE = draw bonds, FALSE = don't */
    long ranseed;    /* seed for initran2 */
};
typedef struct simulation *Simulation;

Simulation readCond(void);
void free_Simulation(Simulation s);

```

```
#endif
```

A.25 sim.c

```
#include <stdio.h>
#include <stdlib.h>
#include "futil.h"
#include "myerr.h"
#include "common.h"
#include "sim.h"

/* readCond: read in some more simulation conditions */
Simulation readCond(void)
{
    FILE *f;
    Simulation s;

    f = openFile("data.sim","r");
    s = (Simulation) malloc (sizeof(*s));
    if (fscanf(f, " %d ", &s->nsteps) != 1)
        die("invalid number of steps in input file");
    eatComment(f, '#');
    if (fscanf(f, " %d ", &s->>wsteps) != 1)
        die("invalid number of data write steps in input file");
    eatComment(f, '#');
    if (fscanf(f, " %d ", &s->msteps) != 1)
        die("invalid number of movie write steps in input file");
    if (s->msteps == -1)
        s->msteps = s->nsteps + 1;
    eatComment(f, '#');
    if (fscanf(f, " %ld ", &s->ranseed) != 1)
        die("invalid number of steps in input file");
    eatComment(f, '#');
    if (fscanf(f, " %d ", &s->drawbonds) != 1)
        die("invalid draw bonds? parameter in input file");

    return s;
    closeFile(f);
}

void free_Simulation(Simulation s)
{
    free(s);
}
```

A.26 ucell.h

```
/* ucell.h: exports structs and routines for
 *          dealing with the unit cell */
```

```

#ifndef ucell_h
#define ucell_h

#include "sim.h"

/* neighborSite: Only for unit cell template; specifies a neighbor of
 *               a unit cell position. */
struct neighborSite {
    int a, b, c, /* unit cell offsets */
        n;      /* unit cell site number */
};

/* cellSite: Uniquely identifies a unit cell position and specifies
 *           what type of site and how to find neighbors. */
struct cellSite {
    float x, y, z; /* coordinate of site in unit cell */
    int n, /* site number */
        state; /* initial state (which kind of site?) */
    struct neighborSite nbr[6]; /* neighbor list */
};
typedef struct cellSite *unitCell;

struct celldim {
    float a,b,c,alpha,beta,gamma;
};
typedef struct celldim cellDim;

void free_cell(unitCell c);
cellDim *getCelldim(void);
int getNpos(void);
int getNumNbrs(int state);
unitCell readCell(void);

#endif

```

A.27 ucell.c

```

/* ucell.c: functions for unit cell */
#include <stdlib.h>
#include <stdio.h>
#include "common.h"
#include "ucell.h"
#include "futil.h"

static float Alpha, Beta, Gamma, A, B, C; /* unit cell params */
static int Npos;

void free_cell(unitCell c)

```



```

{
    free(c);
}

cellDim *getCellDim(void)
{
    cellDim *cd;

    cd = (cellDim *) malloc (sizeof(cellDim));
    cd->a = A; cd->b = B; cd->c = C;
    cd->alpha = Alpha; cd->beta = Beta; cd->gamma = Gamma;
    return cd;
}

int getNpos(void)
{
    return Npos;
}

/* getNumNbrs: returns the number of neighbors that a site should
 *             have based on what kind it is */
int getNumNbrs(int state)
{
    int nbrs;

    switch (state / 100) {
        case 1: nbrs = 6; break;          /* Al */
        case 2: nbrs = 4; break;          /* Si */
        case 3: case 5: nbrs = 2; break;   /* Si-O-Si or Al-OH-Al type 0 */
        case 4: nbrs = 3; break;          /* Si-O-Al2 type 0 */
        default: nbrs = -1; break;
    }
    return nbrs;
}

/* readCell: read in unit cell parameters,
 *            and some simulation conditions */
unitCell readCell(void)
{
    int i, j;
    unitCell c;
    FILE *f;

    f = openFile("data.cell", "r");
    fscanf(f, " %f %f %f ", &A, &B, &C);
    eatComment(f, '#');
    fscanf(f, " %f %f %f ", &Alpha, &Beta, &Gamma);
    eatComment(f, '#');
    fscanf(f, " %d ", &Npos);
    eatComment(f, '#');
    c = (unitCell) malloc (sizeof(struct cellSite) * (Npos + 1));
    for (i = 0; i < Npos; i++) {

```

```

    fscanf(f, " %d %d %f %f %f ", &c[i].n, &c[i].state,
           &c[i].x, &c[i].y, &c[i].z);
    for (j = 0; j < 6; j++) {
        fscanf(f, "%d %d %d %d ", &c[i].nbr[j].n, &c[i].nbr[j].a,
              &c[i].nbr[j].b, &c[i].nbr[j].c);
    }
}
c[Npos].n = -1;
return c;
closeFile(f);
}

```

A.28 mckaol.c

```

/* Libraries */
#include <stdio.h>
#include "common.h"
#include "rxnlist.h"
#include "evtlist.h"
#include "lattice.h"
#include "ucell.h"
#include "myerr.h"
#include "actions.h"
#include "sim.h"
#include "output.h"
#include "ran2.h"

int main(int argc, char *argv[])
{
    Simulation sc;
    reactionList rl;
    eventList el;
    unitCell uc;
    Lattice l;
    int i, nsites;
    float time = 0;
    char fname[50];

    sc = readCond();
    initran2(&sc->ranseed);
    initDatafile();
    rl = readRxns();
    uc = readCell();

    l = makeLattice(uc);
    nsites = getNsites();
    findPairs(l);
    populateSolid(l);
    terminateSurface(l);
    terminateLattice(l);
}

```

```

writeMSI(l, uc, "start", sc->drawbonds);
for (i = 0; i < sc->nsteps; i++) {
    el = new_evtList(l, rl, nsites);
    if (!el) {
        /* no events possible */
        writeMSI(l, uc, "end", sc->drawbonds);
        if (sc->wsteps)
            writeData(l, nsites, time);
        die("out of events");
    }
    time += doEvent(l, el);
    free_evtList(el);
    /* check if time to write stuff */
    if (sc->wsteps && (i % sc->wsteps == 0 || i == 0))
        writeData(l, nsites, time);
    if (sc->msteps && i && i % sc->msteps == 0) {
        sprintf(fname, "step%d", i);
        writeMSI(l, uc, fname, sc->drawbonds);
    }
}
writeSurf(l, uc);
writeXYZ(l, uc);
writeMSI(l, uc, "end", sc->drawbonds);
free_rxnList(rl);
free_lattice(l);
free_cell(uc);
free_Simulation(sc);
return 0;
}

```

A.29 Makefile

```

CC = gcc
CFLAGS = -O3
LIBS = -lm

all: mckaol

mckaol:
$(CC) $(CFLAGS) -o mckaol mckaol.c actions.c bfsearch.c /
    envrn.c evtlist.c futil.c lattice.c myerr.c output.c ran2.c /
    reactions.c rxnlist.c sim.c ucell.c $(LIBS)

clean:
rm -f *.o *~ *.Addr *.Counts *.pixie mon.out logfile

```

Appendix B

Minimum Energy Geometries

The figures in this appendix depict the minimum energy geometries of various complexes obtained at the B3LYP/3-21G** level of theory. In all figures, white circles are silicon, large gray circles are aluminum, small gray circles are hydrogen, medium size gray circles are carbon, and black circles are oxygen atoms. The figures were produced and rendered to encapsulated postscript files using the computer program **XMakemol**, developed by Matthew P. Hodges and distributed under the GNU Public License, which can be obtained free of charge at the URL <http://vegemite.chem.nott.ac.uk/~xmakemol>.

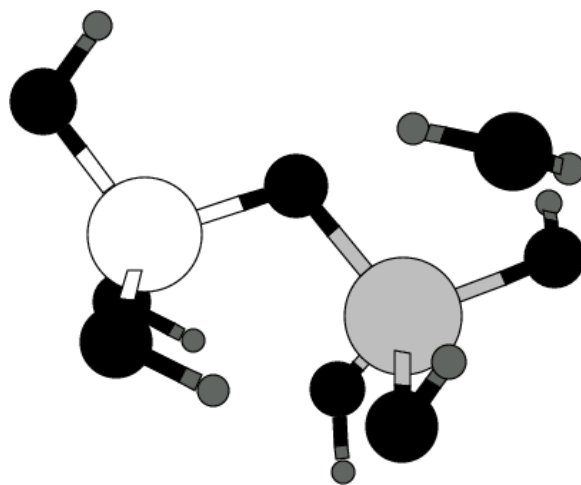


Figure B.1: $H_6SiAlO_7^-$ Dimer with adsorbed water.

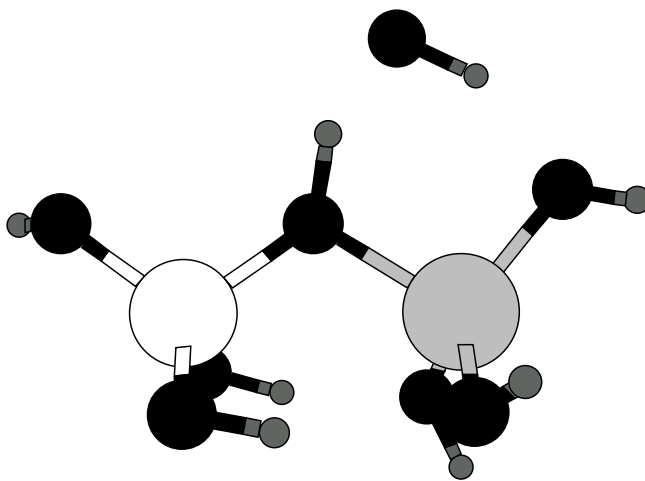


Figure B.2: H_7SiAlO_7 Dimer with adsorbed hydronium.

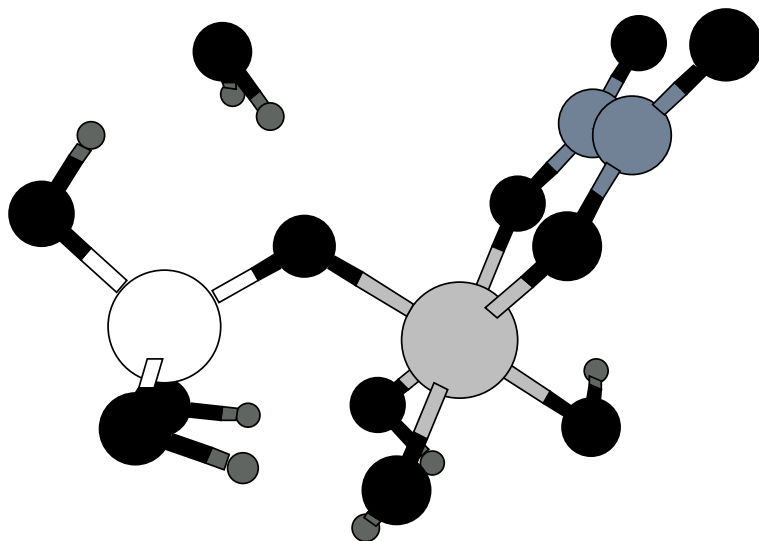


Figure B.3: $H_6SiAlO_7^-$ Dimer with adsorbed oxalate and water.

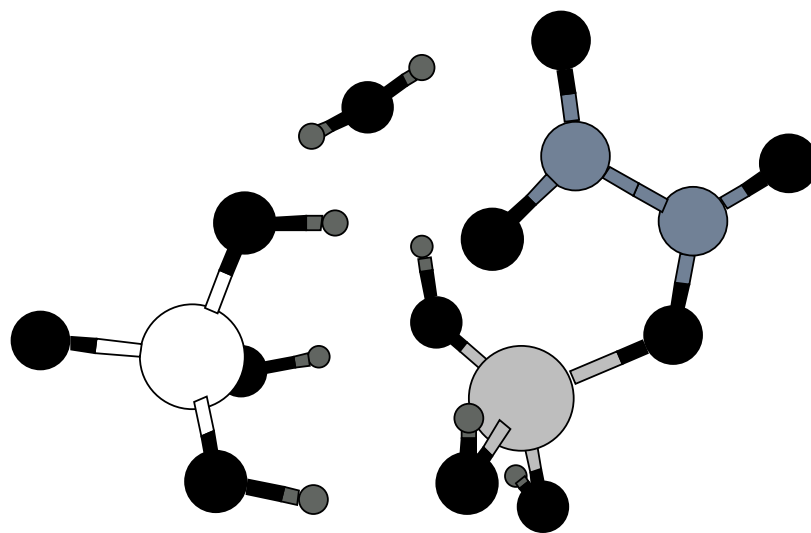


Figure B.4: H_7SiAlO_7 Dimer with adsorbed oxalate and hydronium.

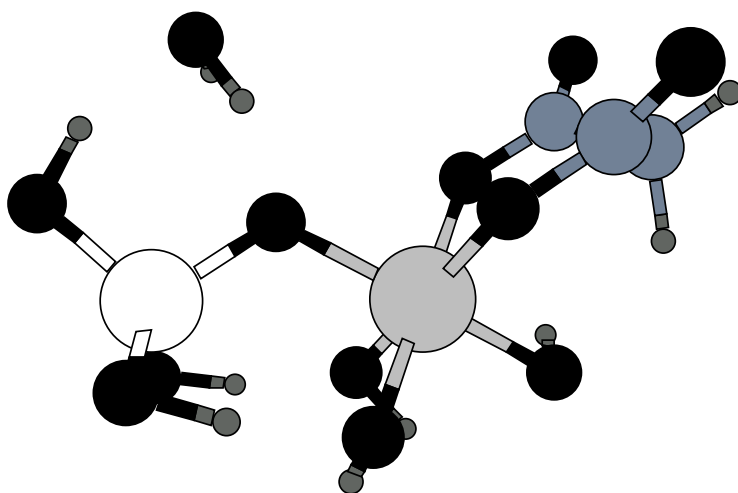


Figure B.5: $H_6SiAlO_7^-$ Dimer with adsorbed malonate and water.

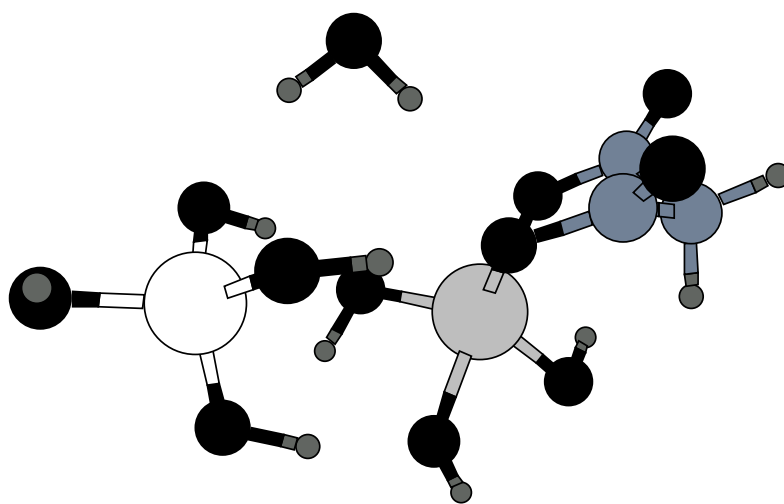


Figure B.6: H_7SiAlO_7 Dimer with adsorbed malonate and hydronium.

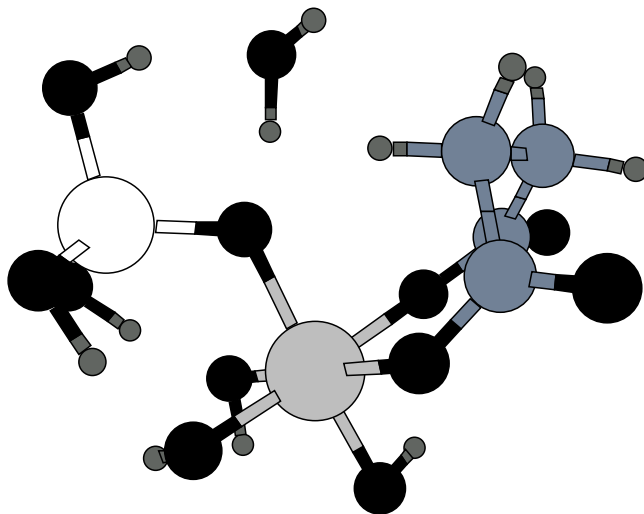


Figure B.7: $H_6SiAlO_7^-$ Dimer with adsorbed succinate and water.

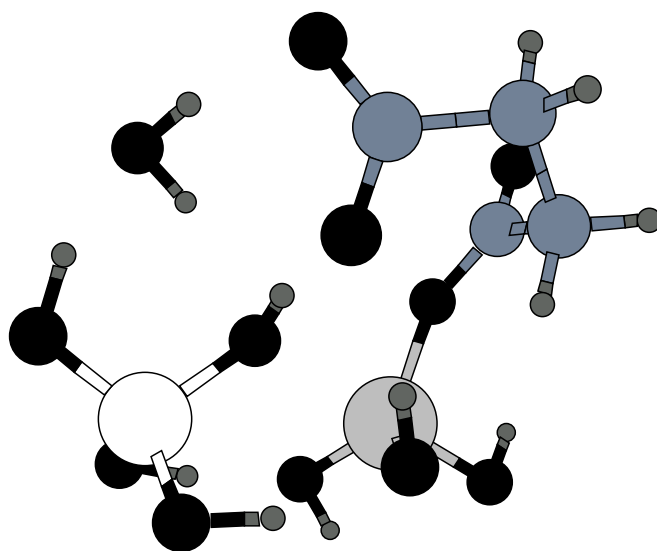


Figure B.8: H_7SiAlO_7 Dimer with adsorbed succinate and hydronium.

Bibliography

- Amrhein, C. and Suarez, D. L. (1988). The use of a surface complexation model to describe the kinetics of ligand-promoted dissolution of anorthite. *Geochimica et Cosmochimica Acta*, 52:2785–2793.
- Amrhein, C. and Suarez, D. L. (1992). Some factors affecting the dissolution kinetics of anorthite at 25°C. *Geochimica et Cosmochimica Acta*, 56:1815–1826.
- Bader, R. F. W. (1990). *Atoms in Molecules — A Quantum Theory*. Oxford University Press.
- Baptist, K. K., Walter, L. M., Budai, J. M., and Ku, T. C. W. (1998). Carbonate and silicate weathering contributions to soil waters in glacial drift deposits, n. michigan; results from laboratory and field experiments. In *Abstracts with Programs — Geological Society of America*, volume 30, page 85.
- Battaile, C. C. (1998). *Atomic-Scale kinetic Monte Carlo simulations of diamond chemical vapor deposition*. PhD thesis, The University of Michigan.
- Battaile, C. C., Srolovitz, D. J., and Butler, J. (1997). A kinetic Monte Carlo method for the atomic-scale simulation of chemical vapor deposition: application to diamond. *Journal of Applied Physics*, 82(12):6293–6300.
- Battaile, C. C., Srolovitz, D. J., and Butler, J. (1998). Atomic-scale simulations of chemical vapor deposition on flat and vicinal diamond substrates. *Journal of Crystal Growth*, 194(3–4):353–368.
- Battaile, C. C., Srolovitz, D. J., Oleinik, I. I., Pettifor, D. G., Sutton, A. P., Harris, S. J., and Butler, J. E. (1999). Etching effects during the chem-

- ical vapor deposition of (100) diamond. *Journal of Chemical Physics*, 111(9):4291–4299,.
- Becke, A. D. (1993). Density-functional thermochemistry. III. the role of exact exchange. *Journal of Chemical Physics*, 98:5648–5652.
- Bennett, P. C. and Casey, W. (1994). Chemistry and mechanisms of low-temperature dissolution of silicates by organic acids. In (Pittman and Lewan, 1994), chapter 7, pages 162–200.
- Bevan, J. and Savage, D. (1989). The effect of organic acids on the dissolution of K-feldspar under conditions relevant to burial diagenesis. *Mineralogical Magazine*, 53:415–425.
- Binder, K. and Heermann, D. W. (1988). *Monte Carlo Simulation in Statistical Physics: an Introduction*, volume 80 of *Springer Series in Solid-State Sciences*. Springer-Verlag.
- Blum, A. E. and Lasaga, A. C. (1987). Monte Carlo simulations of surface reaction rate laws. In (Stumm, 1987), chapter 10, pages 255–292.
- Bortz, A. B., Kalos, M. H., and Lebowitz, J. L. (1975). New algorithm for Monte-Carlo simulation of Ising spin systems. *Journal of Computational Physics*, 17(1):10–18.
- Boyd, R. J. and Edgecombe, K. E. (1988). Atomic and group electronegativities from electron density distributions of molecules. *Journal of the American Chemical Society*, 110:4182–4186.
- Boys, S. F. and Bernardi, F. (1970). The calculation of small molecular interactions by the differences of separate total energies. Some procedures with reduced errors. *Molecular Physics*, 19:553.
- Carroll, S. A. and Walther, J. V. (1990). Kaolinite dissolution at 25°C, 60°C, and 80°C. *American Journal of Science*, 290:797–810.
- Chin, P.-K. F. and Mills, G. L. (1991). Kinetics and mechanisms of kaolinite dissolution: effects of organic ligands. *Chemical Geology*, 90:307–317.
- Dawnkaski, E. J., Srivastava, D., and Garrison, B. J. (1995). Time dependent Monte Carlo simulations of H reactions on the diamond {001}(2×1)

- surface under chemical vapor deposition conditions. *Journal of Chemical Physics*, 102(23):9401–9411.
- Dawnkaski, E. J., Srivastava, D., and Garrison, B. J. (1996). Growth of diamond films on a diamond $\{001\}(2\times 1):H$ surface by time dependent Monte Carlo simulations. *Journal of Chemical Physics*, 104(15):5997–6008.
- Devidal, J.-L., Schott, J., and Dandurand, J.-L. (1997). An experimental study of kaolinite dissolution and precipitation kinetics as a function of chemical affinity and solution composition at 150°C, 40 bars, and pH 2, 6.8, and 7.8. *Geochimica et Cosmochimica Acta*, 61(24):5165–5186.
- Drever, J. I. and Vance, G. F. (1994). Role of soil organic acids in mineral weathering processes. In (Pittman and Lewan, 1994), chapter 6, pages 138–161.
- Eyring, H. (1935). The activated complex in chemical reactions. *Journal of Chemical Physics*, 3:107.
- Fein, J. B. (1991). Experimental study of aluminum-oxalate complexing at 80°C: Implications for the formation of secondary porosity within sedimentary reservoirs. *Geology*, 19:1037–1040.
- Felipe, M. A. and Lasaga, A. C. (1997). *Ab initio* simulation of quartz, albite and gibbsite dissolution using larger and hydrated molecular clusters. In *Abstracts with Programs — Geological Society of America*, volume 29, page 309.
- Fox, T. R. and Comerford, N. B. (1990). Low-molecular-weight organic acids in selected forest soils of the southeastern USA. *Soil Science Society of America Journal*, 54:1139–1144.
- Franklin, S. P., Hajash, Jr., A., and Tieh, T. T. (1991). The role of organic acids in feldspar and quartz dissolution at 100°C/347 bars: an experimental study. *American Association of Petroleum Geologists Bulletin*, 73:576.
- Frisch, M. J., Trucks, G. W., Schlegel, H. B., Gill, P. M. W., Johnson, B. G., Robb, M. A., Cheeseman, J. R., Keith, T., Petersson, G. A., Montgomery, J. A., Raghavachari, K., Al-Laham, M. A., Zakrzewski,

- V. G., Ortiz, J. V., Foresman, J. B., Cioslowski, J., Stefanov, B. B., Nanayakkara, A., Challacombe, M., Peng, C. Y., Ayala, P. Y., Chen, W., Wong, M. W., Andres, J. L., Replogle, E. S., Gomperts, R., Martin, R. L., Fox, D. J., Binkley, J. S., Defrees, D. J., Baker, J., Stewart, J. P., Head-Gordon, M., Gonzalez, C., and Pople, J. A. (1995). Gaussian 94, Revision E.2.
- Furrer, G. and Stumm, W. (1986). The coordination chemistry of weathering. I. Dissolution kinetics of $\alpha - Al_2O_3$ and BeO . *Geochimica et Cosmochimica Acta*, 50:1847–1860.
- Ganor, J., Mogollón, J. L., and Lasaga, A. C. (1995). The effect of pH on kaolinite dissolution rates and on activation energy. *Geochimica et Cosmochimica Acta*, 59(6):1037–1052.
- Gibbs, G. V., Boisen, M. B., Hill, F. C., Tamada, O., and Downs, R. T. (1997a). SiO and GeO bonded interactions as inferred from the bond critical point properties of electron density distributions. Unpublished manuscript.
- Gibbs, G. V., Hill, F. C., and Boisen, Jr., M. B. (1997b). The SiO bond and electron density distributions. In Silvi, B. and D’Arco, P., editors, *Modelling of Minerals and Silicated Materials*, pages 157–177. Kluwer Academic Publishers.
- Gilmer, G. H. (1976). Growth on imperfect crystal faces. 1. Monte-Carlo growth rates. *Journal of Crystal Growth*, 36(1):15–28.
- Gilmer, G. H. (1977). Computer simulation of 2D nucleation and crystal growth. *Abstracts of Paper of the American Chemical Society*, 173:151.
- Gilmer, G. H. (1980). Computer models of crystal growth. *Science*, 208(4442):355–363.
- Graustein, W. C., Cromack, K. J., and Sollins, P. (1977). Calcium oxalate: occurrence in soils and effect on nutrient and geochemical cycles. *Science*, 198:1252–1254.
- Hajash, Jr., A. (1994). Comparison and evaluation of experimental studies on dissolution of minerals by organic acids. In (Pittman and Lewan, 1994), chapter 8, pages 201–225.

- Hajash, Jr., A. and Bloom, M. A. (1991). Marine diagenesis of feldspathic sand: a flow-through experimental study at 200°C, 1 kbar. *Chemical Geology*, 89:359–377.
- Hassold, G. N. and Holm, E. A. (1993). A fast serial algorithm for the finite temperature quenched potts model. *Computers in Physics*, 7(1):97–107.
- Hill, F. C., Gibbs, G. V., and Boisen, M. B. (1997). Electron density distributions of oxide and hydroxyacid molecules. *Physics and Chemistry of Minerals*, 24.
- Huang, K. (1963). *Statistical Mechanics*. Wiley.
- Huang, P. M. and Schnitzer, M., editors (1986). *Interactions of soil minerals with natural organics and microbes*. Number 17 in Special Publications. Soil Science Society of America.
- Huang, W. H. and Keller, W. D. (1970). Dissolution of rock-forming silicate minerals in organic acids: Simulated first-stage weathering of fresh mineral surfaces. *The American Mineralogist*, 55:2076–2094.
- Huang, W. H. and Kiang, W. C. (1972). Laboratory dissolution of plagioclase feldspars in water and organic acids at room temperature. *American Mineralogist*, 57:1849–1859.
- Huang, W.-L. and Longo, J. M. (1992). The effect of organics on feldspar dissolution and the development of secondary porosity. *Chemical Geology*, 98:271–292.
- Huertas, F. J., Chou, L., and Wollast, R. (1998). Mechanism of kaolinite dissolution at room temperature and pressure: Part 1. Surface speciation. *Geochimica et Cosmochimica Acta*, 62(3):417–431.
- Kubicki, J. D., Blake, G. A., and Apitz, S. E. (1996a). Ab initio calculations on aluminosilicate Q^3 species: Implications for atomic structures of mineral surfaces and dissolution mechanisms of feldspars. *American Mineralogist*, 81:789–799.
- Kubicki, J. D., Blake, G. A., and Apitz, S. E. (1996b). Molecular orbital models of aqueous aluminum-acetate complexes. *Geochimica et Cosmochimica Acta*, 60(24):4897–4911.

- Lasaga, A. C. (1992). Ab initio methods in mineral surface reactions. *Reviews of Geophysics*, 30(4):269–303.
- Lasaga, A. C. (1995). Fundamental approaches in describing mineral dissolution and precipitation rates. In (White and Brantley, 1995), chapter 2.
- Lasaga, A. C. and Blum, A. E. (1986). Surface chemistry, etch pits, and mineral-water interactions. *Geochimica et Cosmochimica Acta*, 50(10):2363–2379.
- Lasaga, A. C. and Gibbs, G. V. (1990a). Ab-initio quantum mechanical calculations of water-rock interactions: Adsorption and hydrolysis reactions. *American Journal of Science*, 290:263–295.
- Lasaga, A. C. and Gibbs, G. V. (1990b). Ab initio quantum mechanical calculations of surface reactions: a new era? In (Stumm, 1990), pages 259–289.
- Lee, C., Yang, W., and Parr, R. G. (1988). Development of the Colle-Salvetti correlation-energy formula into a functional of the electron density. *Physical Review B*, 37:785.
- MacGownan, D. B. and Surdam, R. C. (1988). Difunctional carboxylic acid anions in oilfield waters. *Organic Geochemistry*, 12:245–259.
- Manning, D. A. C., Gestsdóttir, K., and Rae, E. I. C. (1991). Feldspar dissolution in the presence of organic acid anions under diagenetic conditions: an experimental study. *Advances in Organic Geochemistry*, 19(4–6):483–492.
- Manning, D. A. C., Rae, E. I. C., and Gestsdóttir, K. (1994). Appraisal of the use of experimental and analogue studies in the assessment of the role of organic acid anions in diagenesis. *Marine and Petroleum Geology*, 11(1):10–19.
- Mast, M. A. and Drever, J. I. (1987). The effect of oxalate on the dissolution rates of oligoclase and tremolite. *Geochimica et Cosmochimica Acta*, 51:2559–2568.
- Metropolis, N., Rosenbluth, A. W., Rosenbluth, M. N., and Teller, A. H. (1953). None yet - look up. *Journal of Chemical Physics*, 21:1087.

- Møller, C. and Plesset, M. S. (1934). Note on an approximation treatment for many-electron systems. *Physical Review*, 46:618–622.
- Nagy, K. L., Blum, A. E., and Lasaga, A. C. (1991). Dissolution and precipitation kinetics of kaolinite at 80°C and pH 3: the dependence on solution saturation state. *American Journal of Science*, 291:649–686.
- Paizs, B. and Suhai, S. (1998). Comparative study of BSSE correction methods for DFT and MP2 levels of theory. *Journal of Computational Chemistry*, 19(6):575–584.
- Paritosh, Srolovitz, D. J., Battaile, C. C., Li, X., and E., B. J. (1999). Simulation of faceted film growth in two-dimensions: Microstructure, morphology and texture. *Acta Materialia*, 47(7):2269–2281.
- Paul, G. J. C. and Kebarle, P. (1990). Thermodynamics of the association reaction $OH^- + H_2O = HOHOH^-$ and $CH_3O^- + CH_3OH = CH_3OHOCH_3^-$ in the gas phase. *Journal of Physical Chemistry*, 94:5184–5189.
- Pittman, E. D. and Lewan, M. D., editors (1994). *Organic Acids in Geological Processes*. Springer-Verlag.
- Poulson, S. R., Drever, J. I., and Stillings, L. L. (1997). Aqueous Si-oxalate complexing, oxalate adsorption onto quartz, and the effect of oxalate upon quartz dissolution rates. *Chemical Geology*, 140:1–7.
- Reed, C. L. and Hajash, Jr., A. (1992). Dissolution of granitic sand by pH-buffered carboxylic acids: A flow-through experimental study at 100°C and 345 bars. *The American Association of Petroleum Geologists Bulletin*, 76(9):1402–1416.
- Rovira, A. D. and Davey, C. B. (1974). Biology of the rhizosphere. In Carson, E. W., editor, *The plant root and its environment*, pages 152–204. University Press of Virginia.
- Rovira, A. D. and McDougall, B. M. (1967). Microbiological and biochemical aspects of the rhizosphere. In McLaren, A. D. and Peterson, G. H., editors, *Soil biochemistry*, volume 1, pages 417–463. Dekker.

- Stevenson, F. J. and Fitch, A. (1986). Chemistry of complexations of metal ions with soil solution organics. In (Huang and Schnitzer, 1986), pages 29–58.
- Stevenson, F. J. and Vance, G. F. (1989). Naturally occurring aluminum-organic complexes. In Sposito, G., editor, *The environmental chemistry of aluminum*, pages 117–146. CRC Press.
- Stoessell, R. K. and Pittman, E. D. (1990). Secondary porosity revisited: The chemistry of feldspar dissolution by carboxylic acids and anions. *The American Association of Petroleum Geologists Bulletin*, 74(12):1795–1805.
- Strandh, H., Pettersson, L. G. M., Sjöberg, L., and Wahlgren, U. (1997). Quantum chemical studies of the effects on silicate mineral dissolution rates by adsorption of alkali metals. *Geochimica et Cosmochimica Acta*, 61(13):2577–2587.
- Stumm, W., editor (1987). *Aquatic surface chemistry; chemical processes at the particle-water interface*. John Wiley and Sons.
- Stumm, W., editor (1990). *Aquatic chemical kinetics; reaction rates of processes in natural waters*. John Wiley and Sons.
- Stumm, W., Furrer, G., Wieland, E., and Sinder, B. (1985). The effects of complex-forming ligands on the dissolution of oxides and aluminosilicates. In Drever, J. I., editor, *The chemistry of weathering*, pages 55–74. Reidel.
- Stumm, W. and Wieland, E. (1990). Dissolution of oxide and silicate minerals: rates depend on surface speciation. In (Stumm, 1990), pages 367–400.
- Surdam, R. C., Boese, S. W., and Crossey, L. J. (1984). The chemistry of secondary porosity. In Surdam, R. C., editor, *Clastic diagenesis*, number 37 in Memorandum, pages 317–345. American Association of Petroleum Geologists.
- Surdam, R. C., Crossey, L. J., Hagen, E. S., and Heasler, H. P. (1989). Organic-inorganic interactions and sandstone diagenesis. *American Association of Petroleum Geologists Bulletin*, 73:1–23.

- Surdam, R. C. and MacGowan, D. B. (1987). Oilfield waters and sandstone diagenesis. *Applied Geochemistry*, 2:613–619.
- Tan, K. H. (1986). Degradation of soil minerals by organic acids. In (Huang and Schnitzer, 1986), pages 1–28.
- Tossell, J. A. (1979). Diverse chemical bond types in minerals. In Brown, Jr., G. E. and Clark, J. R., editors, *Proceedings of the symposium on chemistry and physics of minerals*, number 15 in Transactions of the American Crystallographic Association, pages 47–63. American Crystallographic Association.
- Tossell, J. A. and Vaughn, D. J. (1992). *Theoretical geochemistry; applications of quantum mechanics in the earth and mineral sciences*. Oxford University Press.
- Ullman, W. J., Kirchman, D. L., Welch, S. A., and Vandevivere, P. (1996). Laboratory evidence for microbially mediated silicate mineral dissolution in nature. *Chemical Geology*, 132(1–4):11–17.
- Van der Eerden, J. P., Bennema, P., and Cherepanova, T. A. (1978). Survey of Monte-Carlo simulations of crystal-surfaces and crystal-growth. *Progress in Crystal Growth and Characterization of Materials*, 1(3):219–254.
- Vosko, S. H., Wilk, L., and Nusair, M. (1980). Accurate spin-dependent electron liquid correlation energies for local spin density calculations: a critical analysis. *Canadian Journal of Physics*, 58:1200–1211.
- White, A. F. and Brantley, S. L., editors (1995). *Chemical Weathering Rates of Silicate Minerals*, volume 31. Mineralogical Society of America.
- Wieland, E. and Stumm, W. (1992). Dissolution kinetics of kaolinite in acidic aqueous solutions at 25°C. *Geochimica et Cosmochimica Acta*, 56:3339–3355.
- Xiao, Y. and Lasaga, A. C. (1994). Ab initio quantum mechanical studies of the kinetics and mechanisms of silicate dissolution: $H^+(H_3O^+)$ catalysis. *Geochimica et Cosmochimica Acta*, 58(24):5379–5400.

- Xiao, Y. and Lasaga, A. C. (1996). Ab initio quantum mechanical studies of the kinetics and mechanisms of quartz dissolution: OH^- catalysis. *Geochimica et Cosmochimica Acta*, 60(13):2283–2295.
- Yamabe, S. and Minato, T. (1984). Ab initio calculations of the thermochemical data on the $H_3O^+ + H_2O = H_5O_2^+$ gas-phase clustering. *Journal of Chemical Physics*, 80:1576–1578.